Bộ tài liệu Lab & Bài tập

AIClassDocuments · Bản tổng hợp để in · A4 ngang, một cột · 10 bài

Mục lục

STT	Bài	Nội dung	Trang
01	**Assignment 01**	Dictionary - Quan ly sinh vien	2–9
02	**Lab 04**	Dieu kien va vong lap	10–21
03	**Lab 05**	Danh sach sinh vien	22–27
04	**Lab 05-06**	Hoi quy va Gradient Descent	28–36
05	**Lab 07**	Xac suat va Bayes	37–43
06	**Lab 08**	Thong ke va kiem dinh	44–50
07	**Lab 10**	CSV va phan tich diem	51–58
08	**Pretest**	On tap tong hop	59–65
09	**Math 02**	Tap hop, Jaccard va do thi	66–70
10	**Math 03-04**	Ma tran va PCA	71–75

Mỗi bài gồm: đề tiếng Anh, đề tiếng Việt, một chương trình Python, giải thích cách giải và giải thích các định nghĩa. Có thể dùng dấu trang (Bookmarks) trong trình đọc PDF để chuyển nhanh đến từng mục.

Assignment 1 — Quản lý sinh viên bằng Dictionary

1. Đề bài tiếng Anh

ASSIGNMENT 1

Objective

Develop a menu-driven Python program to manage student information using a dictionary data structure and functions.

Requirements

Write a Python program that implements the following features:

1. **Data Structure:** Store students in a dictionary. Each student includes: `id`, `name`, `score`.
2. **Create (Add student):** Input student information. Check for duplicate ID before adding.
3. **Read (Display students):** Show all students in the system.
4. **Update student:** Update student information based on ID.
5. **Delete student:** Delete a student by ID. Require confirmation before deletion.
6. **Validate data:** Ensure valid input (non-empty name, valid score range, correct format).
7. **Check duplicate ID:** Do not allow adding students with existing IDs.
8. **Sorting:** Sort students by score (ascending/descending).
9. **Advanced statistics:** Highest score student, lowest score student, average score, count students by performance level.
10. **Academic classification:** Classify students based on score (e.g., Excellent, Good, Average, Poor).

Đề được chuyển từ `Ass 1.docx` sang Markdown; nội dung và thứ tự yêu cầu được giữ lại.

2. Đề bài tiếng Việt

BÀI TẬP 1 — Quản lý sinh viên bằng Dictionary (từ điển)

Mục tiêu

Xây dựng chương trình Python có menu quản lý thông tin sinh viên, sử dụng cấu trúc dữ liệu **dictionary** và các hàm.

Yêu cầu

Viết chương trình có đầy đủ các chức năng sau:

1. **Cấu trúc dữ liệu:** Lưu sinh viên bằng dictionary (từ điển), mỗi bản ghi có `id`, `name`, `score`.
2. **Thêm sinh viên (Create):** Nhập thông tin, kiểm tra mã ID trùng trước khi thêm.
3. **Hiển thị (Read):** Xuất toàn bộ sinh viên hiện có.
4. **Cập nhật (Update):** Sửa thông tin sinh viên dựa trên ID.
5. **Xóa (Delete):** Xóa theo ID và **phải có xác nhận** trước khi xóa.
6. **Kiểm tra dữ liệu:** Không cho phép tên rỗng, điểm ngoài khoảng hợp lệ hoặc sai định dạng.

7. **Chống trùng mã:** Không thêm được sinh viên có ID đã tồn tại.
8. **Sắp xếp:** Sắp xếp theo điểm tăng dần hoặc giảm dần.
9. **Thống kê nâng cao:** Tìm sinh viên có điểm cao nhất, thấp nhất, tính điểm trung bình và đếm sinh viên theo mức học lực.
10. **Phân loại học lực:** Dựa trên điểm, chia thành Xuất sắc (Excellent), Khá (Good), Trung bình (Average) và Yếu (Poor).

Lưu ý: Đề Word chỉ nêu ví dụ các mức học lực, **không quy định ngưỡng điểm cụ thể**. Các ngưỡng của code tham khảo phải được trình bày như quy ước triển khai, không phải dữ liệu gốc.

3. Code Python có chú thích

```
from collections import Counter
import sys


def validate(student_id, name, score):  # Xác thực mã, tên và điểm trước khi thay đổi dữ liệu.
    student_id = student_id.strip()
    name = name.strip()
    if not student_id or not name:
        raise ValueError("Student ID and name are required.")  # Thông báo lỗi khi dữ liệu không hợp lệ.
    try:
        score = float(score)
    except (ValueError, TypeError) as error:  # Bắt ngoại lệ (exception: lỗi) để thông báo rõ.
        raise ValueError("Score must be numeric.") from error  # Thông báo lỗi khi dữ liệu không hợp lệ.
    if not 0 <= score <= 10:
        raise ValueError("Score must be between 0 and 10.")  # Thông báo lỗi khi dữ liệu không hợp lệ.
    return student_id, name, score


def add(students, student_id, name, score):  # Kiểm tra trùng mã trước khi thêm sinh viên.
    student_id, name, score = validate(student_id, name, score)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    if student_id in students:
        raise ValueError(f"Duplicate student ID: {student_id}")  # Thông báo lỗi khi dữ liệu không hợp lệ.
    students[student_id] = {"name": name, "score": score}  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.


def display(students):
    if not students:
        print("No students stored.")
    for student_id, record in students.items():  # Duyệt lần lượt dữ liệu bằng vòng lặp (loop: lặp).
        print(f"{student_id} | {record['name']} | "
              f"{record['score']:g}/10 | {classify(record['score'])}")  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.


def update(students, student_id, new_name, new_score):
    if student_id not in students:
        raise KeyError(f"Unknown student ID: {student_id}")  # Thông báo lỗi khi dữ liệu không hợp lệ.
    _, name, score = validate(student_id, new_name, new_score)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    students[student_id] = {"name": name, "score": score}  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.


def delete(students, student_id, confirmed=False):
    if student_id not in students:
        raise KeyError(f"Unknown student ID: {student_id}")  # Thông báo lỗi khi dữ liệu không hợp lệ.
```

```
    if not confirmed:
        return False
    del students[student_id]  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    return True


def classify(score):
    if score >= 9:
        return "Excellent"
    if score >= 7:
        return "Good"
    if score >= 5:
        return "Average"
    return "Poor"


def sort_by_score(students, descending=False):
    return sorted(
        students.items(),  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
        key=lambda pair: pair[1]["score"],
        reverse=descending,
    )  # Đóng danh sách, biểu thức hoặc tập giá trị đang tạo.


def statistics(students):  # Thống kê số lượng và phân loại điểm từ dữ liệu hiện có.
    if not students:
        return None
    scores = [item["score"] for item in students.values()]
    counts = Counter(classify(score) for score in scores)
    return {
        "highest": max(scores),  # Lưu trường tên khóa (key: khóa) và kết quả vào từ điển.
        "lowest": min(scores),  # Lưu trường tên khóa (key: khóa) và kết quả vào từ điển.
        "average": sum(scores) / len(scores),  # Lưu trường tên khóa (key: khóa) và kết quả vào từ điển.
        "classification_counts": dict(counts),  # Lưu trường tên khóa (key: khóa) và kết quả vào từ điển.
    }  # Đóng danh sách, biểu thức hoặc tập giá trị đang tạo.


def sample_students():
    scores = [9.5, 8, 7.5, 6, 5.5, 4.5, 7, 9, 8.5, 6.5, 10, 5]
    students = {}
    for number, score in enumerate(scores, start=1):  # Duyệt lần lượt dữ liệu bằng vòng lặp (loop: lặp).
        add(students, f"S{number:03d}", f"Learner {number:02d}", score)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    return students


def run_demo():
    students = sample_students()
    if not (len(students) == 12): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi dữ liệu không đạt.
    add(students, "S013", "New Learner", 8)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    update(students, "S013", "Updated Learner", 8.5)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    if not (delete(students, "S013", confirmed=False) is False): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi
khi dữ liệu không đạt.
    if not ("S013" in students): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi dữ liệu không đạt.
    if not (delete(students, "S013", confirmed=True) is True): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi
dữ liệu không đạt.
    if not ("S013" not in students): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi dữ liệu không đạt.
    if not (len(students) == 12): raise ValueError("Kết quả kiểm tra không đúng")  # Dùng if (nếu) thay lệnh kiểm tra assert trước đây; báo lỗi khi dữ liệu không đạt.
```

```
    print("Ascending scores (first 4):", sort_by_score(students)[:4])
    print("Descending scores (first 4):",
          sort_by_score(students, descending=True)[:4])  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    print("Statistics:", statistics(students))
    print("Student list:")
    display(students)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.


def interactive_menu():  # Điều khiển các thao tác CRUD từ menu.
    students = sample_students()
    menu = "\n1.Display 2.Add 3.Update 4.Delete 5.Sort 6.Statistics 0.Exit"
    while True:  # Lặp khi điều kiện còn đúng, dừng khi sai.
        print(menu)
        action = input("Choose: ").strip()
        try:
            if action == "0":
                break  # Thoát vòng lặp tại vị trí này.
            elif action == "1":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                display(students)  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
            elif action == "2":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                add(students, input("ID: "), input("Name: "),  # Nhận input (dữ liệu nhập) rồi kiểm tra trước khi dùng.
                    input("Score [0,10]: "))  # Nhận input (dữ liệu nhập) rồi kiểm tra trước khi dùng.
            elif action == "3":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                key = input("ID to update: ").strip()
                update(students, key, input("New name: "), input("New score: "))  # Nhận input (dữ liệu nhập) rồi kiểm tra trước khi dùng.
            elif action == "4":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                key = input("ID to delete: ").strip()
                answer = input("Confirm deletion? [y/N]: ").strip().lower()
                if not delete(students, key, confirmed=(answer == "y")):
                    print("Deletion cancelled.")
            elif action == "5":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                descending = input("Descending? [y/N]: ").strip().lower() == "y"
                for key, item in sort_by_score(students, descending):  # Duyệt lần lượt dữ liệu bằng vòng lặp (loop: lặp).
                    print(key, item)
            elif action == "6":  # Kiểm tra trường hợp thay thế sau nhánh trước.
                print(statistics(students))
            else:  # Xử lý trường hợp còn lại.
                print("Unknown menu option.")
        except (ValueError, KeyError) as error:  # Bắt ngoại lệ (exception: lỗi) để thông báo rõ.
            print("Input error:", error)


if __name__ == "__main__":
    if "--demo" in sys.argv:
        run_demo()  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
    else:  # Xử lý trường hợp còn lại.
        interactive_menu()  # Tiếp tục thao tác theo cấu trúc dữ liệu và phép tính trong code đã học.
```

4. Giải thích nội dung và cách giải

Đây là phần diễn giải cho **một chương trình hoàn chỉnh ở Mục 3**. Các ví dụ code ngắn lặp lại trước đây được chuyển thành giải thích bằng lời để khi in không phải theo dõi nhiều phiên bản code.

Câu 1

Dùng `dict` lồng nhau: khóa ngoài là ID duy nhất, giá trị là đối tượng có `name` và `score`. Hàm tạo dữ liệu giả lập để tránh thông tin cá nhân.

Vì sao dùng dictionary lồng nhau? ID là khóa duy nhất, giúp tìm một sinh viên trực tiếp mà không cần duyệt toàn bộ list; dữ liệu đi kèm có thể đặt trong dictionary con. **Các bước:** Khởi tạo từ điển → dùng ID làm khóa → lưu tên, điểm làm giá trị → truy cập khi cần. **Cú pháp:** `students["S01"]` truy cập bản ghi theo ID; `["name"]` truy cập trường tên. ID trùng sẽ **ghi đè** nếu gán trực tiếp, nên chức năng thêm ở Câu 2 phải kiểm tra trước. Tìm theo khóa trung bình O(1), không có nghĩa luôn tuyệt đối cố định.

Điểm cần kiểm tra: dictionary (từ điển) là cấu trúc lưu trữ chính; ID là khóa.

Ví dụ, cách kiểm tra và lỗi dễ gặp: ID là khóa duy nhất của từ điển; mỗi khóa dẫn tới thông tin người học. Dùng tên làm khóa dễ ghi đè khi trùng tên.

Thuật ngữ cần nhớ: Dictionary (từ điển).

Câu 2

Đọc các trường, chuẩn hóa bằng `.strip()`, đổi điểm bằng `float()`, kiểm tra miền 0–10, rồi kiểm tra ID chưa có trước khi thêm.

Vì sao xác thực trước khi thêm? Nếu ID trùng, dictionary có thể ghi đè sinh viên cũ; điểm ngoài 0–10 sẽ làm sai phân loại. **Các bước:** Làm sạch ID/tên → đổi điểm sang `float` → kiểm tra điểm → so ID đã tồn tại chưa → chỉ gán khi hợp lệ. **Giải thích:** `.strip` bỏ khoảng trắng; `float` chuyển chuỗi số; `in` kiểm tra khóa có tồn tại; `not` phủ định; `0 <= score <= 10` kiểm tra cả hai biên; `raise ValueError` báo dữ liệu sai, không âm thầm ghi đè.

Điểm cần kiểm tra: xác thực `id`, `name`, `score` và chặn trùng ID.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Kiểm tra ID đã có hay chưa trước khi gán dữ liệu, rồi xác thực tên, điểm. Nhập sai cần báo lỗi thay vì ghi nhầm.

Thuật ngữ cần nhớ: Create (thêm mới).

Câu 3

Duyệt `.items()` để lấy cặp ID–thông tin và in nội dung. Giữ nguyên thứ tự thêm vào của từ điển.

Vì sao duyệt `items()`? Dictionary ngoài chứa ID còn dictionary trong chứa tên, điểm. Cần lấy cả khóa và dữ liệu cho mỗi bản ghi. **Các bước:** Kiểm tra có sinh viên không → duyệt từng cặp ID/bản ghi → truy cập tên, điểm → in từng dòng. **Giải thích:** `.items()` trả các cặp (khóa, giá trị), `for` lặp lần lượt, `info["score"]` lấy điểm. Không ép dictionary thành list để tìm kiếm nếu chỉ cần hiển thị.

Điểm cần kiểm tra: thao tác CRUD thêm, đọc, sửa và xóa sau khi xác nhận.

Ví dụ, cách kiểm tra và lỗi dễ gặp: items() cho phép lấy cả mã và thông tin; in đủ ID, tên và điểm. Danh sách rỗng cần trả thông báo dễ hiểu.

Thuật ngữ cần nhớ: Read (đọc dữ liệu).

Câu 4

Từ ID đầu vào, tìm đúng khóa; từ chối ID không tồn tại, xác thực giá trị mới trước khi thay thế.

Vì sao phải kiểm tra ID trước khi sửa? Không được vô tình tạo một sinh viên mới khi người dùng định cập nhật một sinh viên đang có. **Các bước:** Nhận ID → tìm bản ghi → xác thực tên/điểm mới → thay trường tương ứng → giữ nguyên khóa ID. **Cú pháp:** `not in` kiểm tra vắng mặt; `KeyError` báo khóa không có; `0 <= ... <= 10` giới hạn điểm; phép gán vào `["score"]` chỉ cập nhật trường điểm, không thay dictionary khác.

Điểm cần kiểm tra: sắp xếp điểm và tổng hợp bằng `Counter`.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Chỉ thay đổi bản ghi có đúng ID; nếu không tìm thấy mã phải xử lý chứ không tự tạo người mới.

Thuật ngữ cần nhớ: Update (cập nhật).

Câu 5

Tìm ID rồi kiểm tra `confirmed`; khi người dùng từ chối thì không đụng đến dữ liệu. Khi đồng ý mới `del students[id]`.

Vì sao cần xác nhận xóa? Xóa một khóa sẽ mất toàn bộ dữ liệu của người đó, nên chỉ xóa khi đúng ID và đã xác nhận. **Các bước:** Kiểm tra ID tồn tại → đọc xác nhận → chỉ gọi xóa khi được đồng ý → kiểm tra kết quả. **Cú pháp:** `and` chỉ đúng khi **cả hai** điều kiện đúng, `del students[id]` xóa khóa; `confirmed` là Boolean, tương ứng tham số `confirmed=False` mặc định trong hàm hoàn chỉnh để không xóa ngoài ý muốn.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Chỉ xóa khi tìm thấy mã và người dùng đã xác nhận; thử hủy xóa để chứng minh dữ liệu giữ nguyên.

Thuật ngữ cần nhớ: Delete (xóa).

Câu 6

Phân biệt kiểm tra kiểu dữ liệu và kiểm tra khoảng giá trị; nêu rõ vì sao `input()` tạo chuỗi, vì sao cần bắt `ValueError`.

Vì sao kiểm tra dữ liệu ở một chỗ? Các chức năng Add và Update cần dùng cùng quy tắc; viết rải rác dễ chỗ cho điểm âm, chỗ lại không. **Các bước:** Kiểm tra ID/tên không rỗng → chuyển điểm sang số → kiểm tra miền 0–10 → báo lỗi rõ ràng nếu sai. **Cú pháp:** `try/except ValueError` bắt lỗi chuyển chuỗi không phải số; `return` trả kết quả Boolean; <= gồm cả 0 và 10. Trong code cuối bài, hàm `validate` còn kiểm tra ID và tên.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: strip() bỏ khoảng trắng ở tên; float() chuyển điểm sang số; kiểm tra miền điểm của code nguồn.

Thuật ngữ cần nhớ: Validate (xác thực dữ liệu).

Câu 7

Dùng cùng hàm `add` trong code nguồn để chống trùng trước thao tác gán: nếu `id in students` thì báo lỗi.

Vì sao ID phải duy nhất? Hai sinh viên có thể trùng tên nhưng vẫn cần hai hồ sơ tách biệt; dùng tên làm khóa có nguy cơ ghi đè. **Các bước:** Chuẩn hóa mã → kiểm tra `in` để xác định đã có → chỉ lưu ID chưa xuất hiện → xử lý trường hợp nhập trùng. **Giải thích:** `in` trên dictionary kiểm tra **khóa**, không kiểm tra các tên nằm bên trong; `already_exists` lưu giá trị đúng/sai; khóa duy nhất là quy tắc của cấu trúc dữ liệu, không phải hàm tính toán.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Không kiểm tra trùng bằng tên, bởi hai sinh viên có thể có tên giống nhau; cần kiểm tra khóa ID.

Thuật ngữ cần nhớ: duplicate ID (trùng mã).

Câu 8

Sắp theo khóa `score`, dùng `reverse=True/False` để đổi chiều; chương trình không làm biến đổi dữ liệu đang lưu.

Vì sao truyền `key` cho `sorted`? Cần xếp theo **điểm**, không phải theo ID hoặc tên. **Các bước:** Lấy các cặp ID và thông tin → lấy trường điểm để so sánh → chọn tăng/giảm → trả danh sách có thứ tự mới. **Giải thích flag:** `key=lambda pair: pair[1]["score"]` lấy điểm từ dictionary ở vị trí thứ hai của cặp; `reverse=True` đảo sang giảm dần, mặc định `False` tăng dần. `sorted` không thay đổi dictionary gốc, kết quả là list các cặp.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: sorted() sắp điểm số tăng/giảm; điểm là float chứ không phải chuỗi chữ, tránh thứ tự sai.

Thuật ngữ cần nhớ: Sorting (sắp xếp).

Câu 9

Lấy danh sách điểm; tính `max`, `min`, trung bình, rồi đếm nhãn phân loại bằng `Counter`. Cần xử lý trường hợp không có sinh viên để tránh chia 0.

Vì sao tính thống kê trước khi phân loại? Trung bình tóm tắt kết quả toàn nhóm, còn đếm theo loại cho thấy có bao nhiêu người ở mỗi mức; không thể lấy số lượng chia cho 0 nếu chưa có sinh viên. **Các bước:** Lấy danh sách điểm → tính tổng và số lượng → tìm trung bình → gọi hàm `classify` cho từng người → đếm nhãn. **Giải thích:** `.values()` lấy bản ghi không cần ID; `sum` cộng điểm, `len` đếm; biểu thức `... if scores else None` tránh chia 0; `Counter` đếm số lần xuất hiện của từng nhãn. `classify` là hàm định nghĩa trong code hoàn chỉnh, không phải hàm Python có sẵn.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Trung bình bằng tổng điểm/số người; cần xử lý n=0 để tránh chia cho 0; Counter đếm mức học lực.

Thuật ngữ cần nhớ: Statistics (thống kê).

Câu 10

Kiểm tra ngưỡng từ cao xuống thấp để không có hai mức cùng đúng; các mốc lấy từ code tham khảo đã tồn tại, **không giả nhận là đề Word quy định**.

Vì sao xét mức điểm từ cao xuống thấp? Nếu xét ngưỡng thấp trước, sinh viên 9.5 điểm sẽ khớp cả mức 5 và 7, dễ bị phân loại sai. Các nhánh `if/elif` sẽ dừng ở nhánh đầu thỏa. **Các bước:** Kiểm tra điểm hợp lệ → so ngưỡng 9 → 7 → 5 → còn lại mức Poor. **Cú pháp:** >= nghĩa là lớn hơn hoặc bằng; `elif` chỉ kiểm tra khi các điều kiện trước sai; `else` là phần còn lại. Vì thứ tự kiểm tra từ cao xuống, không cần viết lặp cả cận dưới/cận trên. Các ngưỡng thuộc quy tắc của code tham khảo.

Điểm cần kiểm tra: phân loại mức học lực từ cao xuống thấp; đây là ngưỡng do lời giải tham khảo chọn.

Ví dụ, cách kiểm tra và lỗi dễ gặp: Các ngưỡng học lực là quy ước của code tham khảo, không mặc định là quy định từ đề Word.

Thuật ngữ cần nhớ: classification (phân loại).

Đọc toàn bộ lời giải theo một luồng

Đầu vào → kiểm tra → lưu → thao tác → báo cáo. Mỗi sinh viên là một bản ghi có khóa định danh riêng. Hàm `validate` kiểm tra mã và tên sau khi loại khoảng trắng, chuyển điểm sang dạng số, từ chối điểm nằm ngoài thang quy định. Các thao tác thêm hoặc cập nhật phải sử dụng chung quy tắc này; nếu kiểm tra ở mỗi nơi theo một cách khác nhau thì dữ liệu dễ không nhất quán.

Vì sao dùng dictionary? Dictionary truy cập sinh viên qua khóa `student_id`; không cần tìm bằng số thứ tự trong danh sách. Khi thêm cần tránh ghi đè khóa có sẵn. Khi sửa nên giữ mã định danh ổn định và chỉ thay trường được cho phép. Khi xóa cần kiểm tra sự tồn tại trước, tránh nhầm rằng một thao tác đã thành công dù không có mã đó.

Lấy số liệu như thế nào? `len(students)` đếm bản ghi; `values()` lấy thông tin từng người; `Counter(...)` đếm số học viên ở từng nhóm kết quả; `sorted(..., key=...)` tạo thứ tự hiển thị mà không buộc sửa dữ liệu gốc. Cần phân biệt `sort` hoặc `sorted` với phép thống kê: sắp xếp không làm thay đổi điểm trung bình.

Khi tự luyện: Thử thêm mã đã tồn tại, tên chỉ chứa khoảng trắng, điểm ngoài khoảng hợp lệ và mã không có khi sửa/xóa. Quan sát từng hàm báo lỗi thế nào và dữ liệu có bị thay đổi trước khi kiểm tra hay không. Đây là trọng tâm của bài quản lý dữ liệu chứ không chỉ là in menu.

5. Giải thích các định nghĩa

Đọc mục này sau khi xem lời giải: mỗi khái niệm được diễn giải bằng lời, nêu mục đích và ví dụ để dễ nhớ, không cần biết trước ký hiệu toán học.

Dictionary — từ điển

Là gì và dùng để làm gì? Lưu thông tin theo cặp khóa và giá trị; dùng để tìm sinh viên trực tiếp bằng mã thay vì đếm từng hàng.

Ví dụ dễ hiểu: Một mục có khóa `S01` và giá trị là tên, điểm của học viên S01.

Key / value — khóa / giá trị

Là gì và dùng để làm gì? Khóa là mã dùng để tìm đúng một mục; giá trị là thông tin được lưu tại mã đó.

Ví dụ dễ hiểu: `students["S01"]` lấy bản ghi S01; thiếu khóa thì Python có thể báo `KeyError`.

CRUD — tạo, đọc, sửa, xóa

Là gì và dùng để làm gì? Bốn thao tác quản lý dữ liệu: Create, Read, Update, Delete. Dùng để quản lý danh sách sinh viên qua menu.

Ví dụ dễ hiểu: Thêm phải kiểm tra mã trùng, còn sửa hoặc xóa phải kiểm tra mã tồn tại.

Validation — kiểm tra dữ liệu hợp lệ

Là gì và dùng để làm gì? Kiểm tra đầu vào trước khi lưu, để tên không rỗng và điểm đúng kiểu, đúng giới hạn.

Ví dụ dễ hiểu: `float("8.5")` chuyển được thành số; `float("abc")` báo lỗi.

Counter — đếm theo nhóm

Là gì và dùng để làm gì? `Counter` đếm số lần mỗi loại xuất hiện; dùng khi cần số học viên theo mức điểm.

Ví dụ dễ hiểu: Nếu mức xếp loại là `["Giỏi", "Khá", "Giỏi"]` thì Giỏi có 2, Khá có 1.

`sorted()` và `key` — sắp xếp theo tiêu chí

Là gì và dùng để làm gì? `sorted()` tạo kết quả có thứ tự; `key` chỉ rõ dùng trường nào để so sánh.

Ví dụ dễ hiểu: Với điểm 8, 6, 9 và sắp giảm dần (`reverse=True`), kết quả là 9, 8, 6.

Exception — ngoại lệ

Là gì và dùng để làm gì? Là lỗi được Python báo trong khi chạy; `try/except` giúp chương trình phản hồi có kiểm soát.

Ví dụ dễ hiểu: `ValueError`: dữ liệu không đổi được sang số; `KeyError`: không có mã cần tìm.

Lab 4 — Câu lệnh điều kiện và vòng lặp

1. Đề bài tiếng Anh

LAB Session 4

PART 1: CONDITIONAL STATEMENT

Exercise 1 — Sorting numbers

Write a program that inputs three real numbers and prints them in ascending order.

Requirement: Use only `if...else` statements.

Example: Input: `8.5 3.2 6.7` → Output: `3.2 6.7 8.5`

Exercise 2 — Quarter of the year

Write a program that inputs a month (1–12) and determines which quarter of the year it belongs to.

Example: Input: `5` → Output: `Quarter 2`

Requirement: Use `if...elif...else` or `match...case`.

Exercise 3 — Days in a month

Write a program that inputs a month. Determine the number of days in that month.

Example: Input: `3` → Output: `31 days`

Exercise 4 — Leap year

Write a program that inputs a year and determines whether it is a leap year.

Leap year rules:

- A year divisible by 400 is a leap year.
- A year divisible by 4 but not divisible by 100 is a leap year.
- Otherwise, it is not a leap year.

Examples: `2024` → `Leap year`; `2023` → `Not a leap year`.

Exercise 5 — Number name (two digits)

Write a program that inputs a two-digit integer and prints its number name.

Examples: `12` → `muoi hai`; `25` → `hai muoi lam`.

Exercise 6 — Number name (three digits)

Write a program that inputs a three-digit integer and prints its number name.

Example: `123` → `mot tram hai muoi ba`.

Exercise 7 — Electricity bill

Write a program that inputs the amount of electricity consumed in kWh and calculates the electricity bill based on the following rates:

Electricity usage	Price
0–50 kWh	1,800 VND/kWh
51–100 kWh	2,000 VND/kWh
101–200 kWh	2,500 VND/kWh
Above 200 kWh	3,000 VND/kWh

Example in original Word: Input `120` → Output `282000 VND`.

> **Ghi chú hiệu đính:** Nếu giá điện được tính **lũy tiến từng bậc**, 120 kWh cho kết quả **240,000 VND**. Ví dụ `282000` trong Word không khớp bảng giá; `SOLUTION.py` tính lũy tiến và có kiểm tra kết quả. Số liệu ví dụ gốc được giữ ở đây để đối chiếu.

PART 2: LOOP

Exercise 8 — Sum from 1 to n

Input a positive integer `n` and calculate the sum of all integers from 1 to `n`.

Example: Input `10` → `Sum = 55`.

Exercise 9 — Count digits

Input a positive integer and count the number of digits.

Example: `123456` → `Number of digits = 6`.

Requirement: Use a `while` loop.

Exercise 10 — Prime number

Input a positive integer and determine whether it is prime.

Example: `17` → `17 is a prime number.`

Exercise 11 — GCD

Input two positive integers and find their Greatest Common Divisor (GCD).

Example: `24 36` → `GCD = 12`.

Exercise 12 — Rectangle pattern

Input integers `m` and `n`, then print a rectangle with `m` rows and `n` columns using `*`.

For `m = 4`, `n = 6`:

```
******
******
******
******
```

Requirement: Use nested loops.

Exercise 13 — Repeated-digit triangle

Input an integer `n` and print a number triangle. For `n = 5`:

```
1
22
333
4444
55555
```

Requirement: Use nested loops.

Exercise 14 — Consecutive-number triangle

Input an integer `n` and print a triangle containing consecutive numbers. For `n = 4`:

```
1
2 3
4 5 6
7 8 9 10
```

Requirement: Use nested loops and a variable to track the current number.

Đề được chuyển từ `LAB Session 4.docx` sang Markdown; các ví dụ được đặt trong khối văn bản để không lỗi định dạng.

2. Đề bài tiếng Việt

LAB SESSION 4 — Câu lệnh điều kiện và vòng lặp

PHẦN 1: CÂU LỆNH ĐIỀU KIỆN

Bài 1 — Sắp xếp ba số Nhập ba số thực và in ra theo thứ tự tăng dần. **Chỉ được dùng `if...else`**, không dùng hàm sắp xếp có sẵn. Ví dụ: `8.5 3.2 6.7` → `3.2 6.7 8.5`.

Bài 2 — Quý trong năm Nhập tháng từ 1–12 rồi xác định thuộc quý nào. Ví dụ: 5 → Quý 2. **Dùng `if...elif...else` hoặc `match...case`**.

Bài 3 — Số ngày trong tháng Nhập tháng và xác định số ngày trong tháng. Ví dụ tháng 3 có 31 ngày. Nếu xét tháng 2 và có năm, cần xét năm nhuận.

Bài 4 — Năm nhuận Nhập năm rồi xác định năm nhuận. Năm chia hết cho 400 là năm nhuận; hoặc chia hết cho 4 nhưng không chia hết cho 100. Các trường hợp khác không nhuận. Ví dụ 2024 nhuận, 2023 không nhuận.

Bài 5 — Đọc số có hai chữ số Nhập số nguyên hai chữ số, in cách đọc số bằng tiếng Việt. Ví dụ: 12 → `muoi hai`; 25 → `hai muoi lam`.

Bài 6 — Đọc số có ba chữ số Nhập số nguyên ba chữ số, in cách đọc bằng tiếng Việt. Ví dụ: 123 → `mot tram hai muoi ba`.

Bài 7 — Tính tiền điện Nhập lượng điện tiêu thụ (kWh), tính hóa đơn từ bảng giá:

Lượng điện tiêu thụ	Đơn giá
0–50 kWh	1.800 VND/kWh
51–100 kWh	2.000 VND/kWh
101–200 kWh	2.500 VND/kWh
Trên 200 kWh	3.000 VND/kWh

Ví dụ trong Word: 120 kWh → 282.000 VND. *Giá trị này được giữ nguyên khi dịch; lưu ý kiểm tra lại với phương pháp tính lũy tiến.*

PHẦN 2: VÒNG LẶP

Bài 8 — Tổng từ 1 đến n Nhập số nguyên dương `n`, tính tổng các số nguyên từ 1 đến `n`. Ví dụ: 10 → 55.

Bài 9 — Đếm chữ số Nhập số nguyên dương, **dùng `while`** đếm số chữ số. Ví dụ 123456 có 6 chữ số.

Bài 10 — Kiểm tra số nguyên tố Nhập số nguyên dương và xác định có phải số nguyên tố không. Ví dụ 17 là số nguyên tố.

Bài 11 — Ước chung lớn nhất Nhập hai số nguyên dương, tính GCD. Ví dụ 24 và 36 → 12.

Bài 12 — Hình chữ nhật dấu sao Nhập `m`, `n`, dùng **vòng lặp lồng nhau** in hình chữ nhật `m` hàng, `n` cột bằng ký tự `*`. Với `m=4,n=6`:

```
******
******
******
******
```

Bài 13 — Tam giác chữ số lặp Nhập `n`, dùng **vòng lặp lồng nhau** in tam giác. Với `n=5`:

```
1
22
333
```

```
4444
55555
```

Bài 14 — Tam giác số liên tiếp Nhập n, dùng **vòng lặp lồng nhau** và biến đếm liên tục. Với n=4:

```
1
2 3
4 5 6
7 8 9 10
```

3. Code Python có chú thích

```
from math import isqrt

def task01_sort_three(a, b, c):  # So sánh và đôi chỗ để sắp xếp ba số.
    if a > b: a, b = b, a
    if b > c: b, c = c, b
    if a > b: a, b = b, a
    return a, b, c

def task02_quarter(month):
    if not 1 <= month <= 12: raise ValueError("Month must be 1..12")
    if month <= 3: return 1
    elif month <= 6: return 2  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
    elif month <= 9: return 3  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
    else: return 4  # Xử lý trường hợp ngược lại hoặc còn lại.

def leap_year(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)

def task03_month_days(month, year=2026):
    days = [31,28,31,30,31,30,31,31,30,31,30,31]
    if not 1 <= month <= 12: raise ValueError("Month must be 1..12")
    return 29 if month == 2 and leap_year(year) else days[month-1]

def task04_is_leap(year):
    if year < 1: raise ValueError("Year must be positive")
    return leap_year(year)

WORDS = ["khong","mot","hai","ba","bon","nam","sau","bay","tam","chin"]

def task05_two_digit(n):
    if not 10 <= n <= 99: raise ValueError("Expected two-digit integer")
    tens, unit = divmod(n,10)
    if tens == 1:
        head = "muoi"
    else:  # Xử lý trường hợp ngược lại hoặc còn lại.
        head = WORDS[tens] + " muoi"
    if unit == 0: return head
    if unit == 5: return head + " lam"
    if unit == 1 and tens > 1: return head + " mot"
    return head + " " + WORDS[unit]

def task06_three_digit(n):
    if not 100 <= n <= 999: raise ValueError("Expected three-digit integer")
```

```
    hundred, rest = divmod(n,100)
    head = WORDS[hundred] + " tram"
    if rest == 0: return head
    if rest < 10:
        return head + " linh " + ("nam" if rest == 5 else WORDS[rest])
    return head + " " + task05_two_digit(rest)

def task07_electricity_bill(kwh):  # Tính lần lượt từng bậc giá, không áp một giá cho toàn bộ điện.
    if kwh < 0: raise ValueError("Usage cannot be negative")
    remaining = float(kwh)
    total = 0.0
    for width, rate in ((50,1800),(50,2000),(100,2500)):
        amount = min(remaining,width)
        total += amount * rate
        remaining -= amount
        if remaining <= 0: return total
    return total + remaining * 3000

def task08_sum_to_n(n):
    if n <= 0: raise ValueError("n must be positive")
    total=0
    for value in range(1,n+1): total += value
    return total

def task09_count_digits(n):
    if n <= 0: raise ValueError("n must be positive")
    count=0
    while n:  # Tiếp tục vòng lặp khi điều kiện còn đúng.
        count += 1
        n //= 10
    return count

def task10_is_prime(n):  # Chỉ thử ước tới căn bậc hai của n.
    if n <= 1: return False
    for divisor in range(2,isqrt(n)+1):
        if n % divisor == 0: return False
    return True

def task11_gcd(a,b):  # Thuật toán Euclid lấy phần dư liên tiếp.
    if a<=0 or b<=0: raise ValueError("Both numbers must be positive")
    while b:  # Tiếp tục vòng lặp khi điều kiện còn đúng.
        a,b=b,a%b
    return a

def task12_rectangle(rows,cols):
    if rows <= 0 or cols <= 0: raise ValueError("Rows/cols must be positive")
    output=[]
    for _ in range(rows):
        line=""
        for _ in range(cols):
            line += "*"
        output.append(line)  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
    return "\n".join(output)

def task13_repeated_triangle(n):
    if n <= 0: raise ValueError("n must be positive")
    output=[]
    for i in range(1,n+1):
```

```
        line=""
        for _ in range(i):
            line += str(i)
        output.append(line)  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
    return "\n".join(output)

def task14_consecutive_triangle(n):  # Vòng lặp lồng nhau tạo từng dòng của hình.
    if n <= 0: raise ValueError("n must be positive")
    rows=[]
    current=1
    for i in range(1,n+1):
        values=[]
        for _ in range(i):
            values.append(str(current))  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
            current += 1
        rows.append(" ".join(values))  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
    return "\n".join(rows)

def main():
    if not (task01_sort_three(8.5,3.2,6.7)==(3.2,6.7,8.5)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task02_quarter(5)==2): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task03_month_days(2,2024)==29): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task04_is_leap(2024) and not task04_is_leap(2023)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task05_two_digit(25)=="hai muoi lam"): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task06_three_digit(123)=="mot tram hai muoi ba"): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task07_electricity_bill(120)==240000): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task08_sum_to_n(10)==55): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task09_count_digits(123456)==6): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task10_is_prime(17) and not task10_is_prime(1)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task11_gcd(24,36)==12): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task12_rectangle(4,6)=="******\n******\n******\n******"): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task13_repeated_triangle(5)=="1\n22\n333\n4444\n55555"): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (task14_consecutive_triangle(4)=="1\n2 3\n4 5 6\n7 8 9 10"): raise ValueError("Kiểm tra dữ liệu không đạt")
    print("PASS: 14 exercises")
    print("Example electricity 120 kWh:",task07_electricity_bill(120),"VND")
    print("Pattern:",task14_consecutive_triangle(4),sep="\n")

if __name__=="__main__":
    main()
```

4. Giải thích nội dung và cách giải

Đây là phần diễn giải cho **một chương trình hoàn chỉnh ở Mục 3**. Các ví dụ code ngắn lặp lại trước đây được chuyển thành giải thích bằng lời để khi in không phải theo dõi nhiều phiên bản code.

Câu 1

So sánh cặp `(a,b)`, `(b,c)`, rồi `(a,b)` và đổi chỗ khi cặp không đúng thứ tự. Đây là cách dùng điều kiện, không gọi `sort()`.

Vì sao so sánh từng cặp? Đề yêu cầu luyện `if/else`, nên không dùng hàm sắp xếp có sẵn. Đưa số nhỏ về đầu bằng các lần so sánh và hoán đổi. **Quy trình:** So a với b → so b với c → so lại a với b. Sau lượt cuối, thứ tự là tăng dần. **Cú pháp:** > so sánh lớn hơn; `a, b = b, a` hoán đổi hai biến mà không cần biến tạm. Không dùng `sorted` vì trái mục đích luyện điều kiện.

Điểm cần kiểm tra: dùng `if/elif/else` đúng các bài điều kiện và so sánh hai số.

Ví dụ và lỗi thường gặp: Cho ba giá trị 8,5; 3,2; 6,7, lần lượt đổi vị trí theo phép so sánh. Cuối cùng cần có 3,2 ≤ 6,7 ≤ 8,5. Không gọi sorted() vì đề yêu cầu luyện điều kiện.

Thuật ngữ cần nhớ: ascending (tăng dần).

Câu 2

Kiểm tra tháng hợp lệ, sau đó xem tháng nằm ở 1–3, 4–6, 7–9 hay 10–12 để trả quý.

Vì sao chia thành bốn mốc? Một năm có 12 tháng và mỗi quý gồm ba tháng; điều kiện theo mốc dễ đọc và đúng yêu cầu luyện nhánh. **Quy trình:** Kiểm tra tháng 1–12 → xét đến 3, đến 6, đến 9 → còn lại quý 4. **Cú pháp:** <= có tính cả mốc; `not` phủ định điều kiện hợp lệ; `if/elif/else` trong code hoàn chỉnh giúp tránh tính mốc sai. Có thể viết `(month - 1) // 3 + 1` nhưng không dùng nó để thay bài luyện điều kiện; `//` là chia lấy phần nguyên.

Điểm cần kiểm tra: quy tắc năm nhuận và số ngày tháng 2.

Ví dụ và lỗi thường gặp: Chia 12 tháng thành bốn nhóm ba tháng; tháng 5 thuộc quý 2. Kiểm tra đầu vào trước vì tháng 0 và 13 không hợp lệ.

Thuật ngữ cần nhớ: quarter (quý).

Câu 3

Dùng danh sách ngày của 12 tháng. Nếu tháng 2 của năm nhuận thì lấy 29, không lấy 28. Mã này tham chiếu thêm `leap_year()` ở mã nguồn đã học.

Vì sao cần biết năm? Tháng 2 có thể có 28 hoặc 29 ngày, nên chỉ nhập tháng chưa đủ nếu muốn xác định chính xác tháng 2; code tham khảo dùng tham số `year=2026` khi không truyền năm. **Quy trình:** Kiểm tra tháng → lấy số ngày mặc định từ danh sách → riêng tháng 2 kiểm tra năm nhuận. **Giải thích:** `days[month - 1]` đổi tháng 1 thành chỉ số 0; `and` yêu cầu cả tháng 2 và năm nhuận; `year=2026` là **tham số mặc định** của hàm cuối bài, không có nghĩa mọi năm là 2026.

Điểm cần kiểm tra: tách chữ số bằng phép chia nguyên `//` và chia dư `%`.

Ví dụ và lỗi thường gặp: Tháng 2 năm 2024 có 29 ngày, năm 2023 có 28 ngày. Nên tách quy tắc năm nhuận thành hàm để dễ kiểm tra.

Thuật ngữ cần nhớ: month (tháng).

Câu 4

Dùng toán tử `%` kiểm tra các trường hợp chia hết; điều kiện năm dương loại đầu vào không hợp lệ.

Vì sao kiểm tra 400, 100 và 4? Năm chia hết cho 4 thường là nhuận, nhưng năm tròn thế kỷ chỉ nhuận khi chia hết cho 400. Bỏ điều kiện 100 sẽ khiến năm 1900 bị xử lý sai. **Quy trình:** Kiểm tra năm dương → xét chia hết 400 → hoặc chia hết 4 nhưng không chia hết 100. **Cú pháp:** `%` lấy số dư; `== 0` là chia hết; `!=` là khác; `and` cần cả hai điều kiện; `or` cho phép một trong hai trường hợp năm nhuận. Ví dụ 2000 đúng, 1900 sai.

Điểm cần kiểm tra: tiền điện lũy tiến: `50×1800 + 50×2000 + 20×2500 = 240000 VND` cho 120 kWh; đề Word ghi 282000, không khớp.

Ví dụ và lỗi thường gặp: Năm 1900 không nhuận, năm 2000 nhuận. Thử hai năm này để phát hiện lỗi chỉ kiểm tra chia hết cho 4.

Thuật ngữ cần nhớ: leap year (năm nhuận).

Câu 5

Tách hàng chục và đơn vị, chọn từ đọc thích hợp trong danh sách từ đã có (`WORDS`). Một số trường hợp đặc biệt như số 25 đọc `lam`.

Vì sao tách hàng chục và hàng đơn vị? Muốn đọc một số hai chữ số phải biết chữ số hàng chục và phần còn lại để ghép từ theo tiếng Việt. **Quy trình:** Kiểm tra 10–99 → chia thành chục và đơn vị → chọn trường hợp 10–19, chục khác 1, các âm đọc đặc biệt → ghép chuỗi. **Cú pháp:** `divmod(n, 10)` trả cùng lúc thương nguyên và số dư, tương đương `n // 10` và `n % 10`. `WORDS[index]` tra từ đọc số; `return` trả chuỗi, không phải phép cộng số.

Điểm cần kiểm tra: vòng lặp `for`/ `while`, nguyên tố đến √n, Euclid `gcd(a,b)=gcd(b,a%b)`.

Ví dụ và lỗi thường gặp: Đối với hai chữ số, dùng //10 lấy hàng chục, %10 lấy hàng đơn vị. Phát âm số 5 ở hàng đơn vị cần theo quy tắc tiếng Việt.

Thuật ngữ cần nhớ: digit (chữ số).

Câu 6

Tách hàng trăm và phần dư. Nếu phần dư nhỏ hơn 10 dùng cách đọc `linh`, còn lại gọi lại hàm đọc hai chữ số; cần dùng `WORDS` và `task05_two_digit` đã được định nghĩa chung.

Vì sao dùng lại bài hai chữ số? Đọc số ba chữ số gồm hàng trăm và phần còn lại; tái sử dụng hàm đọc hai chữ số tránh viết lại toàn bộ quy tắc. **Quy trình:** Kiểm tra 100–999 → tách trăm và phần dư → đọc trăm → nếu dư bằng 0 kết thúc, dư dưới 10 thêm "linh", còn lại gọi hàm hai chữ số. **Cú pháp:** `divmod(n,100)` tách trăm và phần dư; `+` nối chuỗi; `0 < rest < 10` xét khoảng. Trong code hoàn chỉnh, chữ số 5 có quy tắc đọc riêng, và phần dư hai chữ số được chuyển sang `task05_two_digit`.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Chia số nguyên cho 100 tìm trăm; xét phần dư để đọc chục và đơn vị. Ví dụ 105 cần cách đọc 'linh'.

Thuật ngữ cần nhớ: hundreds (hàng trăm).

Câu 7

Không nhân tất cả kWh với mức giá cao nhất. Lần lượt tính 50 kWh ở bậc 1, 50 kWh tiếp theo bậc 2, phần còn lại 20 kWh bậc 3.

Vì sao tính theo các bậc? Giá mỗi kWh thay đổi theo lượng điện tích lũy. Không thể lấy cả 120 kWh nhân đơn giá 2.500 vì 100 kWh đầu có giá thấp hơn. **Quy trình:** Kiểm tra lượng điện không âm → lấy tối đa 50 kWh cho bậc 1 → tối đa 50 cho bậc 2 → tối đa 100 cho bậc 3 → phần còn lại ở bậc 4 → cộng chi phí. **Cú pháp:** `min` lấy mức điện chưa dùng hoặc độ rộng bậc (giá trị nhỏ hơn); `+=` cộng dồn; `-=` trừ phần đã dùng; `break` ngừng khi hết điện. **Lưu ý:** Ví dụ 282.000 VND trong đề Word không khớp bảng giá; code tính đúng giá lũy tiến ra **240.000 VND** cho 120 kWh.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Tính tiền điện theo các bậc *lũy tiến*: 50×1.800 + 50×2.000 + 20×2.500 = 240.000 VND cho 120 kWh. Số 282.000 VND trong Word mâu thuẫn với bảng giá; không âm thầm sửa đề.

Thuật ngữ cần nhớ: electricity bill (hóa đơn điện).

Câu 8

Tạo biến tổng ban đầu bằng 0, sau đó cộng từng giá trị của `range(1,n+1)`.

Vì sao lặp từ 1 đến n? Đề yêu cầu luyện vòng lặp; cộng dồn từng số cho thấy tổng được hình thành như thế nào và dễ kiểm tra bằng ví dụ nhỏ. **Quy trình:** Kiểm tra n dương → đặt tổng bằng 0 → duyệt 1 đến n → cộng mỗi số vào tổng. **Cú pháp:** `range(1, n + 1)` phải cộng 1 ở biên trên vì range **không lấy cận cuối**; `+=` cập nhật tổng. Có thể kiểm tra bằng `n * (n + 1) // 2` nhưng code chính dùng vòng lặp đúng bài.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: range(1,n+1) lấy cả số n; tổng 1..10 phải là 55. Nếu dùng range(1,n), số n sẽ bị bỏ sót.

Thuật ngữ cần nhớ: sum (tổng).

Câu 9

Mỗi lượt dùng phép chia nguyên cho 10 để bỏ chữ số tận cùng, rồi tăng biến đếm; tiếp tục khi số vẫn lớn hơn 0.

Vì sao chia nguyên liên tục? Mỗi lần bỏ chữ số tận cùng làm số ngắn đi một chữ số; đếm số lần cho biết số chữ số. **Quy trình:** Kiểm tra n nguyên dương → đặt bộ đếm 0 → mỗi vòng tăng đếm, bỏ chữ số cuối → dừng khi số bằng 0. **Cú pháp:** `//= 10` là chia lấy phần nguyên rồi gán lại, nên 123 thành 12; `while` lặp khi điều kiện còn đúng; `+=` tăng bộ đếm. Code này không dùng cho n=0 nếu bài quy định n phải dương.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Trong while, mỗi lượt bỏ một chữ số bằng phép //10 và tăng biến đếm. Kiểm tra ví dụ 123456 phải được sáu lượt.

Thuật ngữ cần nhớ: while loop (vòng lặp điều kiện).

Câu 10

Số nguyên tố phải lớn hơn 1. Thử ước số từ 2 đến căn bậc hai nguyên; nếu tìm thấy ước thì không nguyên tố.

Vì sao chỉ thử đến căn bậc hai? Nếu một số có ước khác 1 và chính nó thì sẽ có ít nhất một ước không lớn hơn căn bậc hai; thử xa hơn là không cần. **Quy trình:** Loại n <= 1 → thử các ước nguyên từ 2 đến căn bậc hai của n → gặp ước chia hết thì trả False → không gặp thì trả True. **Cú pháp:** `isqrt(n)` trả căn bậc hai **nguyên** không vượt quá n; `+1` trong range để xét cả biên; `%` lấy dư, `== 0` nghĩa là chia hết; `break` dừng ngay khi tìm thấy ước. Với n=1 phải trả False.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Số nguyên tố phải lớn hơn 1; thử các ước tới căn bậc hai để tránh kiểm tra thừa. Số 17 là số nguyên tố.

Thuật ngữ cần nhớ: prime (nguyên tố).

Câu 11

Thực hiện thuật toán Euclid bằng phần dư; thay cặp số cho đến khi số thứ hai bằng 0.

Vì sao dùng phép dư? Thuật toán Euclid dựa trên thực tế hai số có chung các ước như cặp số gồm số thứ hai và phần dư khi chia số thứ nhất cho số thứ hai. **Quy trình:** Kiểm tra hai số dương → thay cặp (a,b) bằng (b, phần dư) → tiếp tục đến khi b bằng 0 → a lúc đó là ước chung lớn nhất. **Cú pháp:** `%` là chia lấy dư; `a, b = b, a % b` cập nhật đồng thời hai biến; `while` ngừng khi b bằng 0. Không phải phép chia `/`, vì cần số dư nguyên.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Thuật toán Euclid thay (a,b) bằng (b,a%b) cho tới khi b=0; với 24 và 36 kết quả là 12.

Thuật ngữ cần nhớ: GCD (ước chung lớn nhất).

Câu 12

Vòng ngoài tạo hàng, vòng trong nối từng dấu `*`; ghép các hàng bằng xuống dòng.

Vì sao dùng vòng lặp lồng nhau? Hình chữ nhật có số hàng và số cột cố định; vòng ngoài tạo hàng, vòng trong tạo từng dấu sao. **Quy trình:** Kiểm tra hàng/cột dương → với mỗi hàng, nối đúng số dấu sao bằng vòng lặp trong → gom các dòng → ghép bằng ký tự xuống dòng. **Cú pháp:** `_` là biến lặp không cần dùng; `range(rows)` lặp đúng số hàng; `+=` nối chuỗi dấu sao; `"\n".join(lines)` ghép các dòng xuống hàng. Tổng số dấu sao có thể đối chiếu bằng `rows * cols`, nhưng code thực hành vẫn dùng vòng lặp.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Vòng ngoài tạo m hàng, vòng trong tạo n dấu sao mỗi hàng; ví dụ 4×6 phải có 24 ký tự sao.

Thuật ngữ cần nhớ: nested loop (vòng lặp lồng nhau).

Câu 13

Vòng ngoài chọn số hàng `i`, vòng trong lặp lại ký tự `str(i)` đúng `i` lần.

Vì sao số ký tự tăng theo hàng? Tam giác yêu cầu hàng 1 một ký tự, hàng 2 hai ký tự,... nên vòng trong phải phụ thuộc chỉ số hàng hiện tại, không dùng một số cột cố định. **Quy trình:** Duyệt hàng từ 1 đến n → chuyển số hàng thành chuỗi → lặp đúng i lần để ghép ký tự → lưu dòng. **Cú pháp:** `str(i)` chuyển số thành chữ để ghép chuỗi, `range(i)` lặp i lần, `append` đưa dòng vào list. Số ký tự toàn tam giác đối chiếu bằng `n * (n + 1) // 2`.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Dòng thứ i phải lặp chữ số i đúng i lần; dòng thứ 5 là 55555.

Thuật ngữ cần nhớ: triangle (tam giác).

Câu 14

Biến `current` bắt đầu từ 1; vòng trong in số rồi tăng biến này trước khi sang hàng khác.

Vì sao cần biến current? Đề yêu cầu số tăng liên tục qua nhiều hàng, không bắt đầu lại từ 1 mỗi khi sang hàng mới. **Quy trình:** Khởi tạo `current=1` → hàng i tạo đúng i số → sau mỗi số tăng current → ghép số trong hàng bằng dấu cách → ghép các hàng bằng xuống dòng. **Cú pháp:** `str` chuyển số sang chuỗi; `" ".join(values)` ghép với dấu cách; `+=` tăng số sau mỗi lần in; `"\n".join(rows)` xuống hàng. Số cuối cần bằng `n * (n + 1) // 2`: n=4 cho số cuối 10.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Không đặt lại biến đếm khi xuống hàng: với n=4, hàng cuối phải là 7 8 9 10.

Thuật ngữ cần nhớ: consecutive (liên tiếp).

Mối liên hệ giữa 14 yêu cầu

Bài này luyện **chọn nhánh** và **lặp**, không phải 14 công thức rời rạc. Những bài về tháng, năm nhuận, số điện sử dụng `if/elif/else` vì mỗi trường hợp có quy tắc riêng. Các bài tổng dãy, chữ số, tìm ước và dựng hình dùng `for` hoặc `while` vì phải lặp thao tác nhiều lần. Mỗi hàm `taskXX_...` là lời giải cho một yêu cầu; hàm `main` gọi các hàm theo thứ tự để kiểm tra kết quả.

Chi tiết dễ sai: `range(1, n + 1)` mới chạy tới số `n`; `divmod(n, 10)` trả thương và số dư cùng lúc; `n //= 10` bỏ chữ số hàng đơn vị; `a, b = b, a % b` cập nhật cả hai biến để tìm ước chung lớn nhất. Với số nguyên tố, thử các ước từ 2 tới `isqrt(n)` là đủ bởi nếu có ước lớn hơn căn thì sẽ có một ước nhỏ hơn căn.

Bài tiền điện phải tính lũy tiến. Khi số kWh vượt một bậc, chỉ phần vượt mới chịu đơn giá bậc tiếp theo. Đề gốc có ví dụ 120 kWh ra 282.000 đồng nhưng phép tính theo bảng mức giá đã nêu có thể cho 240.000 đồng; cần dựa đúng *bảng đơn giá của đề* và chỉ rõ khác biệt, không điều chỉnh thuật toán để khớp một ví dụ mâu thuẫn.

Tự kiểm tra kết quả: Chạy cả giá trị biên như tháng 0, 12, 13; năm 1900 và 2000; số 0, 1, 2 đối với kiểm tra nguyên tố; và `n=1` đối với hình tam giác. Các lỗi kiểu này thường đến từ quên kiểm tra biên, không phải cú pháp Python.

5. Giải thích các định nghĩa

Đọc mục này sau khi xem lời giải: mỗi khái niệm được diễn giải bằng lời, nêu mục đích và ví dụ để dễ nhớ, không cần biết trước ký hiệu toán học.

Condition — điều kiện

Là gì và dùng để làm gì? `if`, `elif`, `else` chọn phần code chạy tùy điều kiện đúng hay sai.

Ví dụ dễ hiểu: Khi tháng bằng 2, cần kiểm tra năm nhuận trước khi trả số ngày.

Loop — vòng lặp

Là gì và dùng để làm gì? `for` thường duyệt một dãy đã biết; `while` tiếp tục khi điều kiện còn đúng.

Ví dụ dễ hiểu: `range(1, 5)` chỉ tạo 1, 2, 3, 4; muốn tính cả 5 thì dùng `range(1, 6)`.

Modulo `%` và phép chia nguyên `//`

Là gì và dùng để làm gì? `%` lấy phần dư; `//` lấy phần thương nguyên; dùng để kiểm tra chia hết và tách chữ số.

Ví dụ dễ hiểu: 123 % 10 = 3 (chữ số cuối); 123 // 10 = 12 (phần còn lại).

`divmod()` — nhận cả thương và số dư

Là gì và dùng để làm gì? Một lệnh trả về cùng lúc thương nguyên và số dư, giúp tách số hàng chục và đơn vị.

Ví dụ dễ hiểu: `divmod(25, 10)` trả `(2, 5)`: 2 chục và 5 đơn vị.

Prime number — số nguyên tố

Là gì và dùng để làm gì? Số nguyên lớn hơn 1 chỉ chia hết cho 1 và chính nó; chỉ cần thử ước tới căn bậc hai của số đó.

Ví dụ dễ hiểu: Số 29 chỉ cần thử 2, 3, 4, 5; không có ước nào nên là số nguyên tố.

GCD — ước chung lớn nhất

Là gì và dùng để làm gì? Là số lớn nhất chia hết cả hai số; thuật toán Euclid liên tục dùng phép chia lấy dư.

Ví dụ dễ hiểu: Từ (24,18) → (18,6) → (6,0), vậy ước chung lớn nhất là 6.

Progressive pricing — giá lũy tiến

Là gì và dùng để làm gì? Tính riêng phần sản lượng của từng bậc rồi cộng lại; không áp giá cao nhất cho mọi đơn vị.

Ví dụ dễ hiểu: 50 kWh giá 1.000, 20 kWh tiếp theo giá 2.000: tiền bằng 50.000 + 40.000 = 90.000.

Nested loop — vòng lặp lồng nhau

Là gì và dùng để làm gì? Vòng lặp bên trong chạy cho mỗi lần của vòng ngoài, thường dùng để tạo hàng và cột của hình.

Ví dụ dễ hiểu: Để in hàng thứ 3 là 333, vòng trong in số 3 ba lần.

Lab 5 — Quản lý sinh viên bằng List

1. Đề bài tiếng Anh

LAB Session 5

Objective

Develop a console-based Student Management System in Python using a **list** as the main data structure.

Requirements

Write a Python program that implements the following features:

1. **Add a student:** Input student information (ID, name, score). Store the student in a list.
2. **Display student list:** Show all students in the list.
3. **Search student by name:** Input a name. Display matching students.
4. **Delete a student:** Input student ID. Remove the student from the list.
5. **Sort students by score:** Sort the list in ascending or descending order based on score.
6. **Statistics:** Find the student with the highest score; find the student with the lowest score; calculate the average score.
7. **Exit program.**

Đề được chuyển từ `Lab Session 5.docx` sang Markdown.

2. Đề bài tiếng Việt

LAB SESSION 5 — Quản lý sinh viên bằng List (danh sách)

Mục tiêu

Xây dựng hệ thống quản lý sinh viên trên console bằng Python, sử dụng **list** làm cấu trúc lưu trữ chính.

Yêu cầu

Viết chương trình Python có các chức năng:

1. **Thêm sinh viên:** Nhập ID, tên, điểm; lưu vào một danh sách.
2. **Hiển thị danh sách sinh viên:** In toàn bộ sinh viên trong list (danh sách).
3. **Tìm sinh viên theo tên:** Nhập tên, hiển thị sinh viên khớp.
4. **Xóa sinh viên:** Nhập ID, xóa sinh viên tương ứng khỏi danh sách.
5. **Sắp xếp theo điểm:** Sắp xếp danh sách tăng dần hoặc giảm dần theo điểm.
6. **Thống kê:** Tìm sinh viên có điểm cao nhất, thấp nhất và tính điểm trung bình.
7. **Thoát chương trình:** Cho phép người dùng kết thúc menu.

3. Code Python có chú thích

```
import sys

def validate(student_id,name,score):  # Bảo đảm mã, tên và điểm hợp lệ trước khi thao tác.
    sid=student_id.strip()
    name=name.strip()
    if not sid or not name: raise ValueError("ID and name are required")
    try: score=float(score)
    except ValueError as e: raise ValueError("Score must be numeric") from e
    if not 0<=score<=10: raise ValueError("Score must be in 0..10")
    return {"id":sid,"name":name,"score":score}

def add(students,student_id,name,score):
    data=validate(student_id,name,score)
    if any(item["id"]==data["id"] for item in students):
        raise ValueError("Duplicate student ID")  # Chủ động báo lỗi đầu vào thay vì chấp nhận dữ liệu sai.
    students.append(data)  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).

def find_by_name(students,name):  # Tìm theo tên sau khi chuẩn hóa chuỗi.
    query=name.strip().casefold()
    if not query: return []
    return [s for s in students if query in s["name"].casefold()]

def delete_by_id(students,student_id):
    for index,s in enumerate(students):
        if s["id"]==student_id:
            return students.pop(index)
    raise KeyError(student_id)  # Chủ động báo lỗi đầu vào thay vì chấp nhận dữ liệu sai.

def sorted_by_score(students,descending=False):  # Xếp thứ tự theo điểm bằng khóa so sánh.
    return sorted(students,key=lambda s:s["score"],reverse=descending)

def statistics(students):
    if not students: return None
    return {"highest":max(students,key=lambda s:s["score"]),
            "lowest":min(students,key=lambda s:s["score"]),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
            "average":sum(s["score"] for s in students)/len(students)}  # Gắn tên trường (key: khóa) với giá trị trong từ điển.

def show(students):
    for s in students:
        print(f'{s["id"]} | {s["name"]} | {s["score"]:g}/10')

def sample():
    students=[]
    grades=[5,6,7,8,9,10,5.5,6.5,7.5,8.5,9.5,4]
    for i,score in enumerate(grades,1):
        add(students,f"S{i:03}",f"Learner {i:02}",score)
    return students

def run_demo():
    students=sample()
    if not (len(students)==12): raise ValueError("Kiểm tra dữ liệu không đạt")
    add(students,"S013","Learner 13",7)
    if not (len(find_by_name(students,"learner"))==13): raise ValueError("Kiểm tra dữ liệu không đạt")
```

```
    removed=delete_by_id(students,"S013")
    if not (removed["id"]=="S013" and len(students)==12): raise ValueError("Kiểm tra dữ liệu không đạt")
    print("Sorted:",[(s["id"],s["score"]) for s in sorted_by_score(students)])
    print("Statistics:",statistics(students))
    show(students)

def interactive():  # Menu cho phép thực hiện nhiều thao tác liên tiếp.
    students=sample()
    while True:  # Tiếp tục vòng lặp khi điều kiện còn đúng.
        print("\n1.Add 2.Display 3.Search 4.Delete 5.Sort 6.Statistics 0.Exit")
        c=input("Choice: ").strip()
        try:
            if c=="0":break
            elif c=="1": add(students,input("ID: "),input("Name: "),input("Score: "))  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            elif c=="2":show(students)  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            elif c=="3":show(find_by_name(students,input("Name to search: ")))  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            elif c=="4":delete_by_id(students,input("ID: ").strip())  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            elif c=="5":  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
                show(sorted_by_score(students,input("Descending? [y/N] ").lower()=="y"))  # Nhận input (dữ liệu nhập) để xử lý và kiểm tra.
            elif c=="6":print(statistics(students))  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            else:print("Unknown menu option")  # Xử lý trường hợp ngược lại hoặc còn lại.
        except (ValueError,KeyError) as exc:
            print("Error:",exc)
if __name__=="__main__":
    run_demo() if "--demo" in sys.argv else interactive()
```

4. Giải thích nội dung và cách giải

Đây là phần diễn giải cho **một chương trình hoàn chỉnh ở Mục 3**. Các ví dụ code ngắn lặp lại trước đây được chuyển thành giải thích bằng lời để khi in không phải theo dõi nhiều phiên bản code.

Câu 1

Đầu tiên dùng `validate()` từ code nguồn để làm sạch ID/tên và đổi điểm về `float`. Sau đó kiểm tra trùng ID bằng `any` và dùng `append` thêm bản ghi vào list (danh sách). Các ví dụ DEMO ban đầu đưa thông tin sinh viên vào list nhưng chưa thống nhất kiểu dữ liệu, nên bản tham chiếu kiểm tra kỹ hơn.

Vì sao dùng list? Đề yêu cầu lưu sinh viên vào danh sách; mỗi phần tử có ID, tên và điểm, nên một dictionary cho từng người giúp đọc theo tên trường. **Quy trình:** Nhận dữ liệu → bỏ khoảng trắng → đổi điểm thành `float` → kiểm tra điểm trong 0–10 theo quy ước code → tạo dictionary và `append` vào danh sách. **Giải thích:** `{"id": ...}` lưu theo cặp khóa/giá trị; `.append(...)` thêm vào cuối list; `0 <= score <= 10` kiểm tra gồm cả hai biên. Mốc điểm này là ràng buộc của code tham khảo, không phải dữ liệu được ghi rõ trong đề Word.

Điểm cần kiểm tra: list là cấu trúc chứa chính; mỗi phần tử dùng dictionary (từ điển) có ID/tên/điểm.

Ví dụ và lỗi thường gặp: List (danh sách) chứa các bản ghi có mã, tên, điểm. Dùng append() nối bản ghi mới và kiểm tra mã trùng trước khi thêm.

Thuật ngữ cần nhớ: List (danh sách).

Câu 2

Dùng vòng lặp duyệt từng phần tử của list (danh sách) và truy cập các khóa `id`, `name`, `score` của từng phần tử. Không dùng dictionary (từ điển) làm cấu trúc chứa danh sách chính.

Vì sao phải duyệt? List chứa nhiều dictionary, nên cần lấy từng bản ghi mới hiển thị đầy đủ. **Quy trình:** Xác định danh sách có rỗng không → duyệt theo thứ tự đã lưu → in ID, tên, điểm → đối chiếu số bản ghi. **Giải thích:** `len` đếm người, không tính điểm; `for ... in` lấy mỗi dictionary trong list; `student["name"]` là truy cập trường tên. List rỗng khiến vòng lặp không in hàng nào, cần thông báo phù hợp trong chương trình.

Điểm cần kiểm tra: thêm qua `append` và kiểm tra trùng mã nếu code tham khảo có hỗ trợ.

Ví dụ và lỗi thường gặp: Duyệt từng bản ghi trong list (danh sách) và in đủ trường. Nếu rỗng, cần hiển thị thông báo phù hợp.

Thuật ngữ cần nhớ: Display (hiển thị).

Câu 3

Chuẩn hóa tên bằng `.strip().casefold()` rồi lọc phần tử chứa chuỗi tìm kiếm. Khác với DEMO so sánh tên chính xác, cách này thuận tiện với dữ liệu thực tế.

Vì sao tìm không phân biệt chữ hoa? Người dùng có thể nhập tên khác kiểu viết; `casefold` chuẩn hóa chữ khi so sánh, giúp tìm một phần tên thay vì phải trùng nguyên văn. **Quy trình:** Nhận từ khóa → xóa khoảng trắng → chuẩn hóa chữ → duyệt sinh viên → lấy bản ghi có tên chứa từ khóa. **Giải thích:** `.strip()` bỏ khoảng trắng hai đầu; `.casefold()` chuẩn hóa chữ mạnh hơn `.lower()` cho so sánh không phân biệt hoa/thường; `in` kiểm tra chuỗi con; danh sách kết quả vẫn có thể rỗng. Không nhầm `in` với so khớp ID chính xác.

Điểm cần kiểm tra: tìm tên bằng `casefold()`; duyệt và xóa theo ID bằng `enumerate()/pop()`.

Ví dụ và lỗi thường gặp: Tìm bằng casefold() để không phân biệt hoa/thường; dùng chuỗi con nếu code tham khảo có hỗ trợ.

Thuật ngữ cần nhớ: Search (tìm kiếm).

Câu 4

Dùng `enumerate` lấy cặp chỉ số và bản ghi, sau đó `pop(index)` xóa đúng sinh viên có mã cần tìm. Nếu không thấy ID, báo lỗi.

Vì sao xóa theo ID? Tên có thể trùng, còn mã sinh viên được dùng để xác định một người. Phải tìm đúng mã rồi mới xóa. **Quy trình:** Nhận ID → duyệt danh sách đến dòng khớp → xóa dòng đã xác định → dừng vòng lặp → báo không tìm thấy nếu cần. **Giải thích:** `enumerate` trả cặp vị trí và phần tử; `next(..., None)` lấy khớp đầu hoặc `None`; `pop(index)` xóa theo vị trí; `is not None` phân biệt vị trí 0 với không tìm thấy. Việc duyệt có thể cần xem đến `n` người (O(n)).

Điểm cần kiểm tra: sắp xếp bằng `sorted(..., key=...)`; không so sánh điểm như chuỗi.

Ví dụ và lỗi thường gặp: Dùng enumerate() tìm đúng chỉ số và pop() xóa mục được chọn; ID không tồn tại phải báo lỗi.

Thuật ngữ cần nhớ: Delete (xóa).

Câu 5

Dùng `sorted(students, key=..., reverse=...)` để tạo kết quả có thứ tự theo điểm số; không sắp chuỗi điểm thay cho số.

Vì sao phải chọn khóa sắp xếp? List lưu cả ID và tên; gọi `sorted` mà không chỉ rõ sẽ không biết cần xếp theo điểm. **Quy trình:** Chọn khóa điểm → dùng `sorted` tạo danh sách kết quả → dùng tùy chọn tăng/giảm theo yêu cầu → in để so sánh. **Giải thích flag:** `key=lambda s: s["score"]` lấy điểm của mỗi sinh viên làm khóa; `reverse=False` tăng dần, `reverse=True` giảm dần; `sorted` trả list **mới**, không sửa list cũ. Chi phí sắp xếp thường tăng theo khoảng `n * log(n)` chứ không phải toán tử tính điểm.

Điểm cần kiểm tra: điểm trung bình = tổng điểm/số sinh viên; xử lý list (danh sách) rỗng.

Ví dụ và lỗi thường gặp: sorted() sắp theo điểm số đã đổi thành float, không sắp theo chuỗi văn bản. reverse=True đảo chiều.

Thuật ngữ cần nhớ: Sort (sắp xếp).

Câu 6

Với list (danh sách) không rỗng, dùng `max` và `min` theo khóa điểm để tìm toàn bộ bản ghi đứng đầu/cuối, rồi cộng điểm chia số sinh viên.

Vì sao tính trên cả danh sách? Điểm cao nhất, thấp nhất và trung bình phải phản ánh đúng toàn bộ sinh viên hiện có; nếu không có ai thì không thể chia cho 0. **Quy trình:** Kiểm tra list rỗng → trích cột điểm → tìm max/min → cộng điểm và chia số người. **Giải thích:** `max/min` tìm cao nhất/thấp nhất, `sum` cộng điểm, `len` đếm số người, `/` chia thực; `if scores` kiểm tra list có dữ liệu. `None` thể hiện chưa có kết quả hợp lệ, không phải điểm 0.

Điểm cần kiểm tra: menu `while True` thoát khi chọn 0.

Ví dụ và lỗi thường gặp: Trung bình = tổng điểm/số sinh viên; khi không có sinh viên, tránh phép chia cho 0.

Thuật ngữ cần nhớ: Statistics (thống kê).

Câu 7

Dùng vòng `while True` hiển thị menu, chọn tác vụ theo `if/elif`, xử lý lỗi đầu vào, và `break` khi chọn Exit. Đây là chương trình đã có, không tạo menu mới.

Vì sao Exit là một lựa chọn menu? Chương trình console chạy lặp đến khi người dùng yêu cầu dừng, không tự thoát sau một thao tác. **Quy trình:** Nhận lựa chọn → so sánh với mã thoát → dừng vòng lặp, còn lại thực hiện chức năng tương ứng. **Giải thích:** `input(...)` trả chuỗi; == so sánh chuỗi lựa chọn, `break` dùng **bên trong vòng lặp** mới dừng được vòng lặp; `while True` nghĩa là chạy liên tục cho đến khi có lệnh dừng. Không có phép tính toán học cho chức năng Exit.

Điểm cần kiểm tra: menu `while True` thoát khi chọn 0.

Ví dụ và lỗi thường gặp: Menu lặp đến khi chọn Exit. Chế độ --demo cho phép thử các chức năng mà không cần nhập thủ công.

Thuật ngữ cần nhớ: Exit (thoát).

Tại sao dùng List và cách dữ liệu đi qua các hàm

Khác với Assignment sử dụng dictionary làm bộ lưu chính, Lab 05 đặt nhiều bản ghi sinh viên trong một **list**. `append` bổ sung cuối danh sách; `enumerate` đưa ra cả vị trí lẫn giá trị; hàm tìm kiếm thường phải duyệt các bản ghi vì list không tra cứu trực tiếp theo mã như dictionary. Nếu xóa hay sửa theo vị trí, phải tìm đúng phần tử trước rồi mới thay đổi.

Chuẩn hóa chuỗi trước khi so sánh. `strip()` bỏ khoảng trắng ở đầu/cuối, `casefold()` giúp tìm tên không phân biệt chữ hoa chữ thường. Việc tìm theo tên có thể trả nhiều sinh viên trùng tên; không nên mặc nhiên coi tên là mã định danh duy nhất. Khi sửa/xóa, mã sinh viên là điểm đối chiếu an toàn hơn.

Tính toán và sắp xếp: `len(students)` là số người; trung bình lấy tổng điểm chia số bản ghi nếu không rỗng; `max`/`min` lấy điểm cao nhất/thấp nhất; `sorted(students, key=lambda s: s["score"])` sắp theo trường điểm. Tham số `reverse=True` đảo từ thấp lên cao thành cao xuống thấp.

Các trường hợp biên: danh sách rỗng, mã trùng, tên rỗng, điểm không chuyển được sang số và tìm người không tồn tại. Nên đọc `validate` trước rồi mới đọc các thao tác bởi mọi kết quả cuối cùng đều phụ thuộc dữ liệu đầu vào hợp lệ.

5. Giải thích các định nghĩa

Đọc mục này sau khi xem lời giải: mỗi khái niệm được diễn giải bằng lời, nêu mục đích và ví dụ để dễ nhớ, không cần biết trước ký hiệu toán học.

List — danh sách

Là gì và dùng để làm gì? Lưu nhiều phần tử theo thứ tự; chỉ số bắt đầu từ 0. Dùng để quản lý nhiều bản ghi sinh viên.

Ví dụ dễ hiểu: `students[0]` lấy học viên đầu tiên trong danh sách.

Record — bản ghi

Là gì và dùng để làm gì? Một cụm thông tin cùng thuộc một đối tượng, thường biểu diễn bằng dictionary.

Ví dụ dễ hiểu: `{"id": "S01", "name": "An", "score": 8}` là một bản ghi.

`append()` — thêm cuối danh sách

Là gì và dùng để làm gì? Thêm một phần tử vào list hiện tại; bản thân nó không kiểm tra mã trùng.

Ví dụ dễ hiểu: Danh sách có 2 người, sau `append` thêm 1 người thì `len()` bằng 3.

`enumerate()` — duyệt kèm số thứ tự

Là gì và dùng để làm gì? Khi lặp, tạo cả vị trí lẫn phần tử để không cần quản lý biến đếm thủ công.

Ví dụ dễ hiểu: `enumerate(students, 1)` hiển thị số thứ tự bắt đầu từ 1.

`strip()` và `casefold()` — chuẩn hóa chuỗi

Là gì và dùng để làm gì? `strip()` bỏ khoảng trắng thừa, `casefold()` giúp so tên không phân biệt hoa thường.

Ví dụ dễ hiểu: `" An ".strip().casefold()` được `"an"`.

Mean — trung bình cộng

Là gì và dùng để làm gì? Cộng các điểm rồi chia cho số người có điểm; dùng để đánh giá mức điểm chung.

Ví dụ dễ hiểu: Điểm 6, 8, 10 có trung bình (6+8+10)/3 = 8.

`sorted()` / `lambda` — chọn cách xếp

Là gì và dùng để làm gì? `sorted` xếp các phần tử; `lambda` tạo biểu thức ngắn để chỉ ra trường điểm cần so.

Ví dụ dễ hiểu: Với điểm 9, 6, 8, dùng `reverse=True` thì thứ tự là 9, 8, 6.

Lab 5–6 — Hồi quy tuyến tính và Gradient Descent

1. Đề bài tiếng Anh

LAB SESSION 5 & 6

Problem Description

In this project, you will build a simple machine learning model to predict students' exam scores based on their study hours.

You will use Python, NumPy, Matplotlib, and SymPy to explore mathematical concepts used in machine learning, including **partial derivatives, gradients, gradient descent, and the Hessian matrix**.

Dataset

Study Hours (X)	Exam Score (y)
1	35
2	40
3	50
4	55
5	65
6	70
7	80
8	85

Requirements

Task 1: Create and Visualize the Dataset

- Create two NumPy arrays: `X` (students' study hours) and `y` (students' exam scores).
- Use Matplotlib to create a scatter plot of study hours versus exam scores.
- Add an appropriate title, axis labels, and grid.

Task 2: Build a Simple Linear Prediction Model

Use the following model:

Prediction rule: Predicted exam score = (`w` × study hours) + `b`. For example, `w=5`, `b=30`, and 4 hours gives 50 points.

where `w` is the weight, `b` is the bias, and $\hat{y}$ is the predicted exam score.

- Implement a `predict(X, w, b)` function.
- Implement a `compute_loss(y_true, y_pred)` function using Mean Squared Error (MSE):

MSE (Mean Squared Error): For each student, subtract the predicted score from the actual score, square the difference, add all squared differences and divide by the number of students. For actual scores [60,80] and predictions [50,85], MSE = (100 + 25) / 2 = 62.5.

- Initialize `w = 0` and `b = 0`. Calculate/display initial predictions and MSE.

Task 3: Calculate Partial Derivatives and the Hessian

Consider the loss function for one sample:

Loss for one sample: First calculate 2 × `w` + `b` − 5, then square the result. For `w=1`, `b=1`, loss = (2 + 1 − 5) squared = 4.

- Calculate the partial derivative of `L` with respect to `w`.
- Calculate the partial derivative of `L` with respect to `b`.
- Construct the gradient vector.
- Calculate/display the Hessian matrix using SymPy.

Task 4: Implement Gradient Descent

Implement Gradient Descent from scratch to train the model on the dataset above.

For MSE, use:

MSE gradient: For every student, calculate (predicted score − actual score). For the `w` gradient, multiply each difference by their study hours, add and multiply by 2 / number of students. For the `b` gradient, add the differences without the hours factor, then multiply by 2 / number of students.

Update parameters by subtracting the learning rate multiplied by each gradient component.

- Initialize `w = 0`, `b = 0`.
- Set the learning rate to `0.01` and the number of epochs to `1000`.
- Calculate gradients and update `w`, `b` each epoch.
- Record MSE during training.
- Display final `w`, `b`, and MSE.

Task 5: Analyze the Gradient and Hessian of the Full Dataset Loss

Apply the full-dataset loss:

Loss over all students: For each student, calculate actual score − (`w` × study hours + `b`), square this difference, add all squared differences and divide by the number of students. This is MSE using the model parameters.

- Calculate/display the gradient vector and Hessian matrix.
- Evaluate the gradient at the trained `w` and `b`.
- Calculate Hessian eigenvalues using NumPy.
- Briefly explain what gradient and Hessian tell us about optimization.

Chuyển từ `LAB SESSION 5_6.docx`. Các công thức trong Word được diễn giải thành phép tính bằng lời và ví dụ cụ thể để dễ đọc khi in. Dataset gốc có 8 hàng, được giữ nguyên theo đề.

2. Đề bài tiếng Việt

LAB SESSION 5 & 6 — Hồi quy tuyến tính và Gradient (độ dốc của hàm) Descent (thuật toán giảm độ dốc)

Mô tả bài toán

Xây dựng một mô hình học máy đơn giản để dự đoán **điểm thi của sinh viên** dựa vào **số giờ học**. Dùng Python, NumPy (thư viện tính toán mảng), Matplotlib (thư viện vẽ đồ thị) và SymPy (thư viện toán ký hiệu) để tìm hiểu đạo hàm riêng, gradient, thuật toán Gradient Descent và ma trận Hessian (ma trận đạo hàm bậc hai).

Dữ liệu

Số giờ học (X, giờ)	Điểm thi (y, điểm)
1	35
2	40
3	50
4	55
5	65
6	70
7	80
8	85

Yêu cầu

Bài 1 — Tạo và trực quan hóa dữ liệu Tạo hai mảng NumPy (thư viện tính toán mảng) `X` và `y` chứa số giờ học và điểm. Dùng Matplotlib (thư viện vẽ đồ thị) vẽ scatter plot (biểu đồ phân tán) giờ học so với điểm thi; đặt tiêu đề, nhãn hai trục và lưới.

Bài 2 — Xây dựng mô hình dự đoán tuyến tính Sử dụng mô hình **ŷ = wX + b**, trong đó `w` là trọng số, `b` là hệ số chệch, ŷ là điểm dự đoán. Viết `predict(X,w,b)` và `compute_loss(y_true,y_pred)` dùng MSE (trung bình bình phương sai số):

MSE (sai số bình phương trung bình): Với mỗi học viên, lấy điểm thực tế trừ điểm dự đoán, bình phương sai số, cộng các bình phương rồi chia cho số học viên. Ví dụ điểm thật [60,80], dự đoán [50,85] thì MSE = (100 + 25) / 2 = 62,5.

Khởi tạo `w=0`, `b=0`, hiển thị dự đoán đầu tiên và MSE (trung bình bình phương sai số) ban đầu.

Bài 3 — Đạo hàm riêng và Hessian (ma trận đạo hàm bậc hai) Với hàm một mẫu **$L(w,b)=(2w+b-5)^2$**, tính đạo hàm riêng theo `w`, theo `b`, ghép vector gradient (độ dốc của hàm) và tính ma trận Hessian bằng SymPy (thư viện toán ký hiệu).

Bài 4 — Cài đặt Gradient (độ dốc của hàm) Descent (thuật toán giảm độ dốc) Tự huấn luyện mô hình theo MSE (trung bình bình phương sai số) trên dữ liệu đã cho. Dùng **learning rate (tốc độ học) = 0,01**, **epochs (lượt học) = 1000**, khởi tạo `w=b=0`. Mỗi epoch tính gradient:

Gradient để cập nhật `w` và `b`: Với từng người, tính (điểm dự đoán − điểm thật). Gradient theo `w`: nhân từng chênh lệch với số giờ học, cộng hết rồi nhân 2 / số học viên. Gradient theo `b`: cộng các chênh lệch, không nhân số giờ học, rồi nhân 2 / số học viên. Sau đó trừ tốc độ học nhân gradient để cập nhật từng tham số.

Sau đó trừ learning rate nhân gradient để cập nhật hai tham số, ghi lại MSE qua mỗi epoch. Hiển thị `w`, `b`, MSE cuối cùng.

Bài 5 — Gradient (độ dốc của hàm)/Hessian (ma trận đạo hàm bậc hai) cho toàn bộ dữ liệu Với $L(w,b) = (1/n)\ \Sigma_i\ (y_i - wX_i - b)^2$, tính và hiển thị gradient vector, Hessian. Thay hai giá trị `w,b` đã học vào gradient và tính các eigenvalue (trị riêng) của Hessian bằng NumPy (thư viện tính toán mảng). Giải thích gradient và Hessian cho biết điều gì về quá trình tối ưu.

3. Code Python có chú thích

```
import sys
from pathlib import Path
import numpy as np
import sympy as sp

X = np.array([1,2,3,4,5,6,7,8], dtype=float)
Y = np.array([35,40,50,55,65,70,80,85], dtype=float)

def predict(x,w,b):
    return w*x+b

def compute_loss(y_true,y_pred):  # MSE là trung bình bình phương sai số dự đoán.
    return float(np.mean((y_true-y_pred)**2))

def task01_dataset():
    if not (X.shape == Y.shape == (8,)): raise ValueError("Kiểm tra dữ liệu không đạt")
    return {"hours":X.tolist(),"scores":Y.tolist()}

def task02_initial_model():
    guesses = predict(X,0.0,0.0)
    initial_loss = compute_loss(Y,guesses)
    return guesses,initial_loss

def task03_symbolic_single():  # Dùng SymPy để tìm đạo hàm, không đoán gradient.
    w,b=sp.symbols("w b",real=True)
    loss=(2*w+b-5)**2
    variables=(w,b)
    gradient=sp.Matrix([sp.diff(loss,v) for v in variables])
    hessian=sp.hessian(loss,variables)
    return loss,gradient,hessian

def task04_train(learning_rate=0.01,epochs=1000):  # Lặp cập nhật w và b theo chiều ngược gradient.
    w=b=0.0
    history=[]
    n=X.size
    for _ in range(epochs):
        residual=predict(X,w,b)-Y
        dw=(2/n)*np.dot(X,residual)
        db=(2/n)*residual.sum()  # Đếm hoặc cộng phần tử để tính tổng/tỷ lệ.
        w-=learning_rate*dw
        b-=learning_rate*db
        history.append(compute_loss(Y,predict(X,w,b)))  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
    return w,b,np.array(history)

def task05_symbolic_full(trained_w,trained_b):  # So sánh gradient và Hessian khi mô hình có hai tham số.
    w,b=sp.symbols("w b",real=True)
    values=[sp.Integer(int(x)) for x in X]
    targets=[sp.Integer(int(y)) for y in Y]
    n=len(values)
```

```
    full_loss=sum((yi-w*xi-b)**2 for xi,yi in zip(values,targets))/n
    gradient=sp.Matrix([sp.diff(full_loss,w),sp.diff(full_loss,b)])
    hessian=sp.hessian(full_loss,(w,b))
    gradient_at=gradient.subs({w:trained_w,b:trained_b}).evalf()
    eigvals=np.linalg.eigvalsh(np.asarray(hessian,dtype=float))
    return gradient,hessian,gradient_at,eigvals

def save_plots(w,b,losses):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    root=Path(__file__).resolve().parent/"outputs"
    root.mkdir(exist_ok=True)
    fig,ax=plt.subplots()
    ax.scatter(X,Y,label="Observed scores")
    ax.plot(X,predict(X,w,b),label="Fitted prediction")
    ax.set(xlabel="Study hours",ylabel="Exam score (points)",title="Hours vs. exam scores")
    ax.grid(True)
    ax.legend()
    fig.savefig(root/"hours_vs_scores.png",dpi=140,bbox_inches="tight")
    plt.close(fig)  # Đóng biểu đồ sau khi dùng để giải phóng bộ nhớ.
    fig,ax=plt.subplots()
    ax.plot(np.arange(1,len(losses)+1),losses)
    ax.set(xlabel="Epoch",ylabel="MSE (squared score units)",title="Gradient Descent Training")
    ax.grid(True)
    fig.savefig(root/"loss_curve.png",dpi=140,bbox_inches="tight")
    plt.close(fig)  # Đóng biểu đồ sau khi dùng để giải phóng bộ nhớ.
    print("Wrote",root/"hours_vs_scores.png","and",root/"loss_curve.png")

def main():
    task01_dataset()
    initial_preds,initial_mse=task02_initial_model()
    example_loss,example_grad,example_hessian=task03_symbolic_single()
    w,b,history=task04_train(learning_rate=0.01,epochs=1000)
    full_grad,full_hessian,at_final,eigenvalues=task05_symbolic_full(w,b)
    final_mse=compute_loss(Y,predict(X,w,b))
    if not (history.shape==(1000,)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (final_mse < initial_mse): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (all(eigenvalues > 0)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.isclose(final_mse,history[-1])): raise ValueError("Kiểm tra dữ liệu không đạt")
    print("Initial predictions:",initial_preds)
    print("Initial MSE:",round(initial_mse,3))
    print("Single-sample symbolic loss:",example_loss)
    print("Single-sample gradient:",example_grad,"Hessian:",example_hessian)
    print(f"Trained w={w:.6f}, b={b:.6f}, final MSE={final_mse:.6f}")
    print("Full gradient:",full_grad)
    print("Full Hessian:",full_hessian)
    print("Gradient evaluated at trained parameters:",at_final)
    print("Hessian eigenvalues:",eigenvalues)
    if "--no-plot" not in sys.argv:
        save_plots(w,b,history)
if __name__=="__main__":
    main()
```

4. Giải thích nội dung và cách giải

Đây là phần diễn giải cho **một chương trình hoàn chỉnh ở Mục 3**. Các ví dụ code ngắn lặp lại trước đây được chuyển thành giải thích bằng lời để khi in không phải theo dõi nhiều phiên bản code.

Câu 1

Mô hình dữ liệu: `X=[1,2,3,4,5,6,7,8]` giờ; `y=[35,40,50,55,65,70,80,85]` điểm. Biểu đồ scatter vẽ `X` theo trục ngang và `y` theo trục đứng.

Vì sao cần nhìn dữ liệu trước? Nếu giờ học và điểm thi không có xu hướng liên quan, mô hình đường thẳng có thể không phù hợp. Vẽ các điểm trước giúp người học hiểu dữ liệu, phát hiện giá trị bất thường và hiểu trục nào là đầu vào.

Quy trình: (1) Tạo hai mảng `X` (giờ) và `Y` (điểm). (2) Kiểm tra chúng dài bằng nhau, vì mỗi giờ phải đi cùng một điểm. (3) Vẽ scatter (biểu đồ phân tán), gắn nhãn và lưới. **Cú pháp và tham số:** `dtype=float` chọn số thực; `plt.subplots()` trả một figure (khung hình) và axes (vùng vẽ); `ax.scatter(x, y)` vẽ cặp điểm; `xlabel`, `ylabel` là nhãn trục, `title` là tiêu đề; `grid(True)` bật lưới. Để chạy riêng đoạn minh họa cần nạp `import matplotlib.pyplot as plt`; code cuối bài chỉ vẽ khi không truyền `--no-plot`.

Kiến thức/công thức trọng tâm: giữ nguyên 8 dòng dữ liệu X=1..8 và điểm tương ứng; vẽ scatter và đường hồi quy theo giờ/điểm.

Ví dụ và lỗi thường gặp: Giữ nguyên tám cặp giờ học và điểm từ đề. Trên biểu đồ, trục ngang đo giờ học, trục dọc đo điểm.

Thuật ngữ cần nhớ: Dataset (bộ dữ liệu).

Câu 2

`predict()` nhân từng số giờ với `w`, cộng `b`; `compute_loss()` tính từng hiệu điểm thật-dự đoán, bình phương, lấy trung bình. Khởi tạo cả hai bằng 0 để có đường chuẩn trước khi học.

Vì sao dùng MSE? Mô hình dự đoán bằng đường thẳng cần một cách đo mức sai khác so với điểm thật. Bình phương sai số làm số âm và số dương đều tăng loss; lấy trung bình giúp so sánh qua nhiều dữ liệu.

Quy trình: (1) Đặt tham số hệ số `w` và độ dịch `b`. (2) Tính điểm dự đoán theo giờ học. (3) Lấy điểm thật trừ điểm dự đoán. (4) Bình phương từng sai số rồi lấy trung bình. (5) Đối chiếu mô hình ban đầu `w=0`, `b=0`. **Cú pháp:** `w * X` nhân mọi phần tử của mảng giờ với cùng hệ số, `b` cộng độ dịch vào từng kết quả; `** 2` là bình phương (không dùng `^`); `np.mean` lấy trung bình, cho MSE có đơn vị điểm bình phương. `w=0` và `b=0` chỉ là cách khởi tạo, chưa phải giá trị tốt nhất.

Kiến thức/công thức trọng tâm: MSE = $(1/n)\sum(y-\hat{y})^2$; $\hat{y}=wX+b$; đơn vị MSE là điểm^2.

Ví dụ và lỗi thường gặp: Điểm dự đoán là $\hat{y}=wX+b$; MSE (trung bình bình phương sai số) lấy trung bình của bình phương sai khác. Khi w=b=0, điểm dự đoán đều bằng 0, tạo mốc trước huấn luyện.

Thuật ngữ cần nhớ: Prediction (dự đoán).

Câu 3

Dùng `sp.symbols` tạo biến biểu thức, `sp.diff` lấy đạo hàm, `sp.hessian` trả ma trận đạo hàm bậc hai. Kiểm chứng: gradient (độ dốc của hàm) của biểu thức mẫu có thành phần theo w và b lần lượt bằng `4(2w+b−5)` và `2(2w+b−5)`.

Vì sao tính đạo hàm riêng và Hessian? Hàm loss của một điểm phụ thuộc cả `w` và `b`. Gradient cho biết loss thay đổi theo từng tham số; Hessian cho biết độ cong, giúp giải thích vì sao có hướng thay đổi nhanh/chậm.

Quy trình: (1) Khai báo hai biến ký hiệu. (2) Xây biểu thức loss với đầu vào 2 và điểm thật 5. (3) Lấy đạo hàm theo mỗi biến. (4) Gom đạo hàm thành vector và tính ma trận đạo hàm bậc hai. **Hàm và flag:** `sp.symbols(..., real=True)` khai báo biến thực; `sp.diff(loss, w)` lấy đạo hàm riêng theo `w`; `sp.Matrix` tạo vector cột; `sp.hessian(loss, (w,b))` trả ma trận theo đúng thứ tự biến. `real=True` là giả định số thực trong SymPy. Ở một điểm dữ liệu này, Hessian có một hướng độ cong bằng 0; không kết luận loss có cực tiểu duy nhất từ một điểm.

Kiến thức/công thức trọng tâm: đạo hàm riêng SymPy (thư viện toán ký hiệu) `diff`; Hessian (ma trận đạo hàm bậc hai) ví dụ một điểm $L=(2w+b-5)^2$ có gradient $[4(2w+b-5),2(2w+b-5)]$, Hessian $[[8,4],[4,2]]$.

Ví dụ và lỗi thường gặp: Gradient (độ dốc của hàm) gồm đạo hàm riêng bậc nhất theo w và b; Hessian (ma trận đạo hàm bậc hai) là ma trận đạo hàm bậc hai. Phân biệt loss (hàm mất mát) một điểm ở Bài 3 với loss toàn bộ tập dữ liệu.

Thuật ngữ cần nhớ: Partial derivatives (đạo hàm riêng).

Câu 4

Mỗi vòng dự đoán điểm, lấy phần dư `predict(X,w,b)-Y`, tính gradient (độ dốc của hàm) `dw,db`, trừ learning_rate nhân gradient, lưu MSE (trung bình bình phương sai số) mới; không tạo thuật toán khác với script tham khảo.

Vì sao cập nhật nhiều bước? Dự đoán ban đầu chưa khớp dữ liệu. Gradient cho biết cần điều chỉnh `w` và `b` về hướng nào để giảm MSE, còn tốc độ học quyết định mỗi lần điều chỉnh bao nhiêu.

Quy trình: (1) Khởi tạo `w=b=0`. (2) Tính sai số dự đoán với toàn bộ giờ học. (3) Tính gradient đối với `w` bằng tích vô hướng với `X`, gradient của `b` bằng tổng sai số. (4) Trừ tốc độ học nhân gradient khỏi mỗi tham số. (5) Lặp đúng số vòng, ghi lịch sử MSE để so sánh. **Giải thích tham số/toán tử:** `learning_rate=0.01` là kích cỡ bước; `epochs=1000` là số lần lặp; `range(epochs)` tạo đúng 1.000 lượt; `_` là biến lặp không dùng; `np.dot` nhân và cộng các cặp phần tử; `X.size` là số quan sát; `-=` viết ngắn cho phép trừ rồi cập nhật. Tốc độ học quá lớn có thể làm loss tăng; kiểm tra lịch sử loss để phát hiện.

Kiến thức/công thức trọng tâm: mỗi epoch cập nhật w,b trừ 0.01×gradient; tính lại MSE, lưu đủ 1000 epoch; gradient $w=(2/n)\,\Sigma\, X(wX+b-y)$, $b=(2/n)\,\Sigma(wX+b-y)$.

Ví dụ và lỗi thường gặp: Mỗi epoch (lượt cập nhật) tính phần dư, hai gradient (độ dốc của hàm) dw, db, rồi trừ learning rate (tốc độ học) nhân gradient. Giữ nguyên 1000 epoch và tốc độ học 0,01.

Thuật ngữ cần nhớ: Gradient Descent (thuật toán giảm độ dốc).

Câu 5

Biểu thức loss (hàm mất mát) tổng được tạo từ 8 giá trị gốc trong `X,Y`; SymPy (thư viện toán ký hiệu) tự tính gradient (độ dốc của hàm) và Hessian (ma trận đạo hàm bậc hai). `subs` thay tham số học được, `np.linalg.eigvalsh` cho trị riêng ma trận đối xứng. Không lẫn với Hessian một điểm ở bài 3.

Vì sao tính trên toàn bộ dữ liệu? Đạo hàm từ một điểm ở câu trước không phản ánh đủ tám cặp giờ/điểm. Ở đây phải lập loss trung bình của **tất cả tám điểm**, rồi xem gradient còn lớn hay nhỏ tại tham số đã học và Hessian mô tả độ cong.

Quy trình: (1) Tạo loss là trung bình bình phương sai số của tám điểm. (2) Lấy đạo hàm theo `w` và `b` bằng SymPy. (3) Thay tham số cuối vào gradient để kiểm tra mức gần tối ưu. (4) Tính Hessian và trị riêng để thấy độ cong theo các hướng khác nhau. **Giải thích tham số và cú pháp:** `zip(X,Y)` ghép từng giờ học với đúng điểm; `sum` cộng tám bình phương; `len(X)` chia thành trung bình; `.subs({w: ..., b: ...})` thay tham số vào biểu thức; `.evalf()` tính số gần đúng. `np.asarray(..., dtype=float)` đổi SymPy sang mảng số; `np.linalg.eigvalsh` tính trị riêng của **ma trận đối xứng** như Hessian trong bài. `trained_w` và `trained_b` là kết quả sau bước huấn luyện ở Câu 4.

Kiến thức/công thức trọng tâm: hàm loss đầy đủ 8 mẫu khác Hessian một mẫu; tính eigenvalue (trị riêng) bằng `eigvalsh` vì Hessian đối xứng; so gradient ở nghiệm đã học.

Ví dụ và lỗi thường gặp: Gradient (độ dốc của hàm) cho biết hướng tăng loss (hàm mất mát); Hessian (ma trận đạo hàm bậc hai) và các trị riêng cho biết độ cong. Đánh giá tại tham số đã học chứ không dùng w,b ngẫu nhiên.

Thuật ngữ cần nhớ: Hessian (ma trận đạo hàm bậc hai).

Hiểu một lượt huấn luyện Gradient Descent

Đầu vào là các cặp `X` và `Y`, trong đó `X` là biến mô tả và `Y` là giá trị cần dự đoán. Mô hình tuyến tính dự đoán $\hat{y} = w \times x + b$; `w` điều khiển độ dốc, `b` là điểm cắt trục tung. Hàm `compute_loss` đo **MSE** bằng trung bình bình phương phần chênh giữa dự đoán và giá trị thật; MSE càng nhỏ thì mô hình càng khớp *trên bộ dữ liệu đang xét*, chưa đủ bảo đảm khái quát tốt.

Chu trình cập nhật gồm: tính dự đoán từ `w,b` hiện tại; tính sai số/loss; tìm đạo hàm riêng `dw, db`; sau đó cập nhật `w -= learning_rate × dw`, `b -= learning_rate × db`. Đây là một bước; `epochs` là số vòng lặp huấn luyện, còn `learning_rate` quyết định độ dài mỗi bước. Tốc độ học quá lớn có thể vượt qua điểm cực tiểu; quá nhỏ có thể cần rất nhiều vòng.

Vai trò của SymPy: `sp.symbols` khai báo biến ký hiệu, `sp.diff` lấy đạo hàm, `sp.Matrix` gom gradient thành vector, còn `hessian` mô tả độ cong thông qua đạo hàm bậc hai. Đạo hàm được tính tượng trưng là cơ sở để đối chiếu hướng và kích thước bước cập nhật trong code NumPy.

Khi đọc đồ thị: trục ngang là số vòng huấn luyện, trục dọc là MSE; đường giảm thường cho thấy thuật toán đang cải thiện trên dữ liệu luyện. Nếu loss tăng hoặc dao động, thử xem lại `learning_rate`, dữ liệu đầu vào và việc tính gradient trước khi tăng `epochs`.

5. Giải thích các định nghĩa

Mỗi thuật ngữ dưới đây được giải thích bằng ngôn ngữ thường ngày rồi minh họa bằng một phép tính nhỏ, giúp bạn hiểu bản chất trước khi nhớ tên hoặc ký hiệu.

X, y và ŷ

Là gì và dùng để làm gì? X là số giờ học, y là điểm thi thật, ŷ (y mũ) là điểm mô hình dự đoán.

Ví dụ: Học 2 giờ, dự đoán 45 nhưng thực tế đạt 50 điểm.

w và b

Là gì và dùng để làm gì? w là hệ số cho biết dự đoán tăng bao nhiêu khi thêm 1 giờ học; b là giá trị cộng thêm trong mô hình.

Ví dụ: w=5, b=30, học 4 giờ → 5 × 4 + 30 = 50 điểm.

MSE — sai số bình phương trung bình

Là gì và dùng để làm gì? Lấy điểm thật trừ dự đoán cho từng người, bình phương, cộng tất cả rồi chia số người. Nhờ bình phương, sai số âm không triệt tiêu sai số dương.

Ví dụ: Điểm thật [60,80], dự đoán [50,85] → MSE = (100+25)/2 = 62,5 điểm bình phương.

n, i, Σ và ký hiệu ²

Là gì và dùng để làm gì? n là số dòng; i là thứ tự dòng; Σ nghĩa là cộng tất cả; ² nghĩa là nhân một số với chính nó. Ký hiệu y_i chỉ điểm thật dòng i, ŷ_i chỉ điểm dự đoán dòng i.

Ví dụ: Nếu n=2, chỉ cộng hai sai số bình phương rồi chia cho 2.

Loss — hàm mất mát

Là gì và dùng để làm gì? Đo độ lệch giữa dự đoán và thực tế, thường dùng MSE trong bài này.

Ví dụ: MSE giảm từ 100 xuống 40 nghĩa là sai số bình phương trung bình của tập đang xét giảm.

Gradient — độ dốc của loss

Là gì và dùng để làm gì? Cho biết loss thay đổi theo hướng tăng của w hay b. Gradient chỉ hướng tăng loss nhanh, nên muốn giảm thường lấy giá trị cũ trừ một bước theo gradient.

Ví dụ: Gradient của w là +4, bước học 0,1 thì trừ 0,4 khỏi w.

Learning rate và epoch

Là gì và dùng để làm gì? Learning rate là kích thước bước chỉnh các tham số; epoch là số lượt huấn luyện theo chương trình.

Ví dụ: w=2, gradient=−4, learning rate=0,1 → w mới = 2−0,1×(−4)=2,4.

Partial derivative — đạo hàm riêng

Là gì và dùng để làm gì? Đo loss đổi thế nào khi chỉ thay một tham số và tạm giữ tham số khác cố định.

Ví dụ: Với $L=(2w+b-5)^2$ tại w=1, b=1: đạo hàm theo w=−8, theo b=−4.

Hessian — ma trận đạo hàm bậc hai

Là gì và dùng để làm gì? Bảng gồm đạo hàm bậc hai theo các tham số, giúp mô tả độ cong của hàm loss.

Ví dụ: Với $L=(2w+b-5)^2$, Hessian theo thứ tự (w,b) là [[8,4],[4,2]].

Lab 7 — Xác suất

1. Đề bài tiếng Anh

LAB SESSION 7

Problem Description

Probability, Simulation, Conditional Probability, Bayes' Theorem and Probability Distributions.

Requirements

Task 1 – Sample Space and Basic Probability

A box contains 10 cards numbered from 1 to 10. One card is randomly selected.

Define the sample space and events:

- A: The selected number is even.
- B: The selected number is greater than 6.

Calculate theoretical probabilities `P(A)`, `P(B)`, `P(A ∩ B)`, and `P(A ∪ B)`.

Task 2 – Probability Simulation

A basketball player has a 70% probability of successfully making a free throw.

- Use NumPy to simulate **5,000 free throws** and calculate experimental probability of success.
- Simulate two independent players with success probabilities 0.7 and 0.6.
- Calculate probability both succeed and probability at least one succeeds.
- Compare simulation with theoretical probabilities.

Task 3 – Conditional Probability

A day is randomly selected from a seven-day week.

- Event A: The selected day is a weekday (Monday to Friday).
- Event B: The selected day is Monday, Tuesday, Wednesday or Thursday.

Calculate theoretical conditional probability `P(A | B)`. Simulate 5,000 selections using Python and NumPy, then compare the experimental result.

Task 4 – Bayes' Theorem

An email filtering system has these characteristics:

- 30% of emails are spam.
- 90% of spam emails are correctly identified as spam.

- 5% of non-spam emails are incorrectly classified as spam.

Calculate overall probability an email is classified as spam, then use Bayes' theorem to calculate probability an email is actually spam when classified as spam.

Task 5 – Probability Distributions

A customer service team receives 8 independent requests. Each has a 75% probability of successful resolution.

- Use `scipy.stats.binom.pmf()` to calculate probability of **exactly 6** resolved requests and **at least 6** requests.
- Plot the Binomial probability distribution using Matplotlib.

Chuyển từ `LAB SESSION 7.docx` sang Markdown.

2. Đề bài tiếng Việt

LAB SESSION 7 — Xác suất

Một chuỗi bài thực hành về **xác suất cơ bản, mô phỏng, xác suất có điều kiện, định lý Bayes và phân phối xác suất**.

Toàn bộ yêu cầu

Bài 1

Một hộp có 10 thẻ đánh số từ 1 đến 10, lấy ngẫu nhiên một thẻ. Xác định không gian mẫu, biến cố A (số chẵn), B (số lớn hơn 6). Tính xác suất của A, B, đồng thời A và B, và ít nhất A hoặc B.

Bài 2

Một người chơi bóng rổ ném phạt thành công với xác suất 70%. Dùng NumPy (thư viện tính toán mảng) mô phỏng **5.000** lần để tính tỉ lệ thực nghiệm. Tiếp theo mô phỏng hai người độc lập với xác suất thành công 0,7 và 0,6; tính cả hai thành công và ít nhất một người thành công; so sánh với lý thuyết.

Bài 3

Chọn ngẫu nhiên một ngày trong tuần gồm 7 ngày. A là ngày thường từ thứ Hai đến thứ Sáu, B là thứ Hai đến thứ Năm. Tính `P(A|B)`, mô phỏng 5.000 lần bằng Python/NumPy (thư viện tính toán mảng) và so sánh với kết quả lý thuyết.

Bài 4

Hệ thống lọc email có 30% email spam, phát hiện đúng 90% spam, và báo nhầm 5% email không spam thành spam. Tính xác suất một email được đánh dấu spam và xác suất email thực sự spam khi đã bị đánh dấu.

Bài 5

Nhóm hỗ trợ nhận 8 yêu cầu độc lập, mỗi yêu cầu có khả năng giải quyết thành công 75%. Dùng `scipy.stats.binom.pmf()` tính xác suất giải quyết **đúng 6** và **ít nhất 6**, vẽ phân phối nhị thức bằng Matplotlib (thư viện vẽ đồ thị).

3. Code Python có chú thích

```
from pathlib import Path
import sys
import numpy as np
from scipy.stats import binom
```

```
SEED=2026
TRIALS=5000

def task01_cards():  # Dùng giao/hợp tập hợp để đếm biến cố.
    space=set(range(1,11))
    a={x for x in space if x%2==0}
    b={x for x in space if x>6}
    return {
        "space":sorted(space),"A":sorted(a),"B":sorted(b),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "P(A)":len(a)/len(space),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "P(B)":len(b)/len(space),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "P(A and B)":len(a&b)/len(space),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "P(A or B)":len(a|b)/len(space),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
    }  # Đóng danh sách, mảng hoặc nhóm tham số đang xây dựng.

def task02_free_throws():  # Mô phỏng phép thử ngẫu nhiên với seed để tái lập.
    rng=np.random.default_rng(SEED)
    single=rng.random(TRIALS)<0.7
    first=rng.random(TRIALS)<0.7
    second=rng.random(TRIALS)<0.6
    both=first & second
    either=first | second
    return {
        "experimental_single":single.mean(),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "experimental_both":both.mean(),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "experimental_either":either.mean(),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "theoretical_single":0.7,  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "theoretical_both":0.7*0.6,  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
        "theoretical_either":1-(1-0.7)*(1-0.6),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
    }  # Đóng danh sách, mảng hoặc nhóm tham số đang xây dựng.

def task03_conditional_days():
    A=set(range(5))
    B=set(range(4))
    theoretical=len(A&B)/len(B)
    rng=np.random.default_rng(SEED+1)
    drawn=rng.integers(0,7,size=TRIALS)
    given_b=np.isin(drawn,list(B))
    joint=np.isin(drawn,list(A&B))
    experimental=float(joint.sum()/given_b.sum())  # Đếm hoặc cộng phần tử để tính tổng/tỷ lệ.
    return theoretical,experimental

def task04_bayes():  # Kết hợp xác suất nền và độ nhạy để tính posterior.
    p_spam=0.30
    p_flag_given_spam=0.90
    p_flag_given_clean=0.05
    p_flag=p_spam*p_flag_given_spam+(1-p_spam)*p_flag_given_clean
    posterior=p_spam*p_flag_given_spam/p_flag
    return p_flag,posterior

def task05_binomial():  # Phân phối nhị thức: số lần thành công trong n lượt độc lập.
    k=np.arange(0,9)
    probs=binom.pmf(k,n=8,p=0.75)  # Dùng hàm xác suất/kiểm định đúng tham số của đề.
    return k,probs,float(binom.pmf(6,n=8,p=0.75)),float(probs[6:].sum())

def save_plot(k,probs):
    import matplotlib
    matplotlib.use("Agg")
```

```
    import matplotlib.pyplot as plt
    folder=Path(__file__).resolve().parent/"outputs"
    folder.mkdir(exist_ok=True)
    fig,ax=plt.subplots()
    ax.bar(k,probs)
    ax.set(xlabel="Number of resolved requests (k)",ylabel="P(X=k)",title="Binomial(n=8, p=0.75)")
    ax.set_xticks(k)
    ax.grid(True,axis="y")
    target=folder/"binomial_distribution.png"
    fig.savefig(target,dpi=140,bbox_inches="tight")
    plt.close(fig)  # Đóng biểu đồ sau khi dùng để giải phóng bộ nhớ.
    print("Saved:",target)

def main():
    cards=task01_cards()
    simulated=task02_free_throws()
    cond_theory,cond_exp=task03_conditional_days()
    p_flag,p_actual_spam=task04_bayes()
    k,prob,p_exact,p_at_least=task05_binomial()
    if not (cards["P(A)"]==0.5): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (cards["P(B)"]==0.4): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (cards["P(A and B)"]==0.2): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (cards["P(A or B)"]==0.7): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (cond_theory==1.0): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (abs(simulated["experimental_single"]-0.7)<0.04): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.isclose(p_flag,0.305)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.isclose(prob.sum(),1)): raise ValueError("Kiểm tra dữ liệu không đạt")
    print("Task 1:",cards)
    print("Task 2:",simulated)
    print("Task 3 P(A|B), theoretical/experimental:",cond_theory,cond_exp)
    print("Task 4 P(flag), P(spam|flag):",round(p_flag,5),round(p_actual_spam,5))
    print("Task 5 P(X=6), P(X>=6):",round(p_exact,6),round(p_at_least,6))
    if "--no-plot" not in sys.argv:
        save_plot(k,prob)
if __name__=="__main__":
    main()
```

4. Giải thích nội dung và cách giải

Đây là phần diễn giải cho **một chương trình hoàn chỉnh ở Mục 3**. Các ví dụ code ngắn lặp lại trước đây được chuyển thành giải thích bằng lời để khi in không phải theo dõi nhiều phiên bản code.

Câu 1

Tạo `set(range(1,11))`, lọc từng biến cố bằng set comprehension. Dùng `&` cho giao và `|` cho hợp, rồi chia số phần tử cho kích thước không gian mẫu 10.

Vì sao làm vậy? Mỗi thẻ 1–10 có cơ hội như nhau, nên xác suất là số thẻ hợp điều kiện chia cho tổng 10 thẻ. Hai biến cố cần được biểu diễn thành hai tập hợp để lấy giao (cả hai đúng) và hợp (ít nhất một đúng).

Quy trình: (1) Tạo 10 số bằng `set(range(1, 11))`. (2) Lọc số chẵn bằng `x % 2 == 0`, lọc số lớn hơn 6 bằng `x > 6`. (3) Đếm phần tử mỗi tập; dùng `&` lấy giao và `|` lấy hợp. (4) Chia cho `len(space)` để có xác suất, rồi kiểm tra.

Công thức Python:

Giải thích cú pháp: `range(1, 11)` chứa từ 1 đến 10, **không** lấy 11. `set` là tập hợp không chứa phần tử trùng, `%` là chia lấy dư, `==` so sánh bằng. Trong tập hợp, `&` là giao và `|` là hợp; `len` đếm số phần tử; `/` chia thực. `A | B` không phải chỉ những thẻ có đúng một trong hai tính chất: thẻ vừa chẵn vừa lớn hơn 6 vẫn được tính **một lần**.

Ví dụ và lỗi thường gặp: A gồm năm thẻ chẵn, B gồm bốn thẻ lớn hơn 6, giao gồm 8 và 10. P(A∩B)=2/10, P(A∪B)=7/10.

Thuật ngữ cần nhớ: Sample space (không gian mẫu).

Câu 2

`np.random.default_rng(SEED)` tạo bộ sinh số có thể lặp; mỗi lần so ngẫu nhiên với ngưỡng 0,7/0,6 được một kết quả True/False. `mean()` cho tần suất thành công; dùng `&` và `|` trên hai mảng bool.

Vì sao giải bằng hai cách? Lý thuyết cho giá trị chính xác từ xác suất đã biết; mô phỏng tạo nhiều phép thử ngẫu nhiên để đối chiếu xem tần suất có gần lý thuyết hay không. Không yêu cầu 5.000 lượt phải cho đúng bằng 0.7.

Quy trình: (1) Tạo bộ sinh ngẫu nhiên với seed cố định cho dễ tái lập. (2) Sinh 5.000 số trong khoảng từ 0 đến 1, so sánh với tỷ lệ trúng rổ. (3) Với hai người chơi độc lập, lấy `&` để đếm cùng trúng, `|` để đếm ít nhất một trúng. (4) Tính trung bình các giá trị đúng/sai, rồi so sánh với lý thuyết.

Công thức Python:

Giải thích hàm và tham số: `default_rng(2026)` dùng hạt giống (seed) để cùng code tạo cùng chuỗi số; `random(5000)` tạo đúng 5.000 lượt; `<` xét thành công; `.mean()` trên mảng Boolean lấy tỷ lệ giá trị `True`. `&` yêu cầu cả hai cùng trúng; `|` chỉ cần một người trúng. Biểu thức `1 - ...` lấy phần bù của trường hợp **cả hai cùng trượt**, tránh đếm trùng.

Ví dụ và lỗi thường gặp: Tần suất trong 5.000 lượt rút là kết quả mô phỏng, có thể lệch lý thuyết. Với hai người độc lập, P(cả hai)=0,7×0,6=0,42.

Thuật ngữ cần nhớ: Simulation (mô phỏng).

Câu 3

Đặt thứ Hai đến Chủ nhật ứng với mã 0–6; B={0,1,2,3} là tập con của A={0..4}. Trong mô phỏng, đếm số lượt chọn vào B rồi kiểm tra đồng thời A∩B để được xác suất có điều kiện.

Vì sao dùng mẫu số là B? Đề đã cho điều kiện ngày thuộc B (thứ Hai–Năm); trong bốn ngày này, tất cả đều là ngày làm việc A (thứ Hai–Sáu). Vì vậy xác suất có điều kiện bằng 1. Không chia cho cả 7 ngày trong tuần.

Quy trình: (1) Biểu diễn bảy ngày thành số 0–6; A là 0–4, B là 0–3. (2) Lấy giao `A & B` để tìm ngày thuộc cả hai. (3) Đếm giao và đếm B để chia. (4) Nếu mô phỏng, chỉ giữ lượt rơi vào B rồi so sánh tỷ lệ.

Công thức Python:

Giải thích hàm và tham số: `integers(0, 7, size=5000)` sinh số nguyên từ **0 đến 6** (`7` không được lấy), 5.000 lần. `np.isin` kiểm tra mỗi ngày có thuộc danh sách không, trả mảng Boolean; `.sum()` đếm các giá trị `True`. `size=5000` là số lượt mô phỏng; `len(B)` là số trường hợp của điều kiện đã biết.

Ví dụ và lỗi thường gặp: Biến cố B (thứ Hai–thứ Năm) nằm hoàn toàn trong biến cố A (thứ Hai–thứ Sáu), nên P(A|B)=1.

Thuật ngữ cần nhớ: Conditional probability (xác suất có điều kiện).

Câu 4

Tính tổng hai cách bị hệ thống gắn cờ spam: spam thật bị phát hiện và email sạch bị báo nhầm. Sau đó dùng Bayes đổi chiều điều kiện từ P(flag|spam) sang P(spam|flag).

Vì sao không dùng trực tiếp 90%? 90% là tỷ lệ bị gắn cờ **trong các thư spam**, còn đề hỏi thư đã bị gắn cờ có khả năng thật sự là spam bao nhiêu. Nhóm thư bị gắn cờ còn chứa cả thư sạch bị gắn nhầm. Cần tính cả hai nguồn gắn cờ trước khi chia.

Quy trình: (1) Đặt tỷ lệ spam và thư sạch. (2) Tính phần thư spam bị gắn cờ. (3) Tính phần thư sạch bị gắn cờ nhầm. (4) Cộng hai phần để có tổng xác suất gắn cờ. (5) Chia phần spam bị gắn cờ cho tổng đã tính.

Công thức Python:

Giải thích phép tính: `p_flag_given_spam` là xác suất gắn cờ khi đã biết thư là spam (độ nhạy); `p_flag_given_clean` là tỷ lệ gắn nhầm thư sạch. `*` kết hợp tỷ lệ nhóm với tỷ lệ gắn cờ trong nhóm, `+` cộng hai trường hợp không giao nhau, `/` chuyển sang xác suất **trong nhóm đã bị gắn cờ**. Kết quả khoảng **88.52%**, không phải 90%.

Ví dụ và lỗi thường gặp: P(flag)=0,3×0,9+0,7×0,05=0,305. P(spam|flag)=0,27/0,305, không đánh đồng với P(flag|spam)=0,9.

Thuật ngữ cần nhớ: Bayes (định lý Bayes).

Câu 5

Số thành công tuân theo nhị thức vì có 8 phép thử độc lập cùng xác suất p=0,75. Hàm pmf trả xác suất đúng k; cộng pmf cho k=6,7,8 để lấy ít nhất 6. Hàm `save_plot` trong code tham khảo dùng `plt.bar`.

Vì sao dùng phân phối nhị thức? Mỗi yêu cầu được xử lý thành công hoặc thất bại, có xác suất thành công 0.75 và giả định các lần độc lập. Bài cần xác suất đúng k thành công trong 8 lần, rồi cộng xác suất của k bằng 6, 7 hoặc 8.

Quy trình: (1) Tạo danh sách số lần thành công có thể từ 0 đến 8. (2) Dùng hàm phân phối nhị thức tính xác suất cho từng số k. (3) Lấy xác suất đúng 6 lần. (4) Cộng ba giá trị ứng với k từ 6 trở lên. (5) Vẽ biểu đồ cột để nhìn khả năng từng số lần thành công.

Công thức Python:

Giải thích hàm và tham số: `binom.pmf` là hàm khối xác suất (*probability mass function*) tính xác suất đúng k lần thành công; `n=8` là tổng số lượt, `p=0.75` là tỷ lệ thành công mỗi lượt. `np.arange(0, 9)` tạo các số 0–8; `probs[6:]` lấy các xác suất từ vị trí 6 đến hết; `.sum()` cộng chúng. Nếu vẽ và lưu ảnh, `matplotlib.use("Agg")` dùng backend không mở cửa sổ và `--no-plot` là đối số dòng lệnh bỏ phần vẽ trong code chính.

Ví dụ và lỗi thường gặp: Với tám yêu cầu độc lập, dùng phân phối nhị thức. Đúng sáu dùng pmf(6); ít nhất sáu cộng pmf(6)+pmf(7)+pmf(8).

Thuật ngữ cần nhớ: Binomial distribution (phân phối nhị thức).

Cách xác định đúng mẫu số của xác suất

Ở bài đếm biến cố, đầu tiên phải viết **không gian mẫu** (tất cả kết quả có thể xảy ra). Khi các kết quả đồng khả năng, xác suất là số trường hợp thuận lợi chia tổng số trường hợp. Tập `A & B` là giao (hai điều kiện đều đúng); `A | B` là hợp (ít nhất một điều kiện đúng); `len` đếm số phần tử. Không được lấy tổng số sự kiện bằng cách cộng riêng hai nhóm nếu chúng có phần tử giao mà không trừ phần trùng.

Mô phỏng so với lý thuyết: `np.random.default_rng(SEED)` tạo bộ sinh ngẫu nhiên có thể tái lập; `rng.random(TRIALS) < p` cho mảng `True/False`; `mean()` trên mảng Boolean chính là tỉ lệ `True`. Với 5.000 lượt thử, tỷ lệ thực nghiệm thường gần xác suất thật nhưng không bắt buộc trùng chính xác.

Bayes có hai chiều điều kiện khác nhau. Giả sử 30% thư là spam, hệ thống đánh dấu 90% spam và 5% thư sạch. Xác suất một thư bất kỳ bị đánh dấu là `0.30 × 0.90 + 0.70 × 0.05 = 0.305`. Trong các thư bị đánh dấu, tỷ lệ thật sự là spam là `0.27 / 0.305 ≈ 88,52%`. Không nhầm `P(bị đánh dấu | spam)=90%` với `P(spam | bị đánh dấu)`.

Phân phối nhị thức: `binom.pmf(k, n, p)` tính xác suất đúng `k` thành công trong `n` lượt, khi xác suất thành công cố định và các lượt độc lập. Với câu hỏi "ít nhất 6" phải cộng xác suất của `k=6,7,8`, không lấy riêng `P(X=6)`. Kiểm tra các xác suất luôn ở đoạn từ 0 đến 1.

5. Giải thích các định nghĩa

Các khái niệm được giải thích bằng ví dụ số để bạn có thể tính lại bằng tay.

Không gian mẫu

Là gì? Tập hợp mọi kết quả có thể xảy ra. Chọn một số từ 1 đến 10 có 10 kết quả đồng khả năng.

Biến cố

Là gì? Một nhóm kết quả cùng thỏa điều kiện. Số chẵn trong 1 đến 10 có 5 số nên xác suất là 5/10.

Giao và hợp

Là gì? Giao yêu cầu cả hai điều kiện; hợp chỉ cần một trong hai. Nếu A có 5 kết quả, B có 4 và trùng 2, hợp có 5+4−2=7.

Biến cố độc lập

Là gì? Biết kết quả phép thử này không thay đổi xác suất phép thử kia. Hai lần tung đồng xu độc lập cùng ra ngửa có xác suất 0,5×0,5=0,25.

Xác suất có điều kiện

Là gì? Tỷ lệ trong nhóm B đã biết. Nếu B có 20 trường hợp, 5 trường hợp đồng thời thuộc A thì xác suất A khi B là 5/20=25%.

Bayes

Là gì? Cách điều chỉnh xác suất khi xuất hiện thông tin mới. Ví dụ 20% sản phẩm lỗi, máy báo lỗi đúng 90% và báo nhầm 10% sản phẩm tốt: tỷ lệ sản phẩm lỗi trong nhóm bị báo là 0,18/(0,18+0,08)≈69,23%.

Mô phỏng và seed

Là gì? Dùng máy tạo nhiều phép thử để ước lượng xác suất. Seed cố định giúp tái lập các lần chạy, không bảo đảm mẫu đại diện.

Nhị thức

Là gì? Đếm số lần thành công trong n lượt thử độc lập, xác suất thành công mỗi lượt không đổi. Ít nhất 6 lần trong 8 lượt phải cộng xác suất cho 6,7,8.

Boolean

Là gì? True và False biểu thị đúng và sai; phép lấy trung bình Boolean chính là tỷ lệ đúng. Ba kết quả đúng trong năm lượt cho 3/5=0,6.

Lab 8 — Thống kê

1. Đề bài tiếng Anh

LAB SESSION 8

Problem Description

A mobile application company wants to analyze users' daily app usage time to understand usage patterns, data variability, and statistical characteristics.

You are given daily app usage time for **50 users**, measured in **minutes per day**.

Dataset

Input data — usage_minutes (50 observations, minutes): 35, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 120, 125, 130, 135, 140, 145, 150, 155, 160, 165, 170, 175, 180, 185, 190, 195, 200, 205, 210, 215, 220, 225, 230, 240, 250, 260, 270, 280, 290, 300, 320, 350, 480

Requirements

Task 1 – Descriptive Statistics

- Calculate mean, median, and mode.
- Find min, max, and range.
- Calculate **sample** variance and **sample** standard deviation.

Task 2 – Frequency Distribution and Histogram

Divide usage into the six groups below:

Group	Usage time (minutes/day)
1	0–60
2	60–120
3	120–180
4	180–240
5	240–360
6	360–500

- Use `pd.cut()` and `value_counts()` to create a frequency distribution table.
- Calculate cumulative frequency using `cumsum()`.
- Create a histogram with title and axis labels.

Task 3 – Quartiles and Outlier Detection

- Calculate Q1, Q2, Q3 and IQR.
- Calculate lower and upper bounds using **1.5 × IQR**.
- Identify usage times outside the bounds.

Task 4 – Sampling and Hypothesis Testing

- Randomly select 10 users.
- Calculate sample mean and compare it with mean of all 50 users.
- Test the company's claim that **population mean usage is 120 minutes/day**, using a **one-sample t-test** at $\alpha = 0.05$.
- Based on p-value, decide whether to reject or fail to reject H_0.

Task 5 – Conclusion

Write a short conclusion (3–5 sentences) about:

- Typical daily app usage time.
- Variability and presence of outliers.
- Hypothesis-test result.
- One recommendation for the company.

Chuyển từ `LAB SESSION 8.docx` sang Markdown.

2. Đề bài tiếng Việt

LAB SESSION 8 — Thống kê

Công ty ứng dụng di động phân tích thời gian sử dụng mỗi ngày của **50 người dùng**, tính bằng **phút/ngày**. Các bài cần dùng NumPy (thư viện tính toán mảng), Pandas (thư viện bảng dữ liệu), SciPy (thư viện tính toán khoa học) và Matplotlib (thư viện vẽ biểu đồ).

Dữ liệu 50 người (phút/ngày)

Dữ liệu đầu vào — usage_minutes (50 quan sát, đơn vị phút): 35, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 120, 125, 130, 135, 140, 145, 150, 155, 160, 165, 170, 175, 180, 185, 190, 195, 200, 205, 210, 215, 220, 225, 230, 240, 250, 260, 270, 280, 290, 300, 320, 350, 480

Toàn bộ yêu cầu

Bài 1

Từ 50 người dùng và số phút sử dụng ứng dụng mỗi ngày, tính trung bình (mean (trung bình)), trung vị (median (trung vị)), mốt (mode (mốt)), nhỏ nhất, lớn nhất và khoảng biến thiên. Tính **phương sai mẫu** và **độ lệch chuẩn mẫu** (không dùng phương sai tổng thể).

Bài 2

Chia thời gian dùng ứng dụng thành đúng 6 nhóm: 0–60, 60–120, 120–180, 180–240, 240–360, 360–500 phút/ngày. Dùng `pd.cut()` và `value_counts()` tạo bảng tần suất, `cumsum()` tính tần suất tích lũy, vẽ histogram (biểu đồ tần suất) có tiêu đề và nhãn trục. Quy định rõ dữ liệu nằm ở ranh giới 60/120/... được tính đúng một nhóm.

Bài 3

Tính Q1, Q2, Q3 và IQR; xác định biên dưới và biên trên theo quy tắc `1.5 × IQR`, rồi liệt kê các thời gian nằm ngoài hai biên này.

Bài 4

Chọn ngẫu nhiên 10 người dùng, tính trung bình của mẫu 10 và so sánh với trung bình 50 người. Kiểm định giả thuyết công ty rằng trung bình tổng thể là **120 phút/ngày** bằng one-sample (mẫu) t-test, mức ý nghĩa α=0,05. Dựa vào p-value (giá trị xác suất kiểm định) quyết định bác bỏ hay chưa đủ bằng chứng bác bỏ H0.

Bài 5

Viết kết luận ngắn 3–5 câu về mức sử dụng điển hình, độ biến thiên/ngoại lệ, kết quả kiểm định và một khuyến nghị cho công ty. Không mặc định dữ liệu mô phỏng là bằng chứng về toàn bộ người dùng thực.

Bảng nhóm tần suất từ Word: 0–60, 60–120, 120–180, 180–240, 240–360, 360–500 phút/ngày (6 nhóm).

3. Code Python có chú thích

```
from pathlib import Path
import sys
import numpy as np
import pandas as pd
from scipy import stats

USAGE_MINUTES=np.array([
    35,45,50,55,60,65,70,75,80,85,
    90,95,100,105,110,115,120,120,125,130,
    135,140,145,150,155,160,165,170,175,180,
    185,190,195,200,205,210,215,220,225,230,
    240,250,260,270,280,290,300,320,350,480
],dtype=float)

def task01_descriptive(values):  # Tính trung bình, trung vị, mode và độ phân tán.
    return {
        "mean":float(values.mean()),"median":float(np.median(values)),
        "mode":float(pd.Series(values).mode().iloc[0]),
        "minimum":float(values.min()),"maximum":float(values.max()),
        "range":float(np.ptp(values)),
        "sample_variance":float(values.var(ddof=1)),
        "sample_std":float(values.std(ddof=1)),
    }

def task02_frequency(values):  # Chia thời lượng thành các khoảng và đếm theo nhóm.
    bins=[0,60,120,180,240,360,500]
    labels=["0–<60","60–<120","120–<180","180–<240","240–<360","360–<500"]
    groups=pd.cut(values,bins=bins,labels=labels,right=False,include_lowest=True)
    freq=pd.Series(groups).value_counts(sort=False)
    frame=pd.DataFrame({"frequency":freq})
    frame["cumulative"]=frame["frequency"].cumsum()
    if not (frame["frequency"].sum()==len(values)): raise ValueError("Kiểm tra không đạt")
    return bins,frame

def task03_outliers(values):  # Dùng quy tắc IQR để đánh dấu giá trị bất thường.
    q1,q2,q3=np.percentile(values,[25,50,75])
    iqr=q3-q1
    low,high=q1-1.5*iqr,q3+1.5*iqr
```

```
    unusual=values[(values<low)|(values>high)]
    return q1,q2,q3,iqr,low,high,unusual

def task04_sample_and_test(values):  # Rút mẫu ngẫu nhiên trước khi làm t-test.
    rng=np.random.default_rng(2026)  # Rút mẫu ngẫu nhiên; seed (hạt giống) giúp tái lập.
    sample=rng.choice(values,size=10,replace=False)  # Rút mẫu ngẫu nhiên; seed (hạt giống) giúp tái lập.
    result=stats.ttest_1samp(values,popmean=120.0)  # Tính thống kê theo hàm/tham số đã học.
    return sample,float(sample.mean()),float(result.statistic),float(result.pvalue)

def task05_conclusion(descriptive,outliers,pvalue):  # Viết nhận xét dựa trên kết quả, không suy diễn nhân quả.
    typical=descriptive["median"]
    decision="reject H0" if pvalue<0.05 else "fail to reject H0"
    print(f"Typical use is around median {typical:.1f} minutes/day.")
    print(f"Sample SD is {descriptive['sample_std']:.1f} minutes, with {len(outliers)} IQR outlier(s).")
    print(f"One-sample t-test vs 120 min/day: p={pvalue:.6g}; {decision} at alpha=0.05.")
    print("Recommendation: study long-duration sessions separately and design user-friendly break reminders.")

def save_plot(values,bins):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    output=Path(__file__).resolve().parent/"outputs"
    output.mkdir(exist_ok=True)
    fig,ax=plt.subplots()
    ax.hist(values,bins=bins,edgecolor="black")
    ax.set(xlabel="Daily app usage (minutes)",ylabel="Users",title="Daily usage of 50 users")
    ax.grid(True,axis="y")
    target=output/"usage_histogram.png"
    fig.savefig(target,dpi=140,bbox_inches="tight")  # Vẽ/lưu biểu đồ (plot: đồ thị) để diễn giải kết quả.
    plt.close(fig)  # Đóng biểu đồ để tránh giữ tài nguyên.
    print("Saved:",target)

def main():
    if not (USAGE_MINUTES.shape==(50,)): raise ValueError("Kiểm tra không đạt")
    desc=task01_descriptive(USAGE_MINUTES)
    bins,freq=task02_frequency(USAGE_MINUTES)
    q1,q2,q3,iqr,lo,hi,outliers=task03_outliers(USAGE_MINUTES)
    sample,sample_mean,t,p=task04_sample_and_test(USAGE_MINUTES)
    if not (len(sample)==10): raise ValueError("Kiểm tra không đạt")
    if not (0<=p<=1): raise ValueError("Kiểm tra không đạt")
    print("Descriptive:",desc)
    print("Frequency:\n",freq.to_string())
    print(f"Q1={q1:.2f}, Q2={q2:.2f}, Q3={q3:.2f}, IQR={iqr:.2f}")
    print(f"Outlier limits [{lo:.2f},{hi:.2f}], values:",outliers.tolist())
    print("Sample of 10:",sample.tolist(),"sample mean:",round(sample_mean,2))
    print("t:",round(t,4),"p:",round(p,6))
    task05_conclusion(desc,outliers,p)
    if "--no-plot" not in sys.argv:
        save_plot(USAGE_MINUTES,bins)
if __name__=="__main__":
    main()
```

4. Giải thích nội dung và cách giải

Phần 3 chứa **một chương trình Python thống nhất**. Dưới đây là giải thích chi tiết cách từng nhóm hàm giải quyết yêu cầu, không lặp lại các đoạn code rời.

Câu 1

Dữ liệu 50 dòng được đưa nguyên vào `USAGE_MINUTES`. Dùng NumPy (thư viện tính toán mảng)/Pandas (thư viện bảng dữ liệu) tính các chỉ số; dùng `ddof=1` để phương sai chia cho n−1 chứ không phải n. Mode (mốt) được lấy bằng `.mode().iloc[0]`.

Vì sao cần các chỉ số khác nhau? Trung bình chỉ mức sử dụng chung nhưng dễ bị kéo lên bởi 480 phút; trung vị cho mức giữa; phương sai và độ lệch chuẩn cho thấy độ phân tán. Vì đang mô tả một mẫu 50 người, lời giải dùng `ddof=1` cho thống kê mẫu.

Các bước: Chuyển 50 số phút sang mảng số; tính trung bình, trung vị, mốt, min/max; tính khoảng biến thiên, phương sai mẫu và độ lệch chuẩn mẫu rồi so sánh. **Tham số và cú pháp:** `dtype=float` chọn kiểu số thực; `np.median` tính trung vị; `.mode()` trả một tập các giá trị có tần số cao nhất, `.iloc[0]` chọn giá trị đầu tiên nếu có nhiều mốt; `np.ptp` tính max trừ min. `ddof=1` nghĩa là hiệu chỉnh mẫu: chia cho `n-1` thay vì `n`. Đơn vị của `variance` là phút bình phương, còn `std` là phút.

Ví dụ/lỗi thường gặp: Giữ đủ 50 số phút, kể cả 480. Phương sai và độ lệch chuẩn **mẫu** chia cho n−1 (ddof=1), không chia n như tổng thể.

Từ vựng chính: Descriptive statistics (thống kê mô tả).

Câu 2

Các khoảng liên tiếp trong `bins=[0,60,120,180,240,360,500]`, `right=False` nghĩa là [0,60), [60,120),... Tránh một phút đúng 60 hoặc 120 bị tính hai lần. Lấy tần số của từng khoảng rồi cộng dồn; hình histogram (biểu đồ tần suất) dùng chức năng `save_plot` đã có.

Vì sao gom theo khoảng? Bảng có 50 giá trị riêng lẻ khó thấy mức sử dụng nào phổ biến. Chia vào cùng các khoảng phút giúp đếm người theo nhóm; tần số tích lũy trả lời đã có bao nhiêu người ở các nhóm từ đầu tới nhóm hiện tại.

Các bước: (1) Định nghĩa biên nhóm theo phút. (2) Phân từng thời gian vào một nhóm. (3) Đếm tần số theo thứ tự nhóm, không đảo theo nhóm phổ biến. (4) Cộng dồn và kiểm tra tổng là 50. **Flag và tham số:** `bins` là các ranh giới nhóm; `labels` là tên nhóm; `right=False` dùng khoảng gồm cận trái nhưng **không** gồm cận phải, nên 120 được xếp nhóm 120–180; `include_lowest=True` không bỏ cận thấp nhất. `value_counts(sort=False)` giữ thứ tự nhóm thay vì sắp theo lượng người. `cumsum()` cộng dồn. Đồ thị `ax.hist(values, bins=bins)` phải dùng cùng ranh giới nhóm.

Ví dụ/lỗi thường gặp: Nhóm khoảng theo [0,60), [60,120)... để số 60 không bị đếm hai lần. Tổng tần số phải bằng 50.

Từ vựng chính: Histogram (biểu đồ tần suất).

Câu 3

`np.percentile` lấy ba phân vị, dùng IQR làm thước đo độ trải rộng ít nhạy với ngoại lệ. Các điểm nhỏ hơn biên dưới hoặc lớn hơn biên trên được đánh dấu, không tự động xóa.

Vì sao dùng IQR? Giá trị 480 phút có thể khác thường. Dùng khoảng tứ phân vị giúp xác định điểm ngoài vùng dữ liệu trung tâm mà không tự ý xóa hoặc sửa dữ liệu nguồn.

Các bước: (1) Tính tứ phân vị 25%, 50%, 75%. (2) Lấy Q3 trừ Q1 để tính IQR. (3) Tính biên dưới và biên trên bằng 1.5 lần IQR. (4) Lọc các giá trị nhỏ hơn biên dưới hoặc lớn hơn biên trên. **Tham số và toán tử:** `np.percentile(values, [25, 50, 75])` trả ba mốc phần trăm; `|` kết hợp hai mảng điều kiện theo nghĩa **hoặc**; dấu ngoặc quanh mỗi phép so sánh là cần thiết. `1.5` là hệ số quy ước của cách phát hiện ngoại lệ IQR, **không** phải mức ý nghĩa thống kê. Ngoại lệ được đánh dấu để phân tích, không tự động loại bỏ.

Ví dụ/lỗi thường gặp: IQR=Q3−Q1; biên dưới=Q1−1,5×IQR và biên trên=Q3+1,5×IQR. Không tự xóa ngoại lệ trước khi tính.

Từ vựng chính: Quartiles (tứ phân vị).

Câu 4

`default_rng(2026).choice(...,size=10,replace=False)` chọn 10 giá trị không lặp. Code kiểm định đã có gọi `ttest_1samp(values,popmean=120)`, tức dùng **toàn bộ 50 quan sát** cho phép kiểm định, còn mẫu 10 chỉ minh họa lấy mẫu. Đừng gọi nhầm t-test chỉ trên 10 quan sát. So sánh p-value (giá trị xác suất kiểm định) với 0,05.

Vì sao có hai bước lấy mẫu và kiểm định? Lấy ngẫu nhiên 10 người minh họa sampling (lấy mẫu), còn giả thuyết phải so sánh mức trung bình với mốc 120 phút. Code tham khảo thực hiện kiểm định t trên **đủ 50 giá trị**, không nhầm dùng 10 mẫu vừa rút để kiểm định.

Các bước: (1) Dùng hạt giống cố định rút 10 người không hoàn lại. (2) Lấy trung bình của 10 người để so sánh. (3) Kiểm định t một mẫu với mốc giả thuyết 120 phút bằng 50 giá trị. (4) So `pvalue` với ngưỡng 0.05. **Flag và tham số:** `size=10` là số người rút; `replace=False` nghĩa là **không hoàn lại** nên không chọn một người hai lần; `popmean=120.0` là giá trị trung bình giả định cần kiểm định. `result.statistic` là thống kê t; `result.pvalue` là p-value. `p_value < 0.05` cho biết bác bỏ giả thuyết gốc ở mức ý nghĩa 5% nếu đúng; nếu sai thì **chưa đủ bằng chứng** bác bỏ, không khẳng định giả thuyết gốc là đúng. Kiểm định t cần xem xét giả định thống kê về dữ liệu.

Ví dụ/lỗi thường gặp: Lấy 10 mẫu không hoàn lại, nhưng t-test trong code nguồn dùng 50 quan sát. So sánh p-value (giá trị xác suất kiểm định) với 0,05 trước khi kết luận.

Từ vựng chính: Hypothesis testing (kiểm định giả thuyết).

Câu 5

Hàm `task05_conclusion` của code tham khảo in bốn ý: dùng median (trung vị) đại diện mức điển hình, SD và số điểm ngoại lệ để mô tả độ biến thiên, đọc p-value (giá trị xác suất kiểm định) cho kiểm định, và đề xuất xem xét phiên dùng dài cùng lời nhắc nghỉ. Điều này đáp ứng 3–5 câu của đề.

Vì sao kết luận dựa trên nhiều chỉ số? Chỉ nhìn trung bình có thể bỏ qua độ phân tán và người dùng bất thường. Một khuyến nghị cần dựa trên trung vị, độ lệch chuẩn, ngoại lệ và kết quả kiểm định đã có, không suy diễn tác động nhân quả từ dữ liệu quan sát.

Các bước: (1) Chọn trung vị làm đại diện cho mức dùng thường gặp. (2) Nêu độ lệch chuẩn mẫu và số ngoại lệ IQR. (3) Diễn giải ý nghĩa kiểm định t. (4) Đề xuất quan sát các phiên dùng dài và nhắc nghỉ hợp lý. **Cú pháp và tham số:** `descriptive["median"]` lấy giá trị theo tên trường của dictionary; `len(outliers)` đếm ngoại lệ. Biểu thức `A if điều_kiện else B` chọn câu giải thích đúng theo p-value; `:.1f` trong f-string chỉ quy định **một chữ số thập phân khi in**, không thay số đã tính. Không coi ngoại lệ 480 phút là lỗi dữ liệu chỉ vì giá trị lớn.

Ví dụ/lỗi thường gặp: Đối chiếu median (trung vị), SD, ngoại lệ và kiểm định khi viết 3–5 câu. Không suy rộng số liệu mô phỏng thành kết luận cho mọi người dùng.

Từ vựng chính: Conclusion (kết luận).

Quy trình phân tích một bộ dữ liệu 50 quan sát

Bước 1 — Kiểm tra dữ liệu: 50 giá trị `usage_minutes` là số phút sử dụng; đây là đơn vị của mọi trung bình, trung vị và độ lệch chuẩn. `np.array(..., dtype=float)` bảo đảm phép tính số thực. Không tự loại một giá trị lớn chỉ vì trông khác thường, bởi 480 phút có thể là số đo thật.

Bước 2 — Tóm tắt trung tâm và độ phân tán: `mean` lấy tổng chia số quan sát; `median` là giá trị giữa sau sắp xếp; `mode` là giá trị xuất hiện nhiều nhất. `np.std(values, ddof=0)` chia theo tổng thể quan sát, còn `ddof=1` thường dùng khi ước lượng độ lệch chuẩn từ một mẫu; hai kết quả khác nhau vì mẫu số khác nhau. Phương sai có đơn vị phút bình phương, độ lệch chuẩn có đơn vị phút, nên SD dễ diễn giải hơn trên thang đo gốc.

Bước 3 — Bảng tần suất và ngoại lệ: `pd.cut` tạo nhóm theo các ranh giới đã định; tham số `right=False` nghĩa là khoảng gồm đầu trái nhưng không gồm đầu phải. `value_counts(sort=False)` đếm theo thứ tự nhóm. `np.percentile(values, [25,50,75])` tìm Q1, median, Q3; `IQR = Q3 - Q1`, ngưỡng dưới `Q1 - 1.5 × IQR`, ngưỡng trên `Q3 + 1.5 × IQR`. Điểm ngoài ngưỡng cần được **xem xét**, không tự động xóa.

Bước 4 — Lấy mẫu và kiểm định: Dùng `default_rng(seed)` để tái lập lượt rút. **Standard deviation** (độ lệch chuẩn) mô tả độ phân tán cá nhân; **standard error** (sai số chuẩn) đo mức dao động của *trung bình mẫu*. Khi làm `ttest_1samp`, `p-value` nhỏ hơn mức ý nghĩa được chọn có thể dẫn tới bác bỏ H_0, nhưng không phải xác suất H_0 đúng và không cho thấy một yếu tố gây ra yếu tố khác.

Bước 5 — Kết luận: Nêu được giá trị điển hình, mức phân tán, số ngoại lệ, kết luận kiểm định và giới hạn của dữ liệu; một lời khuyên thực tế phải bám số liệu chứ không suy rộng từ một mẫu thành mọi người dùng.

5. Giải thích các định nghĩa

Các khái niệm được giải thích bằng lời trước, rồi đưa ví dụ nhỏ để đối chiếu từng phép tính.

Mean (trung bình)

Là gì và dùng để làm gì? Cộng các giá trị rồi chia cho số quan sát; dùng để biết mức điển hình nhưng dễ bị số quá lớn kéo lệch.

Ví dụ: [30,40,80] có mean=(30+40+80)/3=50 phút.

Median (trung vị) và mode (yếu vị)

Là gì và dùng để làm gì? Median là số giữa của dãy đã sắp; mode là số xuất hiện nhiều nhất. Median ít bị ảnh hưởng bởi số cực lớn.

Ví dụ: [30,40,200] có median=40, mean=90; [30,40,40,50] có mode=40.

Variance và standard deviation (phương sai và độ lệch chuẩn)

Là gì và dùng để làm gì? Đo độ phân tán quanh trung bình. Phương sai tính trung bình của các bình phương sai lệch; độ lệch chuẩn là căn bậc hai của phương sai, cùng đơn vị với dữ liệu.

Ví dụ: [50,50,50] có SD=0; [20,50,80] có SD lớn hơn vì dữ liệu trải rộng.

ddof=0 và ddof=1

Là gì và dùng để làm gì? `ddof` quy định mẫu số khi tính phương sai: với n giá trị, ddof=0 chia n, ddof=1 chia n−1; thường dùng tương ứng cho tổng thể và ước lượng từ mẫu.

Ví dụ: [2,4,6] có mean=4, tổng sai lệch bình phương=8; phương sai ddof=0 là 8/3, ddof=1 là 8/2.

Frequency, bins (tần suất, khoảng nhóm)

Là gì và dùng để làm gì? Tần suất là số quan sát trong mỗi khoảng. `pd.cut` phân nhóm theo ranh giới bins, và `value_counts` đếm mỗi nhóm.

Ví dụ: [35,45,80] với hai khoảng [0,60), [60,120) có tần suất 2 và 1.

Quartiles, IQR và outlier (tứ phân vị, khoảng giữa, ngoại lệ)

Là gì và dùng để làm gì? Q1 gần mốc 25%, Q2 là trung vị, Q3 gần mốc 75%. IQR=Q3−Q1; điểm ngoài ngưỡng theo quy tắc 1,5×IQR là ngoại lệ cần kiểm tra.

Ví dụ: Q1=40, Q3=100 thì IQR=60; ngưỡng trên=100+1,5×60=190.

Sample, standard error và p-value (mẫu, sai số chuẩn, giá trị p)

Là gì và dùng để làm gì? Mẫu là một nhóm được chọn từ tổng thể. Sai số chuẩn mô tả độ dao động của trung bình mẫu; p-value đo độ bất thường của kết quả khi giả thuyết H0 đúng.

Ví dụ: Nếu p-value=0,03 và mức ý nghĩa 0,05, thường bác bỏ H0; không được nói xác suất H0 đúng là 3%.

Lab 10 — CSV và phân tích điểm sinh viên (lời giải từ PythoClass/main.py)

1. Đề bài tiếng Anh

LAB Session 10

Objective

Develop a Python program to manage student grades using **File I/O, Functions, and Error Handling**.

Requirements

Create a menu-driven program with these functions:

1. **Display All Students:** Read/display student data from `students.csv`.
2. **Add a New Student:** Enter student information and grades. Validate input and prevent duplicate Student IDs.
3. **Search Student by ID:** Search/display a student's information.
4. **Calculate Average Grade:** Display each student's average grade.
5. **Save Students to File:** Save current student data to `students.csv`.
6. **Exit:** Exit the program.

Error Handling

Handle file-not-found errors, invalid numeric input, grades outside **0–100**, and duplicate student IDs.

Sample `students.csv`

```
student_id,name,class_name,Python,Math,ML
S001,Learner 01,AI01,85,78,90
S002,Learner 02,AI01,72,85,80
S003,Learner 03,AI01,90,88,95
S004,Learner 04,AI02,65,70,68
S005,Learner 05,AI02,82,75,85
S006,Learner 06,AI02,78,80,72
S007,Learner 07,AI03,95,92,94
S008,Learner 08,AI03,70,68,75
S009,Learner 09,AI03,88,85,82
S010,Learner 10,AI03,76,90,88
```

Chuyển từ `Lab Session 10.docx` sang Markdown. Tên học viên trong dữ liệu ví dụ đã được thay bằng mã minh họa nhằm tránh công khai thông tin định danh; điểm số, lớp và cấu trúc bảng được giữ nguyên.

2. Đề bài tiếng Việt

LAB SESSION 10 — Quản lý điểm sinh viên bằng file CSV (tệp phân cách bằng dấu phẩy)

Mục tiêu

Xây dựng chương trình Python quản lý điểm sinh viên bằng thao tác **File I/O**, các **hàm** và **xử lý lỗi**.

Các yêu cầu

Tạo chương trình có menu với sáu chức năng:

1. **Hiển thị tất cả sinh viên:** Đọc dữ liệu sinh viên từ `students.csv` và hiển thị thông tin.
2. **Thêm sinh viên:** Nhập thông tin cá nhân và điểm của từng môn; kiểm tra dữ liệu hợp lệ và chống mã sinh viên trùng.
3. **Tìm theo mã sinh viên:** Nhập mã và hiển thị thông tin của sinh viên cần tìm.
4. **Tính điểm trung bình:** In điểm trung bình từng sinh viên.
5. **Lưu vào file:** Ghi dữ liệu đang quản lý vào `students.csv`.
6. **Thoát:** Kết thúc chương trình.

Xử lý lỗi

Cần xử lý **không tìm thấy file**, **nhập số không hợp lệ**, **điểm ngoài phạm vi 0–100** và **mã sinh viên trùng**.

Dữ liệu mẫu students.csv (tệp phân cách bằng dấu phẩy)

Dùng bảng CSV gồm mã sinh viên, tên, lớp, điểm Python, Math và ML. Bảng được giữ nguyên số liệu, tên đã được thay bằng mã/tên giả lập để bảo vệ danh tính:

```
student_id,name,class_name,Python,Math,ML
S001,Learner 01,AI01,85,78,90
S002,Learner 02,AI01,72,85,80
S003,Learner 03,AI01,90,88,95
S004,Learner 04,AI02,65,70,68
S005,Learner 05,AI02,82,75,85
S006,Learner 06,AI02,78,80,72
S007,Learner 07,AI03,95,92,94
S008,Learner 08,AI03,70,68,75
S009,Learner 09,AI03,88,85,82
S010,Learner 10,AI03,76,90,88
```

Các cột của file là `student_id,name,class_name,Python,Math,ML`; tên `Learner 01..10` là dữ liệu minh họa đã được ẩn danh.

3. Code Python có chú thích

```
import pandas as pd  # Pandas (thư viện xử lý bảng dữ liệu) với tên ngắn pd.

def inputHelper(data):  # Đọc đầu vào; chú ý kiểm tra kiểu bên dưới chưa chuyển chuỗi sang số.
    label = data["label"]
    formats = data["formats"]
    inputTemp = input(label)
    if inputTemp == "":
        return(None)
    blocks =  inputTemp.split(";")  # Tách chuỗi đầu vào tại dấu chấm phẩy.


    if len(blocks) != len(formats):
        print("!!![ERROR] Input blocks is not correct!~")


    for i in range(len(formats)):
        if type(blocks[i]).__name__ != formats[i]:  # LƯU Ý: split trả chuỗi nên so kiểu bằng tên không thực sự xác thực dữ liệu.
```

```
            print("!!![ERROR] Input format is not corrects")
    return(blocks)

def displayAll():  # In danh sách tên theo cột có khoảng trắng đầu trong CSV gốc.
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    print("""
==== DISPLAY ALL STUDENT =====""")
    students = df[" name"].values  # Lấy cột ' name' có dấu cách đầu theo CSV gốc PythoClass.
    stt = 1
    for student in students:
        print(f"{stt}. {student}")
        stt += 1

def studentSearch():  # Lọc theo mã sinh viên.
    print("""
==== SEARCH STUDENT =====""")
    inputData = inputHelper({
        "label":"Please enter Student ID:",
        "formats":["str"]
    })
    if inputData:
        inputData = inputData[0]
        df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
        result = df[df["student_id"] == inputData]
        if len(result)!=0:
            print(result)
        else:
            print("!!![ERROR] Student not found.")
        studentSearch()  # Gọi lại chính hàm tìm kiếm; lặp nhiều có thể tạo đệ quy quá sâu.
    else:
        return

def studentAverage():  # Tính trung bình ba môn cho một sinh viên.
    print("""
==== STUDENT RESULT ====""")
    inputData = inputHelper({
        "label":"Please enter Student ID:",
        "formats":["str"]
    })
    if inputData:
        inputData = inputData[0]
        df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
        result = df[(df["student_id"] == inputData)]
        if len(result)!=0:
            values = result.values
            print(f"""
Student name: {values[0][1]}
Python score: {values[0][3]}
Math score: {values[0][4]}
ML score: {values[0][5]}
Average score:{round((values[0][3] + values[0][4] + values[0][5])/3,2)}
Highest score:{max(values[0][3],values[0][4],values[0][5])}
Lowest score:{min(values[0][3],values[0][4],values[0][5])}
""")
        else:
            print("!!![ERROR] Student not found.")
        studentAverage()  # Gọi lại hàm tính điểm; lặp nhiều có thể tạo đệ quy quá sâu.
    else:
```

```
        return

def analyzeSubject():  # Thống kê theo từng cột môn học.
    print("""
==== SUBJECT PERFORMANCE ====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    data = {
        "Average":[df["Python"].mean(),df["Math"].mean(),df["ML"].mean()],
        "Total":[df["Python"].sum(),df["Math"].sum(),df["ML"].sum()],
        "Highest":[df["Python"].max(),df["Math"].max(),df["ML"].max()],
        "Lowest":[df["Python"].min(),df["Math"].min(),df["ML"].min()],
    }
    dfShow = pd.DataFrame(data,index=["Python","Math","ML"])
    print(dfShow)


def analyzeClass():  # groupby/merge tổng hợp kết quả theo lớp.
    print("""
===== CLASS PERFORMANCE =====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    studentCount =  df.groupby("class_name")["student_id"].count()  # Nhóm theo lớp và đếm mã sinh viên.
    pythonMean =  df.groupby("class_name")["Python"].mean()
    mathMean =  df.groupby("class_name")["Math"].mean()
    mlMean =  df.groupby("class_name")["ML"].mean()
    result = pd.merge(studentCount,pythonMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result = pd.merge(result,mathMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result = pd.merge(result,mlMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result["Overall Average"] = (result["Python"] + result["Math"] +  result["ML"])/3  # Trung bình chung từ trung bình ba môn.


    result.reset_index(inplace=True)
    result.rename(columns={"student_id":"Students"}, inplace=True)
    result.rename(columns={"class_name":"Class"}, inplace=True)

    result = result.round(2)

    print(result)

def top3():  # Chức năng 6: xếp hạng ba sinh viên có điểm trung bình cao nhất.
    print("""
===== TOP 3 STUDENTS =====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    df["Average"] =   (df["Python"] + df["Math"] +  df["ML"])/3
    dfSorted = df.sort_values(by="Average",ascending=False)
    dfSorted.drop(columns=["Python","Math","ML"], inplace=True)
    dfSorted.rename(columns={"class_name":"Class","student_id":"Student ID"}, inplace=True)
    dfSorted = dfSorted.round(2)

    print(dfSorted.head(3))  # Chỉ hiển thị ba hàng đứng đầu.

# MENU()
def menu():  # Bảy lựa chọn của main.py gốc, không phải sáu chức năng của đề Word.
    choice = 0
    while True:
        print("""
================================
 STUDENT PERFORMANCE ANALYSIS
--------------------------------------------------------
```

```
1. Display all students
2. Search for a student
3. Calculate student average
4. Analyze subject performance
5. Analyze class performance
6. Show top 3 students
7. Exit
        """)

        inputData =input("Please select (1-7): ")
        if inputData.isnumeric():
            choice = int(inputData)
            if choice == 1:
                displayAll()
            elif choice == 2:
                studentSearch()  # Gọi lại chính hàm tìm kiếm; lặp nhiều có thể tạo đệ quy quá sâu.
            elif choice == 3:
                studentAverage()  # Gọi lại hàm tính điểm; lặp nhiều có thể tạo đệ quy quá sâu.
            elif choice == 4:
                analyzeSubject()
            elif choice == 5:
                analyzeClass()
            elif choice == 6:
                top3()
            elif choice == 7:
                break
            else:
                print("!!![ERROR] Please enter from 1 to 7")
        else:
            print("!!![ERROR] Please enter from 1 to 7")

# goi ham
menu()
```

4. Giải thích nội dung và cách giải

Phần 3 chứa **một chương trình Python thống nhất**. Dưới đây là giải thích chi tiết cách từng nhóm hàm giải quyết yêu cầu, không lặp lại các đoạn code rời.

Tổng quan chương trình thực tế của `main.py`

Tệp này là **bài giải đang có trong repo PythoClass**, sử dụng `pandas` để đọc và phân tích `students.csv`. Chương trình in menu **7 lựa chọn**: hiển thị, tìm sinh viên, thống kê một sinh viên, thống kê theo môn, theo lớp, xếp hạng 3 sinh viên đầu và thoát. Trong khi đó, đề Word Lab 10 yêu cầu menu **6 lựa chọn**, có **thêm sinh viên** và **lưu file**. Đây là **hai phạm vi không hoàn toàn giống nhau**, nên không được coi `main.py` đã đáp ứng toàn bộ đề gốc.

Đọc dữ liệu và cách chương trình chạy

1. **`import pandas as pd`** nạp thư viện Pandas. `pd.read_csv("students.csv")` đọc một file bảng từ **thư mục chạy hiện tại**. Nếu file không có ở đó, lệnh đọc phát sinh `FileNotFoundError`.
2. **`inputHelper(data)`** hiển thị lời nhắc rồi dùng `split(";")` để tách nhiều giá trị. Một mục nhập trống trả về `None`; các hàm tìm kiếm và tính điểm dùng điều này để kết thúc.
3. **`displayAll()`** đọc bảng, lấy cột tên rồi in từng học viên bằng `for` và số thứ tự.
4. **`studentSearch()`** lọc bảng bằng biểu thức Boolean `df["student_id"] == inputData`; nếu lọc ra hàng thì in, nếu không thì báo không tìm thấy.

5. **`studentAverage()`** lấy điểm Python, Math, ML của một học viên, tính **trung bình = tổng ba điểm chia 3**, dùng `round(..., 2)` để hiển thị hai chữ số, `max(...)` / `min(...)` để lấy điểm cao/thấp nhất.
6. **`analyzeSubject()`** gọi `.mean()`, `.sum()`, `.max()`, `.min()` trên từng cột điểm; `pd.DataFrame(data, index=...)` trình bày các thống kê theo môn.
7. **`analyzeClass()`** gọi `groupby("class_name")` để chia theo lớp, `.count()` để đếm sinh viên, `.mean()` tính trung bình; `pd.merge(..., on="class_name", how="outer")` ghép bảng mà vẫn giữ các nhóm lớp, `reset_index(inplace=True)` chuyển chỉ số thành cột, `rename(columns=..., inplace=True)` đổi tên cột, `round(2)` làm tròn.
8. **`top3()`** thêm cột `Average`, gọi `sort_values(by="Average", ascending=False)` để xếp cao xuống thấp; `drop(columns=..., inplace=True)` bớt cột thừa và `head(3)` lấy ba hàng đầu.
9. **`menu()`** dùng `while True`, `if/elif` và `break` để điều hướng 7 lựa chọn.

Những điểm phải biết khi tự chạy và đối chiếu đề

Tên cột `" name"` có dấu cách: CSV gốc trong PythoClass có header `student_id, name,class_name,Python,Math,ML`, nghĩa là tên cột thực tế là `" name"` có khoảng trắng đầu. Bởi vậy `df[" name"]` trong `displayAll()` chạy với **CSV gốc**, nhưng sẽ lỗi `KeyError` nếu dùng dữ liệu mẫu trong đề Word đã chuẩn hóa thành `"name"` không có dấu cách. Khi thay CSV, cần nhất quán tên cột.

`inputHelper()` có hạn chế: `input().split(";")` luôn trả các chuỗi `str`. Cách kiểm tra `type(blocks[i]).__name__` không ép sang số thực/nguyên, và nhánh báo sai số trường không dừng trước vòng lặp: nếu thiếu trường có thể xảy ra `IndexError`. Nên kiểm tra số trường rồi `return` hoặc báo lỗi, sau đó chuyển kiểu thực sự bằng `int()/float()` có xử lý `ValueError` khi mở rộng chức năng.

Gọi đệ quy để hỏi lại: `studentSearch()` và `studentAverage()` gọi lại chính nó sau mỗi lượt tra cứu. Cách này có thể dùng nhanh nhưng gọi nhiều lần có nguy cơ `RecursionError`; vòng `while` là lựa chọn an toàn hơn.

Đối chiếu với đề Word: `main.py` **không có** thao tác thêm sinh viên, kiểm tra mã trùng khi thêm, lưu sinh viên ra CSV và kiểm tra điểm 0–100. Do đó, nếu bài thi yêu cầu đủ các chức năng của Word, vẫn phải bổ sung chúng. Phiên bản `labs/pythoclass/LAB-10-file-csv/SOLUTION.py` là **lời giải tham khảo khác** đã triển khai các yêu cầu đó; không nên nhầm nó với `main.py` của bạn.

Bảo toàn nguồn: Phần code in phía trên giữ nguyên **các dòng lệnh thực thi** trong `PythoClass/main.py`; chỉ bổ sung chú thích tiếng Việt cuối dòng để ôn tập. Những vấn đề được phân tích ở đây thay vì âm thầm sửa logic chương trình. Dùng `students.csv` gốc cùng thư mục chạy khi muốn kiểm tra đúng hành vi mã nguồn.

Đi theo một lượt chạy thực tế từ đầu tới cuối

Khởi động: chương trình bắt đầu khi gọi `menu()` ở cuối file. `while True` tiếp tục in 7 lựa chọn; `input(...)` luôn trả về chuỗi, `.isnumeric()` kiểm tra xem có thể chuyển sang `int` trước khi đối chiếu `choice`. Lựa chọn 7 dùng `break` để dừng menu.

Dữ liệu Pandas: `pd.read_csv("students.csv")` đọc CSV thành `DataFrame`. Mỗi lần chọn một chức năng, chương trình lại đọc file: cách này đơn giản nhưng có chi phí đọc lại, và phụ thuộc thư mục chạy hiện tại. `df[df["student_id"] == inputData]` tạo mặt nạ Boolean rồi lọc hàng; khác với `df["student_id"]` vốn chỉ chọn một cột.

Các cấp thống kê: Khi xem *một học viên*, `result.values[0]` lấy hàng đầu của bảng lọc; các chỉ số 3, 4, 5 lần lượt là điểm Python, Math, ML theo thứ tự cột của CSV gốc. Khi xem *một môn*, `.mean()`, `.sum()`, `.max()`, `.min()` tổng hợp toàn bộ sinh viên. Khi xem *một lớp*, `groupby("class_name")` tạo các nhóm; `pd.merge(..., on="class_name", how="outer")` ghép bảng đếm người và bảng điểm trung bình. Cuối cùng, xếp hạng top 3 cần tính `Average`, sắp giảm dần rồi `head(3)`.

Lỗi và giới hạn quan trọng: `" name"` (có khoảng trắng) khác `"name"`; file CSV dùng sai header sẽ gây `KeyError`. `split(";")` trả về các chuỗi dù người dùng gõ số, nên so sánh tên kiểu với `formats` chưa kiểm tra dữ liệu thực sự. Nếu nhập thiếu mục, cách kiểm tra hiện tại có thể dẫn đến `IndexError`. Hai hàm tìm kiếm và xem điểm gọi lại chính nó nên nếu thao tác quá nhiều có thể tăng độ sâu đệ quy.

So với yêu cầu giảng viên: code hiện tại **chưa thêm học viên, chưa lưu CSV, chưa chặn trùng mã khi thêm và chưa xác nhận điểm 0–100**. Những chức năng này cần viết bổ sung nếu bài thi yêu cầu đúng nguyên văn đề Word. Bản in chỉ phân tích và chú thích `main.py` thật, không biến nó thành một lời giải khác rồi ghi nhầm nguồn.

5. Giải thích các định nghĩa

Tra cứu các thuật ngữ và hàm trong `main.py` theo ví dụ đơn giản bên dưới.

CSV (bảng phân tách bằng dấu phẩy)

Là gì, dùng khi nào? File gồm nhiều hàng và cột. Mỗi hàng trong Lab 10 là một học viên, mỗi cột ghi mã, tên, lớp hoặc điểm.

Ví dụ: CSV gốc có tiêu đề `student_id, name,class_name,Python,Math,ML`; dấu cách trước `name` là một phần tên cột.

Pandas và DataFrame

Là gì, dùng khi nào? Pandas là thư viện xử lý bảng; DataFrame là dạng bảng có tên cột, giúp lọc và tính thống kê.

Ví dụ: `pd.read_csv("students.csv")` đọc CSV; `df["Math"]` lấy một cột điểm.

Boolean filtering (lọc theo điều kiện)

Là gì, dùng khi nào? So sánh từng giá trị của cột với điều kiện, tạo ra True/False, từ đó chỉ giữ các hàng đúng.

Ví dụ: `df[df["student_id"] == "S001"]` lấy hàng sinh viên có mã S001.

Average / highest / lowest (trung bình / cao / thấp)

Là gì, dùng khi nào? Trung bình bằng tổng ba điểm chia 3; `max` tìm số lớn nhất, `min` tìm số nhỏ nhất.

Ví dụ: Điểm 80,70,90: trung bình 80, cao nhất 90, thấp nhất 70.

`groupby()` (gom theo nhóm)

Là gì, dùng khi nào? Gom sinh viên theo lớp rồi tính thống kê cho từng lớp thay vì cho cả trường.

Ví dụ: `groupby("class_name")["Math"].mean()` cho điểm Math trung bình mỗi lớp.

`pd.merge()` (ghép bảng)

Là gì, dùng khi nào? Gộp các bảng thống kê theo một cột chung; `on` xác định cột nối, `how="outer"` giữ khóa xuất hiện ở ít nhất một bảng.

Ví dụ: Ghép bảng đếm sinh viên với bảng điểm trung bình cùng khóa lớp.

`sort_values()` và `head()` (xếp hạng)

Là gì, dùng khi nào? `sort_values(...,ascending=False)` đưa điểm cao lên đầu, `head(3)` lấy ba hàng đầu.

Ví dụ: Sau khi tạo cột Average, kết quả đầu là ba sinh viên có trung bình cao nhất.

`split()` và kiểm tra đầu vào

Là gì, dùng khi nào? `split(";")` tách chuỗi tại dấu chấm phẩy nhưng vẫn trả về chuỗi, chưa chuyển sang số.

Ví dụ: Chuỗi `S001;8.5` tạo `S001` và `8.5`; muốn tính toán phải chuyển phần điểm qua `float()`.

Recursion / loop (đệ quy / vòng lặp)

Là gì, dùng khi nào? Đệ quy là hàm tự gọi lại chính nó; vòng lặp làm lại một nhóm lệnh mà không tạo thêm lần gọi hàm.

Ví dụ: Tìm sinh viên liên tiếp rất nhiều lần bằng đệ quy có nguy cơ RecursionError.

KeyError và FileNotFoundError

Là gì, dùng khi nào? KeyError xảy ra khi tên cột/khóa không tồn tại; FileNotFoundError xảy ra khi thiếu tệp cần đọc.

Ví dụ: Code đọc `df[" name"]` sẽ lỗi nếu CSV chỉ có cột `name` không có dấu cách.

Pretest — Phân tích điểm, xác suất và Gradient Descent

1. Đề bài tiếng Anh

PRETEST

A teacher collects data from five students, including their study hours, attendance rate, and actual exam scores.

Student	Study Hours	Attendance (%)	Actual Score
Student A	2	60	50
Student B	3	65	55
Student C	4	75	65
Student D	5	80	70
Student E	6	95	90

Requirements

Task 1 – Student Performance Analysis and Prediction

1. Create a NumPy matrix student_data containing study hours, attendance, and actual scores from the given dataset.
2. Use the matrix to calculate the mean and standard deviation of actual scores.
3. A simple prediction model is defined as:

   predicted_score = 5 * study_hours + 0.2 * attendance + 10

   Use the matrix to calculate the predicted scores for all five students.

Task 2 – Probability

1. Calculate the probability that a randomly selected student has an actual score greater than or equal to 70.
2. Calculate the probability that a student has an actual score greater than or equal to 70, given that the student studies at least 4 hours.

Task 3 – Gradient Descent

A machine learning model uses the loss function:

Hàm mất mát: lấy `w` trừ 4 rồi bình phương. Ví dụ `w=2` thì loss = `(2−4) × (2−4) = 4`.

1. Calculate the loss when (w=2).
2. Calculate the gradient when (w=2).
3. Perform one Gradient Descent update using learning rate (α=0.1) and initial value (w=2).
4. Briefly explain why Gradient Descent updates the parameter (w).

2. Đề bài tiếng Việt

BÀI KIỂM TRA ĐẦU VÀO — PRETEST

Một giáo viên thu thập dữ liệu của **năm học sinh**, gồm số giờ học, tỷ lệ tham gia lớp học và điểm thi thực tế.

Học sinh	Số giờ học	Điểm danh (%)	Điểm thi thực tế
Học sinh A	2	60	50
Học sinh B	3	65	55
Học sinh C	4	75	65
Học sinh D	5	80	70
Học sinh E	6	95	90

Yêu cầu

Bài 1 — Phân tích và dự đoán kết quả học tập

1. Tạo một ma trận NumPy (thư viện tính toán mảng) tên là student_data chứa số giờ học, tỷ lệ điểm danh và điểm thi thực tế của năm học sinh theo bảng trên.
2. Sử dụng ma trận để tính **điểm trung bình** và **độ lệch chuẩn** của điểm thi thực tế.
3. Mô hình dự đoán đơn giản đã được định nghĩa:

   Điểm dự đoán = 5 × số giờ học + 0,2 × tỷ lệ điểm danh + 10.

   Sử dụng ma trận để tính điểm dự đoán cho cả năm học sinh.

Bài 2 — Xác suất

1. Tính xác suất một học sinh được chọn ngẫu nhiên có điểm thi thực tế **từ 70 trở lên**.
2. Tính xác suất học sinh có điểm thi thực tế **từ 70 trở lên**, với điều kiện học sinh đó học **ít nhất 4 giờ**.

Bài 3 — Gradient (độ dốc của hàm) Descent (thuật toán giảm độ dốc)

Một mô hình học máy sử dụng hàm mất mát:

Hàm mất mát: lấy `w` trừ 4 rồi bình phương. Ví dụ `w=2` thì loss = `(2−4) × (2−4) = 4`.

1. Tính giá trị hàm mất mát khi **w = 2**.
2. Tính gradient (đạo hàm theo w) khi **w = 2**.
3. Thực hiện **một lần cập nhật** Gradient Descent với tốc độ học **α = 0,1** và giá trị khởi tạo **w = 2**.
4. Giải thích ngắn gọn tại sao Gradient Descent cập nhật tham số w.

3. Code Python có chú thích

```
import numpy as np

STUDENT_NAMES = ["Student A", "Student B", "Student C", "Student D", "Student E"]

student_data = np.array([
    [2, 60, 50],
    [3, 65, 55],
    [4, 75, 65],
    [5, 80, 70],
    [6, 95, 90],
], dtype=float)


def task1_student_performance(data):  # Phân tích bảng 5 sinh viên và so sánh dự đoán với thực tế.
    if data.shape != (5, 3):
        raise ValueError("Expected the original 5-student x 3-column table.")

    study_hours = data[:, 0]
    attendance = data[:, 1]
    actual_scores = data[:, 2]

    mean_score = float(np.mean(actual_scores))  # Tính trung bình cộng các giá trị hợp lệ.

    population_sd = float(np.std(actual_scores, ddof=0))  # Tính độ lệch chuẩn để đo sự phân tán.
    sample_sd = float(np.std(actual_scores, ddof=1))  # Tính độ lệch chuẩn để đo sự phân tán.

    predicted_scores = 5 * study_hours + 0.2 * attendance + 10
    return mean_score, population_sd, sample_sd, predicted_scores


def task2_probabilities(data):  # Đếm số điều kiện đúng bằng Boolean và tính xác suất.
    scores = data[:, 2]
    hours = data[:, 0]

    at_least_70 = scores >= 70
    at_least_4_hours = hours >= 4
    both_conditions = at_least_70 & at_least_4_hours

    probability_70 = float(np.count_nonzero(at_least_70) / len(scores))  # Đếm hoặc cộng phần tử để tính tổng và xác suất.

    denominator = int(np.count_nonzero(at_least_4_hours))  # Đếm hoặc cộng phần tử để tính tổng và xác suất.
    if denominator == 0:
        raise ValueError("No students meet the given study-hours condition.")
    conditional_probability = float(
        np.count_nonzero(both_conditions) / denominator  # Đếm hoặc cộng phần tử để tính tổng và xác suất.
    )
    return probability_70, conditional_probability


def task3_gradient_descent(initial_w=2.0, learning_rate=0.1):  # Thực hiện đúng một bước cập nhật tham số w.
    loss_before = (initial_w - 4) ** 2

    gradient = 2 * (initial_w - 4)
```

```
    updated_w = initial_w - learning_rate * gradient

    loss_after = (updated_w - 4) ** 2
    return loss_before, gradient, updated_w, loss_after


def main():
    mean, pop_sd, sample_sd, predicted = task1_student_performance(student_data)
    print("TASK 1 — STUDENT PERFORMANCE")
    print("Actual-score mean:", f"{mean:.2f}")
    print("Population SD (NumPy default, ddof=0):", f"{pop_sd:.6f}")
    print("Sample SD (comparison only, ddof=1):", f"{sample_sd:.6f}")
    print("Student | actual score | predicted score")
    for name, actual, prediction in zip(
        STUDENT_NAMES, student_data[:, 2], predicted
    ):
        print(f"{name} | {actual:.0f} | {prediction:.0f}")

    p70, p70_given_4h = task2_probabilities(student_data)
    print("TASK 2 — PROBABILITY")
    print("P(actual score >= 70):", f"{p70:.6f}")
    print("P(actual score >= 70 | hours >= 4):", f"{p70_given_4h:.6f}")

    loss, derivative, new_w, new_loss = task3_gradient_descent()
    print("TASK 3 — GRADIENT DESCENT")
    print("L(2):", f"{loss:.2f}")
    print("Gradient at w=2:", f"{derivative:.2f}")
    print("Updated w after one step:", f"{new_w:.2f}")
    print("Loss after one update (extra check):", f"{new_loss:.2f}")

    if not (student_data.shape == (5, 3)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(mean, 66.0)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(pop_sd, np.sqrt(194.0))): raise ValueError("Kiểm tra không đạt")
    if not (np.allclose(predicted, [32, 38, 45, 51, 59])): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(p70, 2 / 5)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(p70_given_4h, 2 / 3)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(loss, 4.0)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(derivative, -4.0)): raise ValueError("Kiểm tra không đạt")
    if not (np.isclose(new_w, 2.4)): raise ValueError("Kiểm tra không đạt")
    if not (new_loss < loss): raise ValueError("Kiểm tra không đạt")


if __name__ == "__main__":
    main()
```

4. Giải thích nội dung và cách giải

Phần 3 chứa **một chương trình Python thống nhất**. Dưới đây là giải thích chi tiết cách từng nhóm hàm giải quyết yêu cầu, không lặp lại các đoạn code rời.

Câu 1 — Phân tích điểm và dự đoán

Mục tiêu: Dùng bảng của năm học sinh để tính trung bình, độ lệch chuẩn của **điểm thực tế**, sau đó dự đoán điểm bằng biểu thức được đề cung cấp. Không tự huấn luyện mô hình.

Vì sao làm theo thứ tự này? Cần chuyển bảng thành mảng số trước, lấy đúng cột điểm thi để thống kê, rồi lấy cột giờ học và điểm danh để dự đoán. Nếu trộn các cột, kết quả sẽ sai dù cú pháp Python đúng.

Quy trình giải:

1. Tạo mảng có năm hàng, ba cột. Mỗi hàng là một học sinh; các cột lần lượt là giờ học, tỷ lệ điểm danh và điểm thi. `dtype=float` yêu cầu lưu dạng số thực để thuận tiện tính toán.
2. Lấy cột điểm thực tế bằng `data[:, 2]`. Dấu `:` chọn tất cả hàng, số `2` chọn cột thứ ba vì Python đếm cột từ 0.
3. Dùng `np.mean(scores)` tính trung bình. Dùng `np.std(scores, ddof=0)` tính độ lệch chuẩn của toàn bộ nhóm năm học sinh. Đề không nêu mẫu hay tổng thể, nên cũng tính thêm `ddof=1` để người học so sánh trường hợp xem năm học sinh là mẫu.
4. Tính đồng thời điểm dự đoán theo hệ số đề cho; không đổi 60% thành 0.6, vì biểu thức của đề sử dụng trực tiếp giá trị điểm danh 60, 65, ...

Công thức viết bằng Python:

Giải thích hàm, toán tử và tham số: `np.array` tạo mảng, `dtype=float` chọn kiểu số thực. `np.mean` lấy trung bình. `np.std` đo độ phân tán; `ddof=0` chia phương sai cho số quan sát `n`, còn `ddof=1` chia cho `n-1`. Phép `*` nhân từng phần tử trong mảng NumPy, `+` cộng. `data[:, 0]` lấy giờ học, `data[:, 1]` lấy điểm danh, `data[:, 2]` lấy điểm thực tế.

Ví dụ và kết quả đối chiếu: Trung bình **66**, độ lệch chuẩn tổng thể khoảng **13.9284**, độ lệch chuẩn mẫu khoảng **15.5724**. Điểm dự đoán lần lượt **32, 38, 45, 51, 59**; học sinh A có `5 * 2 + 0.2 * 60 + 10` bằng **32**. Không nhầm điểm dự đoán với điểm thực tế.

Từ vựng: *Mean* (trung bình), *standard deviation* (độ lệch chuẩn), *prediction* (dự đoán).

Câu 2 — Xác suất và xác suất có điều kiện

Mục tiêu: Tìm xác suất đạt ít nhất 70 điểm khi chọn bất kỳ học sinh nào; sau đó tìm xác suất đạt ít nhất 70 điểm **nếu đã biết** học sinh đó học ít nhất 4 giờ.

Tại sao phải giải khác nhau? Câu đầu xét đủ **5 học sinh**, trong đó D và E đạt 70 điểm trở lên. Câu sau chỉ xét nhóm **C, D, E**, gồm **3 học sinh** học từ 4 giờ; trong nhóm đó D và E đạt ít nhất 70 điểm. Xác suất có điều kiện phải dùng đúng số người trong nhóm được biết trước làm mẫu số.

Quy trình giải:

1. So sánh mảng điểm với 70 để tạo các giá trị đúng/sai; dùng `np.count_nonzero` đếm số học sinh đạt.
2. Lấy số đạt (**2**) chia cho tổng số (**5**).
3. So sánh giờ học với 4 để chọn nhóm điều kiện; lấy `&` kết hợp hai điều kiện để đếm người vừa học đủ giờ vừa đạt điểm.
4. Lấy số người thỏa cả hai (**2**) chia cho số người học từ 4 giờ (**3**). Phải kiểm tra mẫu số khác 0.

Công thức viết bằng Python:

Giải thích hàm, toán tử và flag: `>=` là lớn hơn **hoặc bằng**, nên điểm đúng 70 vẫn được tính. `&` có nghĩa là *AND* (và) theo từng vị trí của **mảng Boolean** (mảng đúng/sai). `np.count_nonzero` đếm số phần tử đúng. `len` đếm tổng số phần tử, `/` là phép chia thực. Trong f-string (chuỗi định dạng) `:.0%` hiển thị phần trăm không có chữ số thập phân, `:.2%` hiển thị hai chữ số thập phân; đây là **định dạng in**, không sửa giá trị xác suất. `if count_4h == 0` chặn trường hợp chia cho 0.

Ví dụ và kết quả đối chiếu: `p_pass` bằng **0.4 (40%)**; `p_conditional` khoảng **0.6667 (66.67%)**. Không dùng lại mẫu số 5 cho xác suất có điều kiện.

Từ vựng: *Probability* (xác suất), *conditional probability* (xác suất có điều kiện), *Boolean* (đúng/sai).

Câu 3 — Một bước Gradient Descent

Mục tiêu: Tính *loss* (mức mất mát) tại `w = 2`, tính *gradient* (độ dốc/đạo hàm), cập nhật tham số `w` **một lần** với tốc độ học `0.1` và giải thích tại sao cần cập nhật.

Vì sao dùng gradient? Loss cho biết tham số hiện tại chưa phù hợp đến đâu. Gradient cho biết hướng tăng của loss; muốn giảm loss, ta điều chỉnh tham số theo **hướng ngược gradient**. Với `w = 2`, gradient âm nghĩa là cần tăng `w` để tiến gần giá trị tối ưu `4`.

Quy trình giải:

1. Viết biểu thức loss của đề bằng `(w - 4) ** 2`. Với `w=2`, giá trị là **4**.
2. Áp dụng quy tắc đạo hàm của bình phương, viết `gradient = 2 * (w - 4)`. Với `w=2`, gradient là **-4**.
3. Tính `updated_w = w - learning_rate * gradient`. Vì gradient âm, kết quả mới là **2.4**, tức `w` tăng lên.
4. Dùng biểu thức loss ban đầu để kiểm tra tại `updated_w`. Loss giảm còn **2.56**; không cần lặp nhiều lần vì đề yêu cầu đúng một bước.

Công thức viết bằng Python:

Giải thích toán tử, biến và tham số: `** 2` là lũy thừa bậc hai trong Python, **không dùng ^** (toán tử XOR). `*` là nhân và `-` là trừ. `learning_rate=0.1` là kích thước bước cập nhật: quá lớn có thể vượt qua điểm thấp nhất; quá nhỏ làm tiến triển chậm. `gradient` là đạo hàm theo `w`, không phải xác suất. `loss_after` chỉ là phép kiểm tra thêm; bài không yêu cầu `epochs` (số vòng học).

Ví dụ và kết quả đối chiếu: Loss ban đầu **4**, gradient **-4**, giá trị mới **2.4**, loss sau cập nhật **2.56**. Không trộn số liệu học sinh ở Câu 1 vào hàm loss riêng của Câu 3.

Từ vựng: *Gradient Descent* (thuật toán giảm độ dốc), *loss* (hàm mất mát), *learning rate* (tốc độ học), *parameter* (tham số).

Ba câu hỏi liên kết kiến thức như thế nào?

Câu 1 — Từ bảng dữ liệu đến thống kê: Ma trận `student_data` có **5 hàng × 3 cột**; mỗi hàng là một học viên, lần lượt gồm giờ học, điểm thực tế và điểm dự đoán. Cú pháp `data[:, 1]` lấy tất cả điểm thực tế, `data[:, 2]` lấy tất cả dự đoán. Trung bình điểm thực tế dùng `np.mean`; độ lệch chuẩn quần thể dùng `np.std(..., ddof=0)` và độ lệch chuẩn mẫu dùng `ddof=1`. Nếu muốn biết mô hình dự đoán sai bao nhiêu, hãy đối chiếu từng cặp điểm thật/dự đoán, không so toàn bộ trung bình rồi coi đó là sai số từng người.

Câu 2 — Boolean và xác suất: `scores >= 70` không cho ra một số 0/1 duy nhất, mà cho **mảng Boolean** cùng kích thước với `scores`. Mỗi phần tử đúng là `True`, sai là `False`. `np.count_nonzero(mask)` trả **số lượng phần tử đúng**; `np.mean(mask)` sẽ trả tỷ lệ đúng vì trong phép cộng, True tương đương 1, False tương đương 0. Xác suất `P(điểm ≥70)` bằng số người đạt điều kiện chia 5, không phải kết quả Boolean. Xác suất có điều kiện `P(A|B)` phải chia **số người thuộc B**, không chia tất cả 5 người; phải xử lý trường hợp nhóm B rỗng trước khi chia.

Câu 3 — Gradient Descent: Với `L(w)=(w-4)²`, đạo hàm là `2(w-4)`. Tại `w=2`, gradient bằng `-4`; dùng `learning_rate=0.1` thì `w_mới=2-0.1×(-4)=2.4`. Loss trước bằng 4 và sau bằng 2.56, nên bước này đi đúng chiều giảm loss. Đây là **một bước cập nhật**, không phải huấn luyện tới hội tụ.

Khi ôn thi: phân biệt độ lệch chuẩn với sai số chuẩn, số đếm với xác suất, `and` điều kiện với `&` trên mảng Boolean NumPy, và dấu của gradient khi trừ một số âm. Hầu hết lỗi sai đến từ chọn sai mẫu số hoặc đảo dấu cập nhật chứ không phải lệnh thư viện.

5. Giải thích các định nghĩa

Những khái niệm dễ nhầm trong ba câu Pretest được giải thích lại bằng dữ liệu nhỏ để tự tính nhẩm.

NumPy array (mảng số)

Là gì, dùng khi nào? Là một dãy hoặc bảng số có thể tính cả cột cùng lúc. Chỉ số hàng và cột Python bắt đầu từ 0.

Ví dụ: `data[:, 1]` lấy mọi hàng, cột thứ hai; bảng 5 học viên × 3 thuộc tính có shape (5,3).

Mean (trung bình cộng)

Là gì, dùng khi nào? Lấy tổng điểm chia số học viên để biết mức điểm chung.

Ví dụ: [60,70,80] có mean = (60+70+80)/3 = 70.

Standard deviation (độ lệch chuẩn)

Là gì, dùng khi nào? Đo điểm của mọi người phân tán quanh điểm trung bình bao nhiêu. Số càng lớn, điểm càng trải rộng.

Ví dụ: [70,70,70] có SD=0, còn [50,70,90] có SD lớn hơn.

ddof=0 và ddof=1

Là gì, dùng khi nào? Tính phương sai từ n số: ddof=0 chia n, ddof=1 chia n−1. ddof=1 thường dùng khi ước lượng phương sai tổng thể từ mẫu.

Ví dụ: [2,4,6] có tổng bình phương độ lệch là 8; phương sai ddof=0 là 8/3, ddof=1 là 8/2.

Boolean / True / False và `np.count_nonzero`

Là gì, dùng khi nào? Một điều kiện áp trên mảng tạo nhiều True/False. `count_nonzero` **đếm số True**, không trả một giá trị 0/1 duy nhất.

Ví dụ: [60,75,80] >= 70 cho [False,True,True], đếm được 2 người.

Probability (xác suất)

Là gì, dùng khi nào? Tỷ lệ trường hợp đạt điều kiện trên tổng số trường hợp; khi tất cả học viên được chọn như nhau, lấy số người đạt chia tổng số người.

Ví dụ: Trong 5 người, có 2 người đạt ít nhất 70: xác suất = 2/5 = 40%.

Conditional probability (xác suất có điều kiện)

Là gì, dùng khi nào? Đổi mẫu số thành số người thuộc nhóm B đã biết; không còn chia cho toàn bộ dữ liệu.

Ví dụ: Trong 3 người học ít nhất 4 giờ có 2 người đạt 70, xác suất đạt 70 khi học ≥4 giờ là 2/3.

Loss, gradient và learning rate

Là gì, dùng khi nào? Loss đo mức sai lệch; gradient chỉ chiều loss tăng theo tham số; learning rate quyết định độ dài bước điều chỉnh.

Ví dụ: Với loss=$(w-4)^2$, w=2, gradient=−4, bước 0,1 → w mới=2,4; loss giảm từ 4 còn 2,56.

`**2` và dấu ^ trong Python

Là gì, dùng khi nào? `**2` bình phương một giá trị, còn ^ là toán tử XOR bit, không phải lũy thừa.

Ví dụ: `(3-5)**2=4`; không viết `(3-5)^2` để tính bình phương.

Bài tập Toán 02 — Tập hợp, Jaccard và đồ thị sinh viên

1. Đề bài tiếng Anh

1. Analyze Student Interests: list all unique subjects, identify students interested in both Python and AI, and identify students interested in Python or Math.
2. Compare Two Students: select two valid and different students; find their subject intersection, union, and subjects studied only by the first student.
3. Calculate Jaccard Similarity: calculate the size of the intersection divided by the size of the union, and handle an empty union.
4. Find the Most Similar Pair: evaluate every unordered pair of different students once and find the pair with the highest Jaccard score.
5. Recommend Study Partners: use Jaccard threshold 0.5, exclude the student themselves, and list suitable partners.
6. Build a Simple Student Graph: create an undirected graph linking students when their Jaccard similarity is at least 0.5.

2. Đề bài tiếng Việt

1. Phân tích sở thích học tập: liệt kê các môn học khác nhau, tìm người học cả Python và AI, và tìm người học Python hoặc Math.
2. So sánh hai sinh viên: chọn hai học viên hợp lệ và khác nhau; tìm giao, hợp và các môn chỉ người thứ nhất học.
3. Tính độ tương đồng Jaccard: lấy số môn chung chia cho số môn thuộc hợp; xử lý trường hợp hợp rỗng.
4. Tìm cặp giống nhau nhất: xét mỗi cặp hai học viên một lần, tìm cặp có Jaccard lớn nhất.
5. Gợi ý bạn học: lấy ngưỡng Jaccard 0,5; không gợi ý chính người đang xét; liệt kê các bạn phù hợp.
6. Tạo đồ thị sinh viên đơn giản: nối hai người bằng cạnh vô hướng khi Jaccard từ 0,5 trở lên.

3. Code Python có chú thích

```
"""Lab Math 02 — set operations, Jaccard similarity and student graphs.

Reconstructed and improved from PythoClass/2. Math/labs2.ipynb.
Run: python SOLUTION_ANNOTATED.py
All example names are synthetic; no user input is required.
"""
from itertools import combinations  # Generate unordered pairs without repetition.

# Map each synthetic student to a SET of subjects: duplicates are impossible.
# The subject choices reproduce the patterns in the owner's six-row demo.
STUDENTS = {
    "Student A": {"Python", "AI", "Math"},
    "Student B": {"Python", "AI", "SQL"},
    "Student C": {"Python", "SQL"},
    "Student D": {"Java", "Web"},
    "Student E": {"Python", "AI", "Math"},
    "Student F": {"AI", "Math", "Python"},
}


def analyse_interests(students):  # Tổng hợp môn học chung và tìm theo điều kiện logic.
    """Task 1: compute unique subjects and the two requested filters."""
```

```
    # set().union(*iterable) joins every subject set into one unique collection.
    all_subjects = set().union(*students.values())
    # 'and' keeps people who study BOTH Python and AI.
    python_and_ai = [name for name, subjects in students.items()
                     if "Python" in subjects and "AI" in subjects]
    # 'or' keeps people who study AT LEAST ONE of Python and Math.
    python_or_math = [name for name, subjects in students.items()
                      if "Python" in subjects or "Math" in subjects]
    # Sorting gives reproducible output even though sets have no fixed order.
    return sorted(all_subjects), python_and_ai, python_or_math


def compare_students(students, first, second):
    """Task 2: inspect intersection, union and directional difference."""
    # Reject unknown identifiers before attempting dictionary indexing.
    if first not in students or second not in students:
        raise ValueError("Both student names must exist.")  # Báo lỗĩ nếú dữ liệu không đúng điều kiện.
    if first == second:
        raise ValueError("Choose two distinct students.")  # Báo lỗĩ nếú dữ liệu không đúng điều kiện.
    left, right = students[first], students[second]
    return {
        "shared": sorted(left & right),  # Subjects common to BOTH students.
        "combined": sorted(left | right),  # Subjects studied by EITHER student.
        "only_first": sorted(left - right),  # Subjects absent from second.
    }


def jaccard_similarity(left, right):  # Jaccard = sô´phần tửgiao chia sô´phần tửhợp.
    """Task 3: fraction of unique shared subjects among unique subjects."""
    both = left & right  # Set intersection.
    either = left | right  # Set union.
    # Two empty sets have no shared subjects here; choose 0 rather than /0.
    return len(both) / len(either) if either else 0.0


def most_similar_pair(students):  # combinations tránh xét lại cặp A-B dưới tên B-A.
    """Task 4: compare all unordered pairs, preserving first winning tie."""
    best_pair = None
    best_score = -1.0  # Lower than every legal Jaccard value [0, 1].
    for first, second in combinations(students, 2):  # Duyệt qua các phần tửhoặc cặp phần tử
        score = jaccard_similarity(students[first], students[second])
        if score > best_score:
            best_pair, best_score = (first, second), score
    return best_pair, best_score


def recommend_partners(students, student, threshold=0.5):
    """Task 5: return everyone other than student with similarity >= limit."""
    if student not in students:
        raise ValueError("Unknown student.")  # Báo lỗĩ nếú dữ liệu không đúng điều kiện.
    if not 0.0 <= threshold <= 1.0:
        raise ValueError("Threshold must be between 0 and 1.")  # Báo lỗĩ nếú dữ liệu không đúng điều kiện.
    matches = []
    for other, subjects in students.items():  # Duyệt qua các phần tửhoặc cặp phần tử
        if other == student:  # Never recommend a student to themselves.
            continue
        score = jaccard_similarity(students[student], subjects)
        if score >= threshold:  # Inclusive as in the original notebook.
```

```
            matches.append((other, score))
    # Rank by similarity, then alphabetically when scores are identical.
    return sorted(matches, key=lambda item: (-item[1], item[0]))


def build_graph(students, threshold=0.5):  # Tạo danh sách kề hai chiều cho cạnh vô hướng.
    """Task 6: symmetric adjacency list; isolated vertices remain visible."""
    if not 0.0 <= threshold <= 1.0:
        raise ValueError("Threshold must be between 0 and 1.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    graph = {name: [] for name in students}  # Every student becomes a node.
    for first, second in combinations(students, 2):  # Duyệt qua các phần tử hoặc cặp phần tử.
        score = jaccard_similarity(students[first], students[second])
        if score >= threshold:
            graph[first].append(second)  # Undirected edge in both directions.
            graph[second].append(first)
    return {name: sorted(neighbors) for name, neighbors in graph.items()}


def main():
    """Show each result and assert easy-to-check reference expectations."""
    subjects, both, either = analyse_interests(STUDENTS)
    print("Task 1 - All subjects:", ", ".join(subjects))
    print("Python AND AI:", ", ".join(both))
    print("Python OR Math:", ", ".join(either))

    compared = compare_students(STUDENTS, "Student A", "Student B")
    print("Task 2 - Common:", compared["shared"])
    print("Union:", compared["combined"])
    print("Only Student A:", compared["only_first"])

    score = jaccard_similarity(STUDENTS["Student A"], STUDENTS["Student E"])
    print("Task 3 - Jaccard(A, E):", f"{score:.2f}")
    if not (score == 1.0): raise ValueError("Kiểm tra dữ liệu không đạt")  # These two students have identical interest sets.

    pair, highest = most_similar_pair(STUDENTS)
    print("Task 4 - Best pair:", pair, "score", f"{highest:.2f}")
    if not (highest == 1.0): raise ValueError("Kiểm tra dữ liệu không đạt")  # Duplicate interest sets give perfect similarity.

    matches = recommend_partners(STUDENTS, "Student A", threshold=0.5)
    print("Task 5 - Partners:", matches)
    if not (all(s >= 0.5 for _, s in matches)): raise ValueError("Kiểm tra dữ liệu không đạt")

    graph = build_graph(STUDENTS, threshold=0.5)
    print("Task 6 - Student graph:")
    for name, neighbors in graph.items():  # Duyệt qua các phần tử hoặc cặp phần tử.
        print(f"  {name}: {', '.join(neighbors) if neighbors else 'No connections'}")
    if not (graph["Student D"] == []): raise ValueError("Kiểm tra dữ liệu không đạt")  # Java/Web shares none of these subjects.


if __name__ == "__main__":
    main()  # Do not run the sample automatically when imported.
```

4. Giải thích nội dung và cách giải

Phần 3 chứa **một chương trình Python thống nhất**. Dưới đây là giải thích chi tiết cách từng nhóm hàm giải quyết yêu cầu, không lặp lại các đoạn code rời.

Câu 1

Tập hợp (set) loại phần tử trùng; từ điển `STUDENTS` lưu mỗi người và tập các môn. `set().union(*students.values())` lấy tất cả môn xuất hiện. `and` yêu cầu hai môn cùng có mặt; `or` chỉ cần một trong hai.

Câu 2

Giao (`&`) lấy phần tử có ở cả hai tập, **hợp (`|`)** lấy phần tử có ít nhất một tập, **hiệu (`-`)** lấy các môn bên trái nhưng không ở bên phải. Trước khi tra cứu phải kiểm tra tên học viên hợp lệ, khác nhau.

Câu 3

`jaccard_similarity(left,right)` tính `len(left & right) / len(left | right)`. Ví dụ hai học viên có đúng ba môn giống nhau thì giao/hợp = 3/3 = 1.0 (100%). Với hợp rỗng, lời giải chọn 0 để không chia cho 0.

Câu 4

`combinations(students, 2)` sinh mỗi cặp khác nhau đúng một lần, không đếm lặp A–B và B–A. So Jaccard của từng cặp và giữ điểm cao nhất.

Câu 5

Ngưỡng `threshold=0.5` nghĩa là từ 50% tương đồng trở lên. Bỏ qua `other == student`, lấy các đối tượng đạt ngưỡng và sắp điểm giảm dần bằng `sorted(..., key=...)`.

Câu 6

Đồ thị lưu bằng **adjacency list** (danh sách kề): mỗi học viên là một nút, nếu hai học viên đủ giống nhau thì thêm tên mỗi người vào danh sách của người kia; vì vậy cạnh là **vô hướng**. Giữ cả các nút không có kết nối.

Đối chiếu nguồn và hướng dẫn chạy

Notebook gốc chứa các tiêu đề Task 1–6 và code mẫu. Lời giải đầy đủ là `labs/pythoclass/LAB-MATH-02-sets-jaccard/SOLUTION.py`, đã được cấu trúc lại từ notebook, sử dụng tên giả lập để không công khai dữ liệu cá nhân. Phần đề chi tiết bên dưới được diễn giải từ các task và code, không nhận là bản dịch nguyên văn một đề Word.

Khác biệt trình bày: Các điều kiện `assert` có trong `SOLUTION.py` được đổi thành `if not ...: raise ValueError(...)` để người mới hiểu cách kiểm tra rõ ràng hơn; không đổi mục tiêu tính toán.

Nguồn bài tập: `PythoClass/2. Math/labs2.ipynb` trên repo `nguyenletrian/PythoClass`.

Đọc các kết quả theo bản chất tập hợp

Giao, hợp và hiệu là ba câu hỏi khác nhau. Nếu học viên A quan tâm Python, AI, Math còn B quan tâm Python, AI, SQL, giao có hai môn (Python, AI), hợp có bốn môn (Python, AI, Math, SQL) và hiệu A−B chỉ còn Math. `&`, `|` và `-` trên `set` làm các phép này mà không tính trùng môn.

Jaccard so sánh theo tỷ lệ, không phải số môn chung tuyệt đối. Với A và B nói trên, Jaccard = 2/4 = 0.5; nếu hai bộ môn giống hệt nhau thì kết quả 1; nếu không có môn nào chung thì kết quả 0. Hàm phải kiểm tra hợp rỗng để không chia cho 0. Vì `set` không giữ thứ tự cố định, dùng `sorted` trước khi in để có kết quả ổn định.

Xét các cặp và lập đồ thị: `combinations(students, 2)` chỉ sinh các cặp không lặp; `most_similar_pair` so sánh điểm số để tìm cặp tương đồng nhất. Với ngưỡng 0.5, `recommend_partners` gợi ý người có mức tương đồng từ 50% trở lên nhưng bỏ qua bản thân; `build_graph` tạo hai cạnh kề tương ứng A→B, B→A cho mỗi mối quan hệ vì đồ thị là vô hướng.

Các lỗi dễ mắc: dùng `and` giữa hai tập thay vì xét điều kiện về môn; chia giao cho tổng độ dài hai tập (đếm phần trùng hai lần); tính A−B rồi coi bằng B−A; hoặc gợi ý chính người dùng làm bạn học của họ. Đọc đầu ra Task 1–6 để thấy mỗi lỗi thay đổi kết quả ra sao.

5. Giải thích các định nghĩa

Ghi nhớ bằng ví dụ tập môn học thay vì bắt đầu bằng ký hiệu toán.

Set (tập hợp)

Là gì và dùng khi nào? Lưu những phần tử không trùng lặp, rất phù hợp để liệt kê các môn mà từng người quan tâm.

Ví dụ: `{"Python","AI","Python"}` thực tế chỉ có hai môn.

Intersection (giao)

Là gì và dùng khi nào? Chọn các môn có mặt ở cả hai tập; Python dùng toán tử `&`.

Ví dụ: A={Python,AI,Math}, B={Python,SQL} thì giao chỉ có Python.

Union (hợp)

Là gì và dùng khi nào? Chọn mọi môn có ở ít nhất một trong hai tập, mỗi môn chỉ tính một lần; Python dùng `|`.

Ví dụ: A={Python,AI}, B={Python,Math} → hợp gồm Python, AI, Math.

Difference (hiệu)

Là gì và dùng khi nào? Chọn phần tử ở tập A nhưng không có trong B, nên A−B và B−A thường khác nhau.

Ví dụ: A={Python,AI,Math}, B={Python,SQL} thì A−B gồm AI, Math.

Jaccard similarity (độ tương đồng Jaccard)

Là gì và dùng khi nào? Lấy **số môn chung chia cho số môn phân biệt thuộc hợp**. Bằng 0 nếu không chung môn và bằng 1 nếu hai tập giống nhau.

Ví dụ: Hai người có 2 môn chung, hợp gồm 4 môn → Jaccard = 2/4 = 0,5.

`combinations()` (liệt kê cặp)

Là gì và dùng khi nào? Tạo mỗi cặp hai học viên đúng một lần, tránh xét A-B rồi xét lại B-A.

Ví dụ: Với ba người A,B,C, có ba cặp AB, AC, BC.

Threshold (ngưỡng)

Là gì và dùng khi nào? Mức so sánh để quyết định có gợi ý bạn học hay kết nối đồ thị không.

Ví dụ: Ngưỡng 0,5: cặp Jaccard 0,6 đạt, cặp 0,4 không đạt.

Graph, node, edge (đồ thị, nút, cạnh)

Là gì và dùng khi nào? Node là sinh viên, edge là kết nối khi tương đồng đạt ngưỡng. Đồ thị vô hướng cần có kết nối theo cả hai chiều.

Ví dụ: Nếu A và B đủ giống, danh sách kề của A có B và của B có A.

Bài tập Toán 03–04 — Ma trận, hệ tuyến tính và PCA

1. Đề bài tiếng Anh

1. Data Exploration and Matrix Creation: create an eight-row store matrix with Orders, Customers, Revenue, and Employees; display matrix shape, customer values, and average customers.
2. Finding Evaluation Weights Using a Linear System: solve the supplied 3 × 3 system A @ w = b for three store-evaluation weights.
3. Calculating and Ranking Store Performance: scale Orders, Customers and Revenue into comparable score units, compute S @ w, rank all eight stores and identify the best store.
4. Data Standardization and Dimensionality Reduction Using PCA: standardize the four store features, apply PCA to reduce to two components, and inspect projected shape and explained variance.

2. Đề bài tiếng Việt

1. Khám phá dữ liệu và tạo ma trận: lập bảng tám cửa hàng với Đơn hàng, Khách hàng, Doanh thu và Nhân viên; in kích thước ma trận, cột Khách hàng và số khách trung bình.
2. Tìm trọng số đánh giá bằng hệ tuyến tính: giải hệ 3 × 3 đã cho theo biểu thức A @ w = b để tính ba trọng số đánh giá.
3. Tính và xếp hạng hoạt động cửa hàng: quy đổi Đơn hàng, Khách hàng và Doanh thu thành các thang điểm tương ứng, tính S @ w, xếp tám cửa hàng và tìm cửa hàng cao điểm nhất.
4. Chuẩn hóa dữ liệu và giảm số chiều bằng PCA: chuẩn hóa bốn cột dữ liệu, đưa về hai thành phần chính, kiểm tra kích thước sau biến đổi và phần trăm phương sai được giữ lại.

3. Code Python có chú thích

```
"""Lab Math 03-04 — store evaluation using matrices, linear systems and PCA.

Adapted from 2. Math/labs3_4.ipynb. Run:
    python SOLUTION_ANNOTATED.py

Requires numpy and scikit-learn; prints results without opening a GUI.
"""
import numpy as np  # Arrays, descriptive statistics and matrix multiplication.
from sklearn.preprocessing import StandardScaler  # Scale each feature.
from sklearn.decomposition import PCA  # Reduce correlated features to 2 axes.

# Rows A-H represent stores. Each column has a specified real-world unit:
# Orders (count), Customers (count), Revenue (currency units), Employees (count).
STORE_NAMES = np.array(list("ABCDEFGH"))
STORE_DATA = np.array([
    [300, 400, 100000, 20],
    [400, 500, 140000, 25],
    [200, 200, 70000, 15],
    [500, 600, 180000, 30],
    [350, 450, 115000, 22],
    [450, 550, 160000, 28],
    [250, 300, 85000, 18],
    [600, 700, 210000, 35],
], dtype=float)  # Shape (8,4), with numeric values compatible with NumPy.
```

```
# A and b are supplied evaluation coefficients and observed benchmark scores.
EQUATION_MATRIX = np.array([[3, 4, 10], [4, 5, 14], [2, 2, 7]], dtype=float)
EQUATION_SCORES = np.array([135, 182, 86], dtype=float)


def task1_explore(stores=STORE_DATA):  # Tám cưả hàng, môĩ cưả hàng có bốn đặc trưng.
    """Inspect matrix dimensions, customer counts and mean customer traffic."""
    if stores.ndim != 2 or stores.shape[1] != 4:
        raise ValueError("Expect a matrix with four store features.")  # Báo lôĩ nêú dữ liệu không đúng điêù kiện.
    customers = stores[:, 1]  # ':' means all rows; index 1 is Customers.
    mean_customers = float(np.mean(customers))  # Mean over all eight stores.
    return stores.shape, customers, mean_customers


def task2_weights(A=EQUATION_MATRIX, b=EQUATION_SCORES):  # Giải A @ w = b đêʾtìm trọng sôʻ, không nhân từng phần tư.
    """Solve A @ w = b for weights of Orders, Customers and Revenue."""
    # np.linalg.solve avoids manually computing the matrix inverse.
    if A.shape != (3, 3) or b.shape != (3,):
        raise ValueError("Expected a 3x3 matrix and a three-element score vector.")  # Báo lôĩ nêú dữ liệu không đúng điêù kiện.
    if np.linalg.matrix_rank(A) < 3:  # A singular matrix has no unique answer.
        raise ValueError("The benchmark system has no unique solution.")  # Báo lôĩ nêú dữ liệu không đúng điêù kiện.
    weights = np.linalg.solve(A, b)
    if not (np.allclose(A @ weights, b)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Reconstruct the input equations.
    return weights


def task3_rank(stores=STORE_DATA, names=STORE_NAMES, weights=None):  # Đôỉ thang đo rôì tính điểm̉ và xêṕ hạng cưả hàng.
    """Build dimensionless components, combine weights, rank all stores."""
    if weights is None:
        weights = task2_weights()
    # Scale columns as in the original notebook before applying weights.
    components = np.column_stack((
        stores[:, 0] / 100,  # One score unit per 100 orders.
        stores[:, 1] / 100,  # One score unit per 100 customers.
        stores[:, 2] / 10000,  # One score unit per 10,000 revenue units.
    ))
    totals = components @ weights  # (8x3) @ (3,) -> eight store scores.
    ranked = sorted(zip(names.tolist(), totals.tolist()),
                    key=lambda pair: (-pair[1], pair[0]))
    winner_index = int(np.argmax(totals))  # Index of highest score.
    return components, totals, ranked, names[winner_index]


def task4_pca(stores=STORE_DATA):  # Chuẩn hóa trước khi giảm xuống hai thành phần chính.
    """Standardize four columns, then project stores onto two PCA axes."""
    # Different units (currency, people) must not bias distance by magnitude.
    scaler = StandardScaler()  # By default centers and scales each column.
    standardized = scaler.fit_transform(stores)
    # PCA finds directions containing the largest variation in scaled data.
    pca = PCA(n_components=2)  # We want a shape of (8 stores, 2 axes).
    projected = pca.fit_transform(standardized)
    retained = pca.explained_variance_ratio_  # Fraction on each retained axis.
    if not (projected.shape == (stores.shape[0], 2)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.allclose(standardized.mean(axis=0), 0, atol=1e-12)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.allclose(standardized.var(axis=0), 1, atol=1e-12)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.isfinite(projected).all()): raise ValueError("Kiểm tra dữ liệu không đạt")
    return standardized, projected, retained
```

```
def main():
    """Print intermediate results for every English task and check shapes."""
    shape, customers, mean_customers = task1_explore()
    print("Task 1 - Store matrix shape:", shape)
    print("Customers:", customers.astype(int).tolist())
    print("Mean customers:", round(mean_customers, 2))

    weights = task2_weights()
    print("Task 2 - Weights:", np.round(weights, 6).tolist())

    components, scores, ranking, best = task3_rank(weights=weights)
    print("Task 3 - Component matrix shape:", components.shape)
    print("Scores:", np.round(scores, 2).tolist())
    print("Ranking:", [(name, round(score, 2)) for name, score in ranking])
    print("Best store:", best)

    standardized, projection, ratios = task4_pca()
    print("Task 4 - Standardized shape:", standardized.shape)
    print("PCA projected shape:", projection.shape)
    print("Explained variance fractions:", np.round(ratios, 4).tolist())
    print("Total variance retained:", f"{ratios.sum() * 100:.2f}%")
    if not (len(ranking) == len(STORE_NAMES)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (projection.shape == (8, 2)): raise ValueError("Kiểm tra dữ liệu không đạt")


if __name__ == "__main__":
    main()  # Execute this demonstration only when launched as a script.
```

4. Giải thích nội dung và cách giải

Phần 3 chứa **một chương trình Python thống nhất**. Dưới đây là giải thích chi tiết cách từng nhóm hàm giải quyết yêu cầu, không lặp lại các đoạn code rời.

Câu 1

`np.array` tạo ma trận có tám hàng và bốn cột nên `shape` là `(8, 4)`. `stores[:, 1]` có nghĩa là lấy **mọi hàng**, riêng cột chỉ số 1 (Customers); `np.mean` tính trung bình tám số khách.

Câu 2

`np.linalg.solve(A, b)` giải **hệ phương trình** `A @ w = b`, không phải nhân từng phần tử. Ma trận 3 × 3 chỉ có nghiệm duy nhất nếu đủ hạng; `np.linalg.matrix_rank(A)` kiểm tra điều này.

Câu 3

`np.column_stack` ghép ba cột đã chia thang 100, 100, 10000; `components @ weights` nhân ma trận 8 × 3 với vector 3 phần tử tạo tám điểm. `np.argmax` cho vị trí điểm lớn nhất; `sorted` sắp xếp cửa hàng.

Câu 4

Doanh thu tính bằng tiền lớn hơn hẳn số nhân viên, nên cần `StandardScaler()` để đưa các đặc trưng về thang đo có trung bình gần 0 và phương sai gần 1. `PCA(n_components=2)` biến dữ liệu tám hàng bốn cột thành tám hàng hai cột; `explained_variance_ratio_` cho biết tỷ lệ phương sai mỗi thành phần giữ lại. PCA không bảo đảm dự đoán hoàn hảo.

Đối chiếu nguồn và hướng dẫn chạy

Notebook PythoClass gốc có bốn tiêu đề đề bài tiếng Anh và mã minh họa. Lời giải đầy đủ bên dưới lấy từ `labs/pythoclass/LAB-MATH-03-04-matrix-pca/SOLUTION.py`, bản tái cấu trúc từ notebook, vẫn sử dụng dữ liệu 8 cửa hàng và hệ phương trình gốc.

Khác biệt trình bày: Các điều kiện `assert` có trong `SOLUTION.py` được đổi thành `if not ...: raise ValueError(...)` để người mới hiểu cách kiểm tra rõ ràng hơn; không đổi mục tiêu tính toán.

Nguồn bài tập: `PythoClass/2. Math/labs3_4.ipynb` trên repo `nguyenletrian/PythoClass`.

Thứ tự xử lý và lý do làm từng bước

1. Dữ liệu gốc là ma trận 8 × 4. Mỗi hàng mô tả một cửa hàng A–H, các cột là đơn hàng, khách hàng, doanh thu và số nhân viên. `stores[:, 1]` lấy cột khách hàng theo chỉ số 1 (chỉ số Python bắt đầu từ 0); `np.mean` lấy số khách trung bình. `.shape` giúp đối chiếu kích thước trước khi nhân ma trận hoặc PCA.

2. Tìm trọng số bằng hệ tuyến tính. `EQUATION_MATRIX` có 3 hàng và 3 cột, `EQUATION_SCORES` có 3 giá trị. `np.linalg.solve(A, b)` tìm vector `w` sao cho `A @ w = b`; toán tử `@` là tích ma trận, khác `*` là nhân từng phần tử trong NumPy. `matrix_rank(A)` kiểm tra ma trận có đủ hạng để nghiệm duy nhất tồn tại. Kiểm lại bằng `np.allclose(A @ w, b)` bởi số thực có sai số làm tròn.

3. Tính điểm cửa hàng: ba cột đơn hàng, khách hàng, doanh thu có thang rất khác nhau nên code chia lần lượt cho 100, 100 và 10.000 để tạo ma trận `components` kích thước 8 × 3. Nhân với trọng số `w` sẽ cho tám điểm số. `np.argmax` tìm vị trí cao nhất; `sorted(..., key=...)` xếp hạng từ cao xuống thấp.

4. PCA dùng bốn cột gốc. `StandardScaler().fit_transform(stores)` đưa mỗi cột về thang đo so sánh được (trung bình gần 0, phương sai gần 1). `PCA(n_components=2)` chọn hai hướng có phương sai lớn, cho ma trận kết quả 8 × 2. `explained_variance_ratio_` là tỷ lệ phương sai mà từng hướng giữ lại, và tổng của chúng cho mức thông tin biến thiên được giữ. PCA không phải thuật toán bảo đảm dự đoán đúng hay phương pháp tự chọn cửa hàng tốt nhất.

Lỗi nên tránh: lấy `S * w` thay vì `S @ w`, quên chuẩn hóa trước PCA khi đơn vị khác nhau, hoặc nhầm `explained_variance_ratio_` là xác suất dự đoán đúng.

5. Giải thích các định nghĩa

Các thuật ngữ ma trận và PCA được diễn giải bằng tình huống tám cửa hàng trước khi cần đến tên toán học.

Matrix (ma trận)

Là gì và dùng khi nào? Bảng gồm hàng và cột, để lưu nhiều đặc trưng của nhiều cửa hàng trong cùng một cấu trúc.

Ví dụ: 8 cửa hàng, 4 đặc trưng mỗi cửa hàng → ma trận 8 hàng × 4 cột.

Shape (kích thước mảng)

Là gì và dùng khi nào? Cho biết số hàng và số cột; giúp kiểm tra hai ma trận có thể nhân hay không.

Ví dụ: Ma trận trên có `shape = (8,4)`.

`@` — phép nhân ma trận

Là gì và dùng khi nào? Kết hợp từng hàng của đối tượng bên trái với các giá trị phù hợp bên phải; khác `*`, là phép nhân từng phần tử của NumPy.

Ví dụ: Ma trận 8×3 nhân vector có 3 trọng số tạo 8 điểm, mỗi điểm thuộc một cửa hàng.

Linear system (hệ phương trình tuyến tính)

Là gì và dùng khi nào? Nhiều phương trình có chung các ẩn cần tìm. `np.linalg.solve(A,b)` tìm các trọng số để A nhân trọng số bằng b.

Ví dụ: Một hệ 3 phương trình có thể xác định 3 trọng số nếu ma trận A đủ hạng.

Weight (trọng số)

Là gì và dùng khi nào? Hệ số quyết định mức đóng góp của một tiêu chí khi tính điểm tổng.

Ví dụ: Điểm = 2 × chỉ số đơn hàng + 1 × chỉ số khách hàng: trọng số lần lượt là 2 và 1.

Scaling và standardization (đổi thang và chuẩn hóa)

Là gì và dùng khi nào? Đổi thang chia số liệu theo một đơn vị quy ước. Chuẩn hóa thường lấy số đo trừ trung bình rồi chia độ lệch chuẩn, giúp các cột có thang tương đương.

Ví dụ: Doanh thu 100.000 chia 10.000 thành 10 điểm là đổi thang; `StandardScaler` dùng trung bình và độ lệch chuẩn của cả cột.

PCA (phân tích thành phần chính)

Là gì và dùng khi nào? Tìm các hướng có mức biến thiên lớn trong dữ liệu, biểu diễn thông tin bằng ít cột hơn. PCA không phải thuật toán phân loại chính xác/sai.

Ví dụ: `PCA(n_components=2)` biến bảng 8×4 thành 8×2; số cửa hàng giữ nguyên.

Explained variance ratio (tỷ lệ phương sai giữ lại)

Là gì và dùng khi nào? Cho biết mỗi hướng PCA giữ bao nhiêu phần biến thiên ban đầu. Cộng các tỷ lệ của các hướng đã chọn để được tổng phần biến thiên giữ lại.

Ví dụ: Nếu PC1 giữ 70%, PC2 giữ 20% thì cả hai giữ 90% phương sai, **không phải dự đoán chính xác 90%**.

Eigenvalue (trị riêng)

Là gì và dùng khi nào? Trong PCA, trị riêng biểu diễn độ lớn phương sai theo một hướng riêng; trị riêng cao thường ứng với hướng giữ nhiều biến thiên.

Ví dụ: Nếu các hướng có trị riêng 9 và 1, hướng 9 giải thích nhiều biến thiên hơn.